

UNIP

UNIVERSIDADE PAULISTA

Práticas de Programação Orientada a Objetos



???. ??

UNIP

UNIVERSIDADE PAULISTA

Práticas de Programação Orientada a Objetos

Autor: Prof. Ricardo da Costa Veras

Colaboradores: Profa. Vanessa dos Santos Lessa

Profa. Larissa Rodrigues Damiani

Professor conteudista: Ricardo da Costa Veras

Mestre em Engenharia da Informação pela Universidade Federal do ABC (UFABC) – 2018; especialista em Orientação a Objetos pela Faculdade de Informática e Administração Paulista (Fiap) – 2004 e graduado em Engenharia Elétrica com ênfase em Eletrônica pela Escola de Engenharia Mauá (EEM) – 1994. Desde 2009 é professor da Universidade Paulista (UNIP) e ministra aulas para os cursos de Ciência da Computação, Sistemas de Informação e Engenharia Básica. Possui experiência na área de Tecnologia da Informação desde 1995, tendo trabalhado com consultoria e prestação de serviços de Análise e Desenvolvimento de Sistemas em diversas empresas.

Dados Internacionais de Catalogação na Publicação (CIP)

V476a

Veras, Ricardo da Costa.

Práticas de Programação Orientada a Objetos / Ricardo da Costa Veras. – São Paulo: Editora Sol, 2026.

112 p., il.

Nota: este volume está publicado nos Cadernos de Estudos e Pesquisas da UNIP, Série Didática, ISSN 1517-9230.

1. Layout. 2. Evento. 3. Relatório. I. Título.

CDU 681.3.062

?????? – 26

Prof. João Carlos Di Genio
Fundador

Profa. Sandra Rejane Gomes Miessa
Reitora

Profa. Dra. Marília Ancona Lopez
Vice-Reitora de Graduação

Profa. Dra. Marina Ancona Lopez Soligo
Vice-Reitora de Pós-Graduação e Pesquisa

Profa. Dra. Claudia Meucci Andreatini
Vice-Reitora de Administração e Finanças

Profa. M. Maria Cecília Migliaccio
Vice-Reitora de Extensão

Prof. Fábio Romeu de Carvalho
Vice-Reitor de Planejamento

Prof. Marcus Vinícius Mathias
Vice-Reitor das Unidades Universitárias

Profa. Silvia Renata Gomes Miessa
Vice-Reitora de Recursos Humanos e de Pessoal

Profa. Laura Ancona Lee
Vice-Reitora de Relações Internacionais

Profa. Melânia Dalla Torre
Vice-Reitora de Assuntos da Comunidade Universitária

UNIP EaD

Profa. Elisabete Brihy
Profa. M. Isabel Cristina Satie Yoshida Tonetto

Material Didático

Comissão editorial:

Profa. Dra. Christiane Mazur Doi
Profa. Dra. Ronilda Ribeiro

Apoio:

Profa. Cláudia Regina Baptista
Profa. M. Deise Alcantara Carreiro
Profa. Ana Paula Tôrres de Novaes Menezes

Projeto gráfico:

Prof. Alexandre Ponzetto

Revisão:

Kleber Souza
Auriana Malaquias

Sumário

Práticas de Programação Orientada a Objetos

APRESENTAÇÃO	7
INTRODUÇÃO	8

Unidade I

1 GUI	11
1.1 O AWT	12
1.1.1 Componentes principais do AWT	13
1.2 O Swing	15
1.2.1 Componentes principais do Swing	15
1.2.2 Criando a primeira tela	17
2 APROFUNDAMENTO EM COMPONENTES COMO LAYOUT, MENUS E TABELAS	19
2.1 Layouts	19
2.2 O layout nulo	20
2.2.1 Sequência para criação de telas com layout nulo	22
2.3 O Border Layout	22
2.4 O Grid Layout	23
2.5 O Card Layout	24
2.6 Trabalhando com menus	26
2.6.1 Inserindo os elementos de menus	27
2.7 Trabalhando com tabelas	29
2.7.1 Elementos e características da construção de uma tabela	30
3 EVENTOS	33
3.1 Utilizando-se dos eventos (controlando-os)	35
3.1.1 Passos (na programação) para uso dos eventos	36
3.2 Os eventos de ação	38
3.3 Os eventos de janela	40
4 JDBC	42
4.1 Os conectores/drivers	43
4.2 Conectando a um banco de dados do MySQL	45
4.3 Lendo informações do BD	47
4.4 Alterando informações no BD	50

Unidade II

5 DESIGN PATTERNS (PADRÕES DE PROJETOS)	58
5.1 DTO/VO	59

5.2 MVC.....	60
5.3 Data Access Object (DAO)	62
6 O HIBERNATE	62
6.1 Persistindo dados com o Hibernate	66

Unidade III

7 INTRODUÇÃO A APLICAÇÕES WEB – JSP E JSTL.....	73
7.1 Servidores de aplicação	73
7.2 Um pouco de HyperText Markup Language (HTML)	74
7.3 Salvando uma página como JSP	77
7.3.1 JSP e seus elementos.....	77
7.3.2 Fazendo funcionar uma JSP.....	80
7.4 O JSP Standard Tag Library (JSTL)	81
7.4.1 JSTL – core "c:catch": tratando erros.....	89
8 RELATÓRIOS: GERANDO RELATÓRIOS COM O MS EXCEL	92

APRESENTAÇÃO

Prezada(o) aluna(o),

Por meio do presente livro-texto, colocaremos em prática o que vimos na disciplina de **Linguagem de Programação Orientada a Objetos**, porém introduziremos alguns novos conceitos importantes e direcionados a certas estruturas de programação, assim como para a construção de sistemas, conteúdos esses a serem aplicados na fase de desenvolvimento.

Também vamos ver e utilizar efetivamente elementos e conceitos de **orientação a objetos (OO)** já estudados anteriormente, como:

- Classes e objetos.
- Métodos e atributos.
- Encapsulamento.
- Herança.
- Método construtor.
- Implementação de interfaces.
- Polimorfismo de classes e de métodos.
- Tratamento de exceções.

Nosso objetivo será aplicar esses conceitos na construção de programas para desktop (inicialmente), além de abordarmos como gerar sistemas mais organizados, seguindo regras e convenções (e padrões) criadas para este fim, e utilizarmos tais conteúdos na criação de sistemas para web.

Assim, serão apresentados alguns conceitos que, apesar de serem ainda básicos, já possibilitarão a geração de sistemas completos, com acesso a dados externos (em banco de dados) e manipulação eficaz das informações.

Aproveite bem este curso e garanta o seu sucesso.

Bons estudos!

INTRODUÇÃO

Quando estudávamos a **teoria inicial de orientação a objetos (OO)**, muitas vezes não entendíamos ao certo o porquê de precisarmos aprender todos aqueles conceitos, já que uma linguagem Java trabalha basicamente com três elementos: classes, métodos e atributos.

A ideia principal é que com eles construiremos sistemas mais seguros, confiáveis, eficazes, e que também nos ajudarão na utilização de alguns frameworks, o que permitirá uma compreensão mais rápida e eficaz de tudo o que envolve a construção dos sistemas com que se trabalha profissionalmente. Também temos que lembrar que um sistema não se constrói sozinho, e não se menciona aqui que ele poderia fazê-lo, mas que um sistema não é gerado por apenas um desenvolvedor, e sim por uma equipe na qual cada um trabalha com seu elemento (conhecimento) de domínio e todos estão voltados ao mesmo fim.

Portanto, é importante que a equipe seja comunicativa e saiba dividir seus conhecimentos, independentemente de eles serem técnicos ou sobre o negócio com que se está trabalhando. Somente assim um sistema será criado no melhor tempo e sob as melhores condições. O trabalho com acesso a banco de dados envolve uma cooperação mútua entre pelo menos três grupos de tecnologia da informação (TI) de uma empresa: os analistas, que procuram compreender e transformar em linguagem técnica as necessidades do cliente, de acordo com as plataformas de trabalho disponíveis na corporação; os desenvolvedores que, a partir das ideias dos analistas, elaboram e criam sistemas concretos, gerando a melhor solução no melhor tempo possível; o DataBase Administrator (DBA) que cuida e administra os bancos de dados a fim de que seja possível acessá-los da forma mais rápida e direta possível.

Portanto, é importante que o espírito de equipe esteja sempre presente, já que na maioria das vezes trabalha-se sob a pressão do tempo ou das necessidades do cliente.

Veremos a construção de sistemas que permitem a interação com o usuário através de telas equivalentes a formulários, assim como a persistência dos dados em banco de dados. Mostraremos também as informações solicitadas pelo cliente em forma de relatórios (a serem gerados em Excel), como organizar nosso sistema a fim de torná-lo eficaz e mais fácil de receber manutenção, e como fazê-lo funcionar via web.

Aplicações desktop e aplicações web

Ao criar aplicações nas quais se pretende que haja interação com o usuário, o desenvolvedor utilizará, em seu código, recursos que possibilitam essa interação, além de pensar a lógica correta para aplicar esses recursos seguindo as regras definidas pelo cliente, prevendo sempre que o usuário talvez não possua (necessariamente) conhecimentos sobre informática (ou tecnologia da informação), mas apenas sobre a sua própria área de negócio.

É importante ainda, antes de codificar um sistema, saber se a interação do usuário com o sistema deverá acontecer via browser (se funcionará via web), ou via sistemas próprios que precisam rodar na máquina do usuário.

A aplicação desktop é aquela construída para funcionar no próprio micro do usuário, utilizando-se dos recursos de interação disponíveis a partir do sistema operacional da máquina. Felizmente, a linguagem Java possui a característica de ser multiplataforma, permitindo que, ao se construir um sistema em uma plataforma, ele funcione em qualquer outro micro que possua a máquina virtual do Java instalada (a JVM). A aplicação desktop é acionada e manipulada totalmente a partir de um browser no micro do usuário. Neste caso, os recursos visuais e de interação utilizados têm como base a linguagem HTML e devem seguir as regras da W3C, a fim de garantir que ela funcione na maior parte dos browsers (ao menos naqueles homologados pela W3C).

Os sistemas construídos para rodar via desktop, muitas vezes, não precisam necessariamente de um servidor para funcionar. Já os sistemas construídos com Java, para funcionarem via browser, necessitam estar em um servidor de aplicações com serviço http, responsável pelas solicitações de acesso dos usuários.

A ideia deste curso é mostrar como gerar telas, com os diversos componentes disponíveis, que permitem a interação com o usuário (como os campos de texto, os labels, os botões etc.).

Servidores de aplicação

Em geral, um servidor de aplicação Java é (corporativamente) uma máquina (um servidor) que tem instalado um software servidor que permite a vários usuários acessarem serviços (programas web) gerados em uma linguagem Web-Dinâmica (a JSP, por exemplo), de forma que o serviço fica disponível a qualquer momento para vários usuários simultaneamente. No mercado existem alguns servidores de aplicação (gratuitos e/ou proprietários), como o WebSphere (da IBM), JBoss (da Red Hat JBoss Middleware), o GlassFish Server, entre outros.

Neste curso, abordaremos a utilização de aplicações web que utilizam o servidor HTTP e aplicações Apache Tomcat, um software que pode ser baixado gratuitamente e instalado em seu micro, transformando-o em um servidor web de aplicações Java.

Sobre as fases de um desenvolvimento de software

Ao trabalharmos sob determinado sistema, devemos compreender que seu processo de criação possui fases até a sua implantação. De forma geral, segundo o que chamamos de Processo Unificado da Rational (RUP), as fases de desenvolvimento de sistemas são divididas basicamente em quatro etapas:

- **Fase de concepção:** aqui o cliente apresenta as suas necessidades e é onde há um levantamento de tudo o que o sistema necessitará realizar como solução àquelas demandas.
- **Fase de elaboração:** nela será complementada a fase de concepção, porém com uma visão mais técnica, verificando-se as disponibilidades do ambiente de desenvolvimento e de servidor da empresa.

- **Fase de construção:** aqui a equipe técnica trabalhará na construção (codificação) do sistema propriamente dito, elaborando seus códigos e disponibilizando os acessos necessários às informações.
- **Fase de transição:** nela o sistema criado será colocado em testes para o usuário final, sendo depois posto em funcionamento, algo que chamamos de ambiente de produção (implantação do sistema).

Compreende-se, portanto, que este não é um processo a ser realizado de forma individual, e por isso precisamos aprender a trabalhar em equipe, organizando a construção de nosso código e lembrando que ele poderá ser complementado (implementado) por outros profissionais, e que tais indivíduos deverão ser capazes de compreender a estrutura e a lógica utilizada em sua construção. Portanto, estudaremos alguns padrões de desenvolvimento, ou de estruturas de construção de softwares (os design patterns), que permitem essa organização, assim como a compreensão geral do sistema (por parte da equipe de desenvolvimento do sistema).

Sobre a organização deste livro-texto

Este livro-texto está dividido em três unidades, cujo conteúdo possui a seguinte estruturação:

- **Unidade I:** serão apresentados os componentes básicos de programação desktop em Java (AWT e Swing) na criação de telas de formulários que permitem a interação do sistema com o usuário. Também aprenderemos a trabalhar com os eventos mais comuns de interação e a acessar dados externos provindos de um banco de dados (no caso, o MySQL).
- **Unidade II:** serão explicados os conceitos básicos sobre alguns dos padrões de desenvolvimento (design patterns), o porquê, e como são utilizados na construção de sistemas, assim como utilizar um framework de acesso a dados de um banco de dados: o Hibernate.
- **Unidade III:** será mostrado como podemos extrair relatórios a partir das informações existentes em um banco de dados, e iniciaremos o estudo da criação de programas para web, aprendendo seu funcionamento básico, e alguns recursos importantes para a sua funcionalidade.

Unidade I

1 GUI

Ao pensarmos na criação de sistemas desktop, ou seja, elaboração de programas que funcionarão diretamente no micro pessoal do usuário (e não em um servidor web a ser acessado via browser), este programa também deverá possibilitar a interação do usuário com o sistema, prevendo que ele vai inserir informações, ou solicitar aquelas que o sistema possa fornecer.

Para essa interação, supõe-se que o sistema seja capaz de funcionar a partir de eventos, que são as reações do sistema às ações do usuário sobre os componentes disponíveis na tela (quando o indivíduo clica sobre um botão, ou escreve um texto em um campo texto, ou quando seleciona itens de uma lista etc.). Cada uma dessas ações é considerada um evento que deve ser compreendido (ou traduzido) pelo sistema.

Para isso, deve-se gerar uma interface gráfica com o usuário ou graphical user interface (GUI), que é um ambiente gráfico sobre o qual o usuário consegue interagir com o sistema.

Nas primeiras versões do Java, as interfaces gráficas eram programadas com base nos componentes do Abstract Window Toolkit (AWT), um conjunto de ferramentas e componentes de janela disponíveis ao desenvolvedor para que ele pudesse (e ainda possa) criar os ambientes gráficos de interação.

Com a evolução dos computadores, evoluíram também os componentes de interação, de forma que surgiu um novo conjunto de componentes com a biblioteca Swing, que proporcionou relativamente uma maior liberdade de disposição dos componentes na tela, uma aparência mais agradável em relação às telas de interação, com recursos estendidos, e muitas outras facilidades e melhorias.

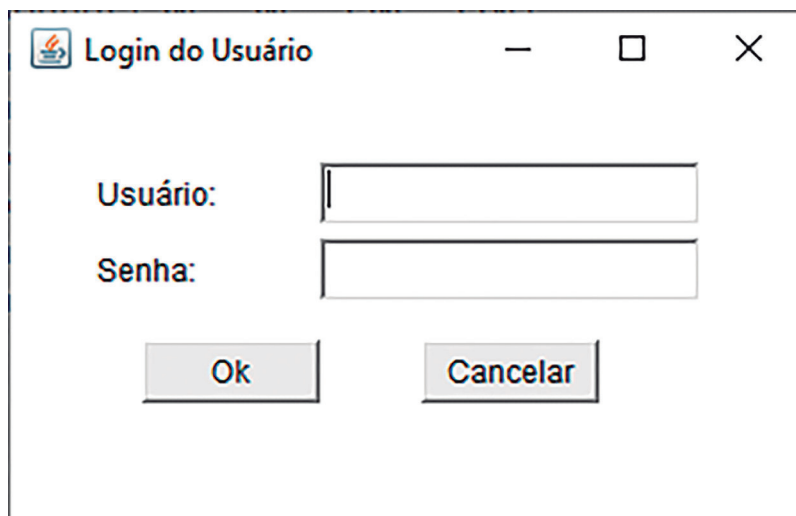


Figura 1 – Tela de login construída a partir da biblioteca AWT

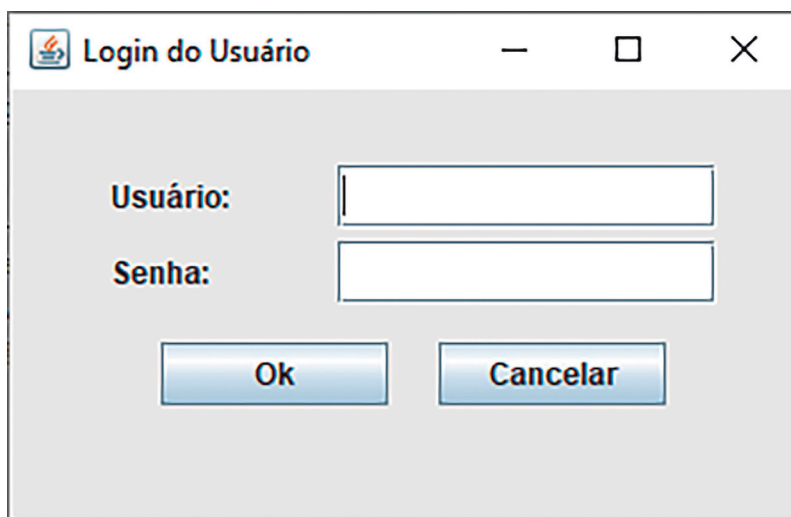


Figura 2 – Tela de login construída a partir da biblioteca Swing

Pode-se observar nas figuras anteriores que a tela que utiliza os componentes da biblioteca AWT lembra versões mais antigas do Windows, enquanto a tela gerada com os componentes da biblioteca Swing tem uma aparência mais atual.

Mesmo com a evolução das telas e dos componentes, alguns itens do AWT ainda são necessários na construção de telas interativas (por exemplo, os componentes que definem alguns layouts comuns de janela ou classes que tratam de eventos básicos).

Importante

Este curso foi idealizado supondo-se que a(o) aluna(o) irá utilizar-se dos seguintes softwares:

- Eclipse® como IDE para a programação relacionada ao Java;
- MySQL® como banco de dados (BD) relacional;
- Apache Tomcat® como servidor HTTP e de aplicações.

A utilização de outros softwares, com finalidades equivalentes às descritas anteriormente, pode ser realizada, porém como não está prevista, não será comentada ou explicada.

1.1 O AWT

O AWT é um conjunto (uma biblioteca) de ferramentas utilizado para a construção de janelas interativas.

Quando a linguagem Java foi lançada, uma das características mais marcantes dessa linguagem era o fato de ela ser multiplataforma (o que ainda é), o que significa que um programa gerado em uma plataforma Windows deve rodar em outra, como o MacOS ou o Linux, de forma a não ter diferença de

funcionamento, porém cada um com a sua interface (aparência) característica da plataforma. Na verdade, o AWT se utilizava dos componentes GUI nativos de cada plataforma para gerar as telas definidas na programação do sistema.

Essa biblioteca de componentes e ferramentas existe no pacote `java.awt` desde a versão 1.0 da linguagem.

1.1.1 Componentes principais do AWT

Apresentam-se aqui alguns dos principais componentes do AWT. Com eles, é possível construir a maioria dos formulários de interação básica, a partir de alguns componentes, como:

Frame (ou janela)

Este componente (o frame) representa uma janela com seus elementos básicos (bordas, espaço interno, barra de títulos, além dos três botões padrão no canto superior direito – minimização, maximização e fechamento da janela). Ele é considerado um container cuja característica é poder receber internamente outros componentes.

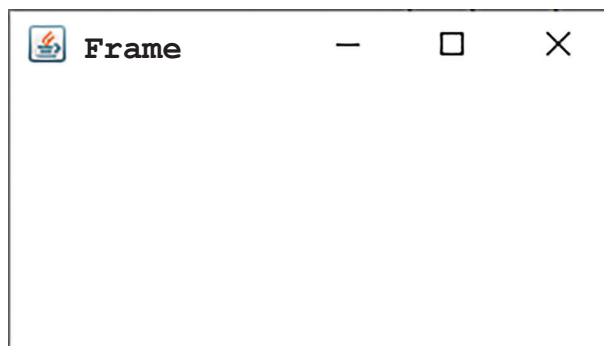


Figura 3 – Frame (vazio) gerado a partir da biblioteca AWT

Panel (ou painel)

O panel (painel) é um componente que serve como uma janela interna, porém sem a barra de títulos ou os botões padrão de manipulação. Ele também é considerado um container, pois recebe outros componentes em seu interior.



Figura 4 – Panel gerado a partir da biblioteca AWT



Observação

Não é possível abrir na tela (deixar visível) diretamente apenas um panel (sem que ele esteja inserido em frame). Apenas é possível visualizá-lo a partir de (dentro de) um frame, o que significa que sua visibilidade dependerá também da visibilidade do frame ao qual ele está inserido, direta ou indiretamente.

Label (ou rótulo)

O label é todo texto que precede um campo para explicar e definir qual significado do conteúdo deve ser nele inserido. Em geral, qualquer texto no interior de uma janela-formulário em que não é possível permitir alteração de seu valor diretamente na tela do usuário (através do teclado) pode ser considerado um label (ou seja, criado a partir de um label).

TextField (ou campo de texto simples)

O textfield é um campo de texto simples, mais conhecido como caixa de texto, que aceita apenas texto linear (no qual não é possível gerar mais de uma linha de texto).

Button (ou botão)

O button é um botão de formulário que, quando acionado (por exemplo, ao clicar sobre ele com o mouse), inicia a execução de algum método (determinado evento).

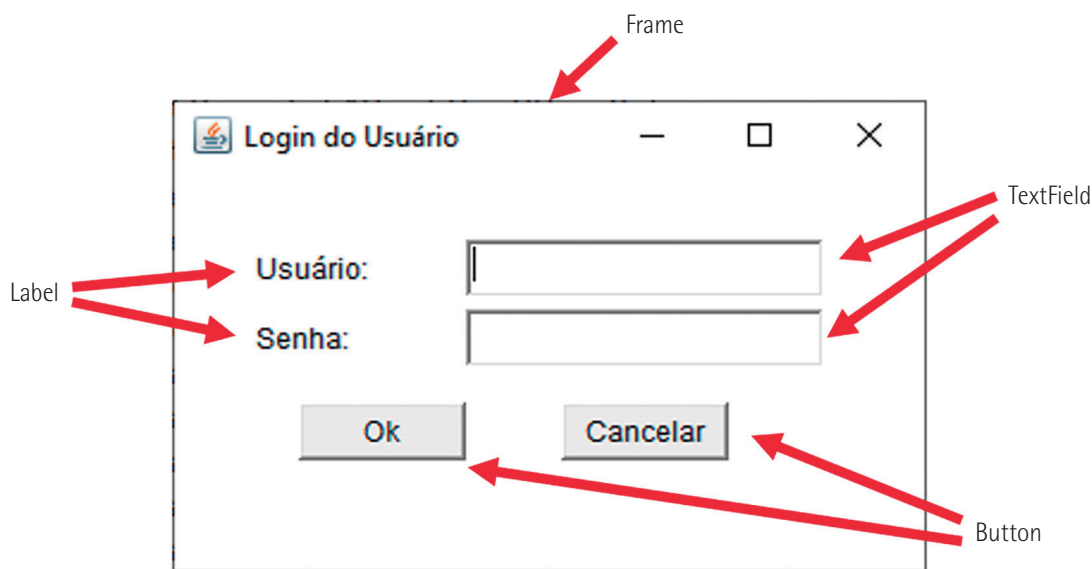


Figura 5 – Possíveis componentes gerados a partir da biblioteca AWT

1.2 O Swing

A biblioteca AWT utiliza componentes nativos do sistema operacional para construir suas telas. Isto fazia com que em cada plataforma as telas geradas tivessem diferenças, às vezes significativas, que prejudicavam o seu funcionamento ou sua visualização.

Com a evolução da tecnologia digital, a partir do lançamento da linguagem Java, novas funcionalidades foram adicionadas à biblioteca disponibilizada, de acordo com as necessidades dos sistemas que eram desenvolvidas.

Com o tempo, os desenvolvedores Java criaram então uma biblioteca de componentes específicos (o Swing), que não usava mais componentes gráficos nativos ao sistema operacional, passando a conter seus próprios componentes.

Essa evolução no visual dos componentes fez com que a tela vista em uma plataforma fosse a mesma em qualquer plataforma, homogeneizando os sistemas, deixando-os com igual aparência em todo micro que possuísse instalada uma máquina virtual Java (JVM). Além disso, utilizar a biblioteca Swing tornou mais rápida a renderização das telas (mecanismo de exibição de elementos gráficos) pelo compilador.



Saiba mais

Para aprender mais sobre componentes GUI, leia o capítulo 11 da obra a seguir:

DEITEL, P.; DEITEL, H. *Java: como programar*. São Paulo: Pearson Education do Brasil, 2017.

1.2.1 Componentes principais do Swing

Para uma parte dos componentes básicos, a diferença da sintaxe dos nomes das classes dos componentes do Swing em relação às das equivalentes do AWT é a presença da letra "J" antecedendo o seu nome, por exemplo:

- JFrame do Swing (equivalente ao frame do AWT);
- JPanel do Swing (equivalente ao panel do AWT);
- JLabel do Swing (equivalente ao label do AWT);
- JTextField do Swing (equivalente ao textfield do AWT);
- JButton do Swing (equivalente ao button do AWT).

Vale lembrar que nem toda classe de componente do Swing possui a característica de levar o mesmo nome do componente do AWT antecedido pela letra "J", como os componentes de seleção a seguir:

- JComboBox do Swing (equivalente ao choice do AWT);
- JCheckBox do Swing (equivalente ao checkbox do AWT) no qual a diferença está também na letra b da palavra "Box";
- JRadioButton do Swing (equivalente ao checkbox do AWT quando utilizado com o CheckboxGroup da mesma biblioteca).

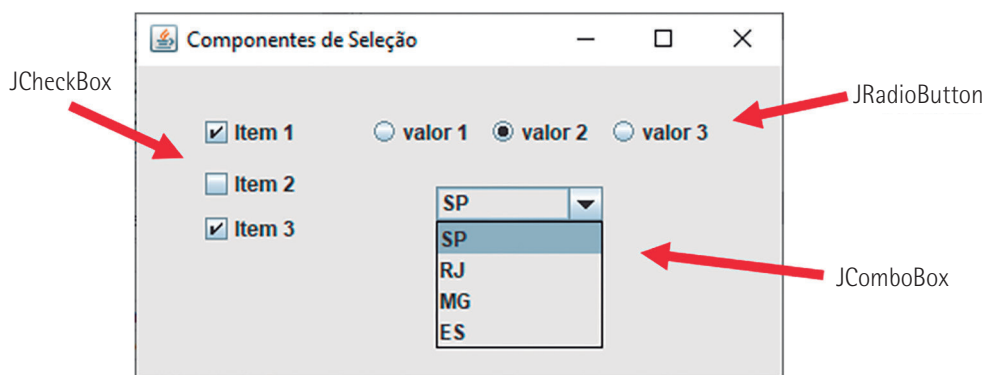


Figura 6 – Componentes de seleção gerados a partir da biblioteca Swing

Deve-se considerar também que a biblioteca Swing é mais completa, havendo componentes que não existem diretamente na lista da biblioteca AWT, tais como:

- JPasswordField: para campos de senha, ocultando o texto nela escrito, utilizando símbolos (normalmente um ponto • ou um asterisco *);
- JEditorPane e JTextPane: campos de textos que podem ser estilizados (ou formatados) separadamente, assim como mostram textos descritos na linguagem HTML;
- Entre outros recursos e/ou componentes.



Lembrete

Vimos que todo componente é uma classe Java, de forma que toda classe pode conter atributos (que definem as suas características e o local onde serão guardadas as informações dos objetos), métodos (que definem as ações que podem ou não ser efetivamente acionadas a partir do objeto em memória) e os métodos construtores (que estabelecem como as instâncias podem ser criadas na memória).

1.2.2 Criando a primeira tela

Um conceito essencial que devemos compreender é a ideia de que, com a herança, uma subclasse passa a ser a superclasse: se uma classe `Aluno` herda a classe `Pessoa`, podemos dizer que `Aluno` é `Pessoa`; se uma classe `Fusca` herda a classe `Carro`, então `Fusca` é `Carro`.

Assim, para que uma classe `Tela` (que será nossa primeira janela) se torne efetivamente um frame (uma janela), basta que ela herde a classe equivalente da biblioteca AWT (`Frame`) ou Swing (`JFrame`), e este será sempre o nosso primeiro passo na criação de uma janela.

```
import javax.swing.JFrame;

public class Tela extends JFrame {

}
```

A classe `Tela` do exemplo anterior já pode ser considerada efetivamente uma janela (window), já que ela herda a classe `JFrame` por meio do método `setVisible(...)` herdado da classe `JFrame`, ela já poderá ser visualizada.

O exemplo a seguir é o de um programa que, quando acionado, gera uma janela de 400 pixels de comprimento por 300 pixels de altura, distanciado a 200 pixels da lateral esquerda da tela do monitor, e a 100 pixels da parte superior da tela do monitor.

```
import javax.swing.JFrame;
public class Tela extends JFrame {
    public static void main(String[] args) {
        Tela tp = new Tela();
    }
    public Tela() {
        this.montarTela();
    }
    public void montarTela() {
        this.setBounds(200, 100, 400, 300);
        this.setVisible(true);
    }
}
```

De acordo com o exemplo citado, ao acionarmos a opção de menu "Run As – Java Application" do Eclipse para o seu programa, que possui o método "main", o compilador vai instanciar a classe `Tela`, de modo a rodar seu método construtor. Este, por sua vez, vai acionar o método "montarTela()" que possui duas ações:

- posicionar e dimensionar a janela (com o método "setBounds(...)");
- deixar a tela visível (com o método "setVisible(...)").

Método `setBounds(...)`

Este método é utilizado para posicionar e dimensionar um componente na tela (ou em um container, o que será explicado posteriormente).

Sua sintaxe é:

```
componente.setBounds(posX, posY, tamHor, tamVer);
```



Figura 7 – Representação de um JFrame dimensionado

onde:

- **posX**: distância horizontal (em pixels) do canto superior esquerdo da margem do componente até a margem esquerda do container ao qual está inserido (no caso do frame da figura anterior, a referência é a própria tela do computador).
- **posY**: distância vertical (em pixels) do canto superior esquerdo da margem do componente até a margem superior do container ao qual está inserido (no caso do frame da figura anterior, a referência é a própria tela do computador).
- **tamHor**: comprimento (tamanho) horizontal (em pixels) do componente.
- **tamVer**: comprimento (tamanho) vertical (em pixels) do componente.

Este único método (`setBounds(posX, posY, tamHor, tamVer)`) pode substituir os dois outros que também poderiam ter sido utilizados:

- método `setLocation(posX, posY)`;
- método `setSize(tamHor, tamVer)`.

onde o método `setLocation(...)` apenas posiciona o componente na tela (ou em seu container), e o método `setSize(...)` somente define as dimensões do componente.

Método `setVisible(...)`

Este método torna visível ou não, dependendo do valor inserido como parâmetro (que pode ser **true**, ou **false**), um componente na tela (ou em um container, elemento que será explicado posteriormente). Em seu parâmetro, colocamos o valor **true** se queremos que o componente esteja visível, e **false** se desejamos que ele não fique visível.

Para o frame, o valor default, ou seja, aquele que é atribuído automaticamente a este método (`setVisible(...)`), quando não o fazemos explicitamente, é o valor **false**, e por isso temos que colocar o comando (chamando este método) com o parâmetro **true** para o frame. Já para os componentes internos do frame, o valor default da característica de visibilidade é **true**, e, portanto, não é necessário redefini-lo ao longo do código.

Sua sintaxe é:

```
componente.setVisible(<valor_logico>);
```

onde o valor lógico pode ser **true** (verdadeiro) ou **false** (falso).



Observação

Normalmente inserimos o comando (`this.setVisible(true)`) ao final do método que monta a tela após configurarmos e posicionarmos todos os componentes que aparecerão nela, e ao longo dos códigos do método construtor da classe.

2 APROFUNDAMENTO EM COMPONENTES COMO LAYOUT, MENUS E TABELAS

2.1 Layouts

O container é um componente que pode receber internamente outros (inclusive outros containers). O layout define como os componentes internos poderão estar dispostos em um container. A maioria dos layouts utilizados faz parte da biblioteca AWT.

Para trabalharmos com a criação de telas e formulários, o Java possui uma série de layouts específicos, com características que definem como devemos trabalhar (disponibilizar) os seus componentes internos.

Tanto o frame (ou o `JFrame`) quanto o panel (ou o `JPanel`) possuem um layout padrão, que para o frame é o Border Layout, e para o panel é o Flow Layout. Cada um desses layouts tem características próprias que serão descritas a seguir.

2.2 O layout nulo

Veremos agora que os layouts seguem padrões e configurações específicas que obrigam a disposição de componentes segundo as regras de cada um deles.

Para que possamos inserir componentes em qualquer posição da tela, e com qualquer tamanho, devemos definir o layout como nulo (**null**) da seguinte forma:

```
componente.setLayout (null);
```

Utilizando essa definição, é possível posicionar cada um dos componentes com o método `setBounds(posX, posY, tamHor, tamVer)`, que apenas indicará onde, e com qual tamanho, ele estará localizado na janela (tendo a própria janela como referência para as posições).

Exemplo de aplicação

Como exemplo, vamos criar a seguinte tela (janela) de senha:

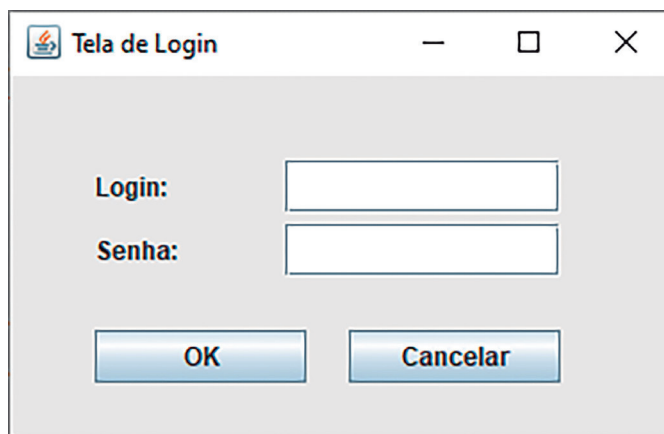


Figura 8 – Representação da tela de login

A qual pode ser gerada a partir do seguinte programa:

```
// Criação da Classe TelaLogin (Janela de Login)
import javax.swing.*;
// Lembrando que para nossa Classe "ser" uma Janela, ela tem que
// ...herdar um Frame (ou um JFrame)
public class TelaLogin extends JFrame {
    // Declarando Componentes como Atributos da Classe
    JLabel l1 = new JLabel("Login:");
    JTextField tf1 = new JTextField("");
    JLabel l2 = new JLabel("Senha:");
    JPasswordField tf2 = new JPasswordField ();
    JButton b1 = new JButton("OK");
    JButton b2 = new JButton("Cancelar");
    // O método main apenas instancia a Classe
    public static void main(String[] args) {
        TelaLogin t1 = new TelaLogin();
    }
}
```

```
}  
// O método construtor "constrói a Tela"  
public TelaLogin() {  
    this.setBounds(200, 100, 330, 210);  
    // Para aceitar componentes em localizações diversas:  
    this.setLayout(null);  
    this.setTitle("Tela de Login");  
    // Dimensionando cada um dos Componentes  
    l1.setBounds(40, 40, 80, 25);  
    l2.setBounds(40, 70, 80, 25);  
    tf1.setBounds(130, 40, 130, 25);  
    tf2.setBounds(130, 70, 130, 25);  
    b1.setBounds(40, 120, 100, 25);  
    b2.setBounds(160, 120, 100, 25);  
    // Adicionando os componentes à Janela (Tela)  
    this.add(l1);  
    this.add(l2);  
    this.add(tf1);  
    this.add(tf2);  
    this.add(b1);  
    this.add(b2);  
    // Por último, deixamos a tela visível  
    this.setVisible(true);  
}  
}
```



Lembrete

O termo `this` utilizado no exemplo prévio indica o uso do elemento (seja um atributo ou um método) da própria classe, de forma que, apesar dos métodos acionados não existirem diretamente na própria classe, eles constam na classe herdada (JFrame).

Observações:

- A tela gerada pelo programa mencionado não nenhuma tem funcionalidade, já que este será um assunto a ser tratado em outro item posteriormente.
- Tanto na instância do label quanto na do button, pode-se inserir como parâmetro o texto que ele conterá:

```
JLabel lab = new JLabel("texto do label");
```

```
JButton bot = new JButton("texto do botão");
```

- A classe `JPasswordField` representa um campo de texto simples, porém com caracteres escondidos, sendo utilizado para a entrada de senhas de usuários.

2.2.1 Sequência para criação de telas com layout nulo

Consta adiante uma sequência de passos para a criação de uma janela com layout nulo:

- Criar a classe que vai representar a janela.
- Fazer com que essa classe herde a classe Frame (se for seguir a biblioteca AWT) ou JFrame (se for seguir a biblioteca Swing).
- Colocar todos os componentes que farão parte da janela, como atributos dessa classe (principalmente os campos de valores textuais ou de seleção).
- Elaborar um método de construção da janela e de seus componentes (pode ser o próprio método construtor da classe, ou um outro que será chamado por ele) e que deverá:
 - Definir a localização e o tamanho da janela.
 - Estabelecer o layout da janela (como nulo).
 - Determinar a localização e o tamanho de cada um de seus componentes.
 - Adicionar cada um dos componentes à janela (senão eles não aparecem).
 - Deixar a janela visível.

2.3 O Border Layout

O Border Layout é o layout padrão do Frame (AWT), ou do JFrame (Swing). Esse layout divide a janela em 5 partes: norte, sul, leste, oeste e central, segundo a figura a seguir:

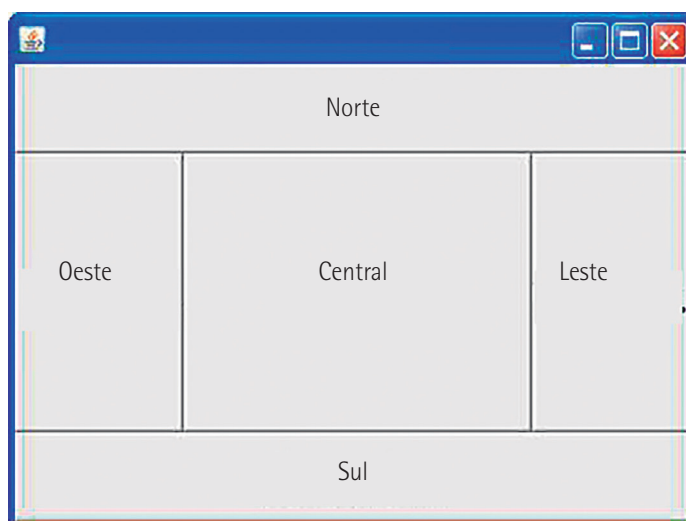


Figura 9 – Representação de um Border Layout

Em um Border Layout, não é necessário utilizar todas as partes das divisões nele existentes, de forma que se pode usar apenas algumas delas.

Em cada parte do layout, somente é possível adicionar um (e apenas um) componente. No entanto, o componente adicionado pode ser um container (Por exemplo: um Panel ou JPanel) dentro do qual dará para inserir vários outros.

Para adicionar um componente, precisamos utilizar o método `add(...)`, definindo em qual parte do Border Layout ele deverá ser inserido. Caso não se defina a parte onde o componente será adicionado, o compilador automaticamente o colocará na parte central (center).

Assim, para adicionar um componente, faz-se:

```
container.add(componente, BorderLayout.PARTE);
```

Por exemplo:

```
this.add(botOk, BorderLayout.SOUTH);
```

(o comando anterior adicionará o objeto botOK – botão OK – na parte sul do Border Layout do frame na própria janela)

2.4 O Grid Layout

O Grid Layout é um layout que divide a tela (seja um frame ou um panel) em partes de igual tamanho (ou dimensão), dispostos em linhas e colunas.



Figura 10 – Representação de um Grid Layout de 3 linhas por 4 colunas

Em cada parte do layout, somente é possível adicionar um (e apenas um) componente (que poderá ser um container, o qual pode conter vários outros componentes). Para adicionar um componente nas suas partes, existe uma sequência de adição, fazendo-se:

```
container.add(componente);
```

Por exemplo:

```
this.add(botOk);
```

Nesta linha de código (prévia), queremos adicionar um botão (que já deve ter tido sua configuração definida em linhas de código anteriores) à própria tela (**this**), quando a classe em questão é (através de herança) um frame ou panel.

A sequência de adição de componentes para um GridLayout deve ser: da esquerda para a direita, de cima para baixo (como na sequência numérica da figura de exemplo deste layout, mostrada previamente).

2.5 O Card Layout

O Card Layout é um layout que permite gerar várias telas de formulários, relatórios etc., mostrando a cada momento apenas uma das telas (aquela que vai ser utilizada). Ele viabiliza a troca de telas, de forma a permitir gerar um sistema completo, com suas várias telas de acesso para persistência de dados.

Nesse tipo de layout, um painel com layout Card Layout recebe vários outros painéis com layouts diversos.

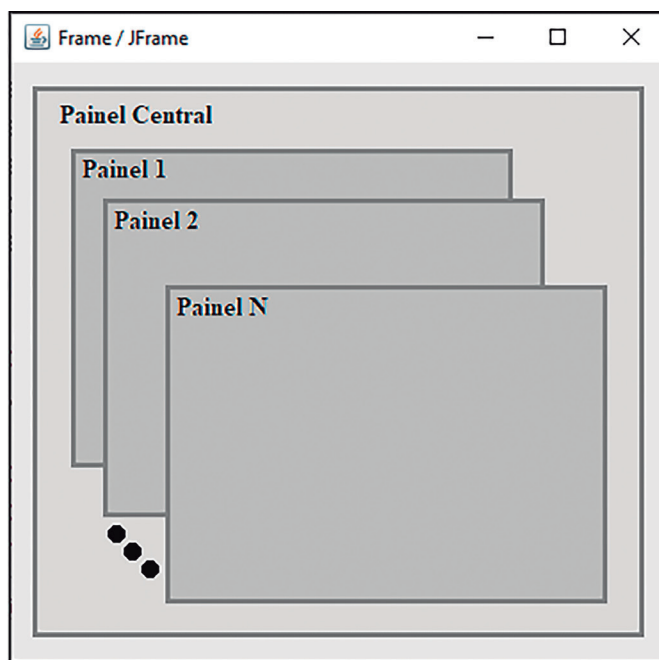


Figura 11 – Representação de um Card Layout

Para utilizar esse layout, geralmente inserimos (adicionamos) na parte central do frame (com seu Border Layout) um painel com Card Layout, definido como Painei Central. Neste Painei Central inserimos (adicionamos) vários outros painéis com layouts diversos, e que estarão visíveis à medida que se solicite sua visualização.

Não há uma sequência específica de inserção de painéis em um Card Layout, apenas devemos estar atentos ao fato de que o primeiro deles estará visível ao abrirmos a tela (ao acionarmos o sistema), isso ocorrerá com o primeiro painel adicionado ao Painei Central ao longo do código.

Exemplo:

Imagine que o objeto `pc` seja o Painei Central (adicionado ao centro do frame). A ele vamos adicionar 4 painéis representando quatro telas diferentes, de forma que, como o painel "p1" será o primeiro inserido, ele será o primeiro visível ao abrirmos (acionarmos) o nosso sistema.

```
// Declarando os Painéis
Panel pc = new Panel(); // Representando o Painei Central
Panel p1 = new Panel(); // Representando o Painei 1
Panel p2 = new Panel(); // Representando o Painei 2
Panel p3 = new Panel(); // Representando o Painei 3
Panel p4 = new Panel(); // Representando o Painei 4
...
this.add(pc); // adicionando o Painei Central (pc) no
              // ...centro (CENTER) do Border Layout
pc.setLayout(new CardLayout); // definindo o layout //...do painel central como
CardLayout
pc.add(p1, "p1"); // será o primeiro painel
                  //...a ser visualizado
pc.add(p2, "p2");
pc.add(p3, "p3");
pc.add(p4, "p4");
...
```

Perceba que ao adicionarmos um painel ao Painei Central, inserimos uma string como 2º parâmetro do método `add(...)`. Como sugestão, daremos o mesmo nome do objeto que está sendo adicionado, mas que na verdade pode ser qualquer nome, desde que cada painel tenha uma denominação. Esta string será necessária quando formos utilizar o comando de alteração de painel (troca de painel), e com isso alternar as telas do sistema.

Para mostrar determinado painel específico, a classe `CardLayout` traz métodos que alteram o painel que está sendo mostrado.

O método `show` da classe `CardLayout` permite indicar o Painei Central (o qual já contém todos os painéis adicionados a ele) e o nome (string) do painel específico que deverá ser mostrado a partir daquele momento.

```
objCardLayout.show(paineiCentral, nomeStringDoPainei);
```



Observação

O objeto da classe `CardLayout`, que representará o layout da tela (que é um `CardLayout`), deverá ser montado (criado) a partir da captura do layout do Painel Central:

```
CardLayout objCardLayout = (CardLayout) (painelCentral.  
getLayout());
```

O casting realizado ao capturar o layout do Painel Central será o seguinte:

```
CardLayout cL = (CardLayout) (painelCentral.getLayout());  
  
cL.show(painelCentral, "nomeDoPainelASerMostrado");
```

Outros métodos da classe `CardLayout` também permitem visualizar os painéis inseridos no Painel Central, mas estes dependerão da sequência de adição dada a eles, como: `first(painelCentral)`, `last(painelCentral)`, `next(painelCentral)`, `previous(painelCentral)`, que indicam, respectivamente, primeiro, último, próximo e anterior.

2.6 Trabalhando com menus

Os menus são componentes que permitem a navegação e o acionamento dos diversos recursos disponíveis em um sistema. Eles podem permanecer na parte superior de uma janela (barra de menus) ou aparecer como opções de ação à medida que clicamos com o botão direito do mouse (menus de contexto).

Vamos mostrar agora como gerar uma barra de menus, como dispor seu conteúdo, e como trabalhar sua funcionalidade. Para a criarmos na parte superior de uma janela, e seu conjunto de elementos, utilizamos basicamente quatro classes:

- **MenuBar OU JMenuBar:** que representam a barra superior de menus das bibliotecas AWT e Swing, respectivamente.
- **Menu OU JMenu:** que representam os elementos de menus, aqueles que fornecem novas opções de menus.
- **MenuItem OU JMenuItem:** que representam os elementos de menus que fornecem ou acionam alguma ação direta.
- **JSeparator:** que representa o separador de categorias de menus. Essa classe existe apenas na biblioteca Swing. Para adicionarmos um separador de categorias utilizando a biblioteca AWT, usamos os métodos `addSeparator()`, ou `insertSeparator(...)`, da classe `Menu`.

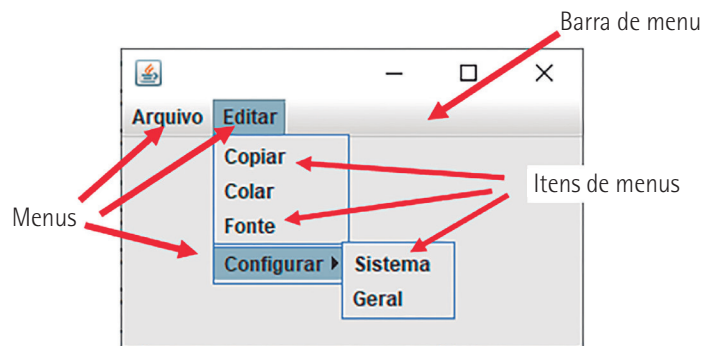


Figura 12 – Representação de uma janela com elementos de menus utilizando-se da biblioteca Swing

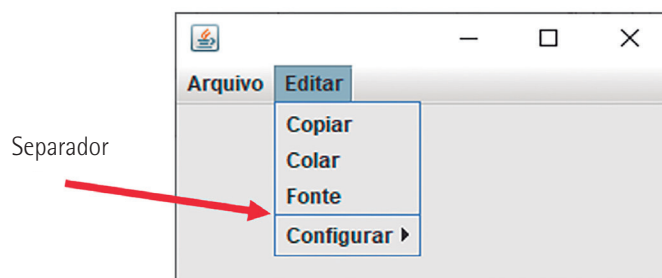


Figura 13 – Representação de uma janela com um separador de categorias de menus



Saiba mais

Para aprender mais sobre menus e itens de menus, verifique o tutorial disponibilizado e administrado pela Oracle no seguinte link:

THE JAVA™ Tutorials. Oracle, [s.d.]. Disponível em: <https://tinyurl.com/336f5bha>. Acesso em: 28 jul. 2026.

2.6.1 Inserindo os elementos de menus

De modo geral, os elementos de menus devem ser adicionados de acordo com a sua ordem de aparecimento, seja na barra de menus (da esquerda para a direita), seja em algum menu (de cima para baixo).

A barra de menus precisa ser definida no frame (ou no JFrame) através do método `setMenuBar(...)` ou do método `setJMenuBar(...)`, dependendo de qual biblioteca se está utilizando (se AWT ou se Swing).

Por exemplo: `this.setJMenuBar(objetoJMenuBar);`

Na barra de menus, adicionamos os elementos de menus que primeiro serão visualizados no formulário.

Por exemplo: `objetoJMenuBar.add(objetoJMenu);`

Em cada menu, adicionamos os elementos de menus (Menus e/ou Menultens) na sua ordem de aparecimento (de cima para baixo).

Por exemplo: `objetoJMenu.add(elementoMenu);`

Assim como adicionamos elementos, podemos adicionar uma linha de separação:

Por exemplo: `objetoJMenu.add(new JSeparator());`

Exemplo de aplicação

Como exemplo, vamos gerar o código que monta a seguinte janela de menus:

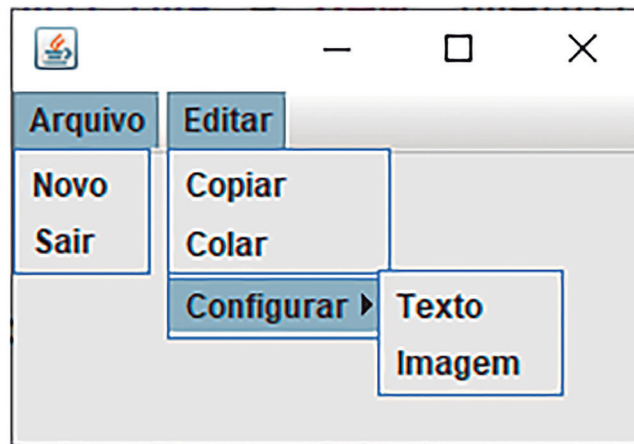


Figura 14 – Janela de exemplo de elementos de menus

Para construir esta janela, basta gerar o seguinte código (classe `TelaComMenus`):

```
// Importações necessárias
import javax.swing.JFrame;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;
import javax.swing.JSeparator;
// Classe TelaComMenus
public class TelaComMenus extends JFrame {
    // A Barra de Menu
    JMenuBar mb = new JMenuBar();
    // Os Menus da Janela
    JMenu mArq = new JMenu("Arquivo");
    JMenu mEdit = new JMenu("Editar");
    JMenu mConfig = new JMenu("Configurar");
    // Os Itens de Menu da Janela
    JMenuItem miNovo = new JMenuItem("Novo");
```

```
JMenuItem miSair = new JMenuItem("Sair");
JMenuItem miCopiar = new JMenuItem("Copiar");
JMenuItem miColar = new JMenuItem("Colar");
JMenuItem miCTexto = new JMenuItem("Texto");
JMenuItem miCImg = new JMenuItem("Imagem");
// Separador
JSeparator mSep = new JSeparator();
// Método main para instanciar a Classe
public static void main(String[] args) {
    TelaComMenus tp = new TelaComMenus();
}
// Método construtor que constrói a Tela e seus Menus
public TelaComMenus() {
    this.setBounds(100, 100, 250, 170);
    // "setando" a barra de menu na janela
    // ... (para que a exista)
    this.setJMenuBar(mb);
    // adicionando os menus na barra de menus
    mb.add(mArq);
    mb.add(mEdit);
    // adicionando os elementos do menu "Arquivo"
    mArq.add(miNovo);
    mArq.add(miSair);
    // adicionando os elementos do menu "Editar"
    mEdit.add(miCopiar);
    mEdit.add(miColar);
    mEdit.add(mSep);
    mEdit.add(mConfig);
    // adicionando os elementos do menu "Configurar"
    mConfig.add(miCTexto);
    mConfig.add(miCImg);
    //-----
    this.setVisible(true);
}
}
```



Lembrete

Na montagem da tela, a última ação é deixar a janela visível. Isso acontece para que não ocorra problemas na ocasião da construção e disponibilização dos componentes nas telas geradas em memória.

2.7 Trabalhando com tabelas

Uma forma de trabalhar com tabelas em Java é utilizar a estrutura do componente `JTable`, que possibilita inúmeras funcionalidades práticas ao trabalho com dados lidos de BD.

Para criarmos uma tabela de dados em Java, basicamente utilizamos três classes, duas delas definem efetivamente a estrutura e a disponibilização das informações na forma de tabela, enquanto a outra permite a visualização das várias informações da tabela, mesmo que não caibam em apenas

uma tela. A barra de rolagem é um componente geral de telas que possibilita a visualização de elementos que não estão visíveis por ocuparem um espaço maior do que o definido para amostrá-los, ou por se localizarem abaixo ou acima da linha visível da tela.

Sendo assim, as três classes principais utilizadas na confecção de tabelas são:

- **Classe `JTable`**: representa a estrutura da tabela.
- **Classe `DefaultTableModel`**: representa os itens da tabela (modelo), ou ainda, os elementos internos da tabela.
- **Classe `JScrollPane`**: representa um painel com a barra de rolagem, já que a `JTable`, por si mesma, não possui tal barra, e é muito comum precisarmos dela para ver todas as informações disponíveis na tabela.



Observação

A classe `JScrollPane` representa um painel com barra de rolagem, ou seja, é um container que recebe internamente outros componentes (ou containers), e disponibiliza os eixos de barra de rolagem para permitir a visualização dos elementos que ficam fora do campo de visão definido pelo tamanho da janela (ou da área estabelecida para amostragem dos elementos).

2.7.1 Elementos e características da construção de uma tabela

Apresentaremos agora como devemos preparar a instância das classes que permitem a construção de uma tabela.

No parâmetro do construtor do modelo (`DefaultTableModel`) inclui-se um array de strings com os títulos de cada coluna.

```
objDTM = new DefaultTableModel(arrayStringTit, 0);
```

Observação: `objDTM` é o nome do objeto que representa a classe `DefaultTableModel`. Após a sua instância, pode-se adicionar novas colunas com o método `objDTM.addColumn("TítuloDaColuna")`;

Cada linha de informações (dos dados) da tabela deve ser adicionada ao modelo utilizando-se o método `addRow(...)`, inserindo em seu parâmetro o array de strings com as informações da linha (um array com a mesma quantidade de informações que de títulos):

```
objDTM.addRow(arrayStringLinha);
```

Na instância da tabela (`JTable`), ou ainda no parâmetro de seu construtor, inclui-se o modelo (`DefaultTableModel`) criado.

```
objJT = new JTable(objDTM);
```

Observação: `objJT` é o nome do objeto que representa a classe `JTable`.

Para inserir a tabela ao painel de barra de rolagem, podemos utilizar dois possíveis caminhos:

- Na instância do painel da barra de rolagem, ou ainda, no parâmetro de seu construtor (`JScrollPane`), inclui-se o objeto representante da estrutura da tabela (`JTable`):

```
objJScrollPane = new JScrollPane(objJT);
```

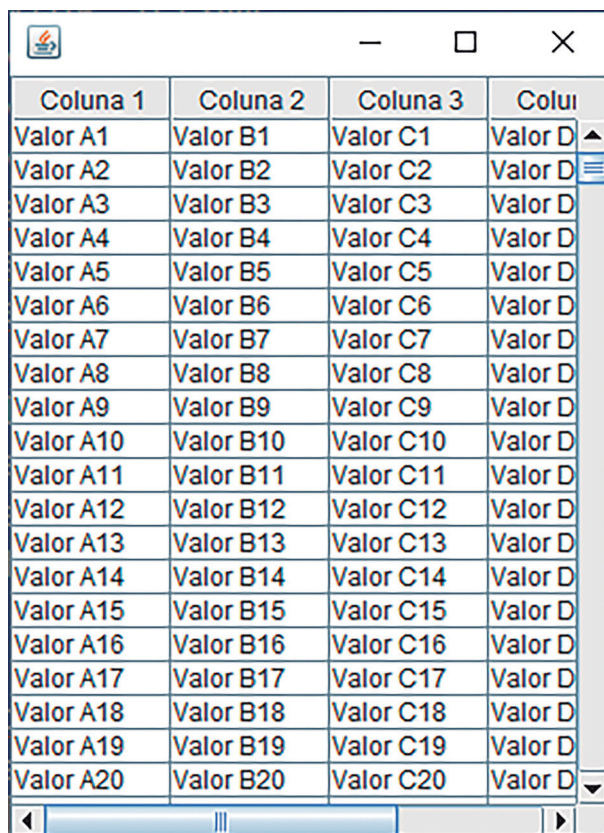
- Utilizando o método `setViewportView(...)` da classe `JScrollPane` (em seu objeto já instanciado), e recebendo como parâmetro a estrutura da tabela (`JTable`):

```
objJScrollPane.setViewportView(objJT);
```

Observação: `objJScrollPane` é o nome do objeto que representa a classe `JScrollPane`.

Exemplo de aplicação

Agora veremos o exemplo da janela com uma tabela:



Coluna 1	Coluna 2	Coluna 3	Coluna 4
Valor A1	Valor B1	Valor C1	Valor D1
Valor A2	Valor B2	Valor C2	Valor D2
Valor A3	Valor B3	Valor C3	Valor D3
Valor A4	Valor B4	Valor C4	Valor D4
Valor A5	Valor B5	Valor C5	Valor D5
Valor A6	Valor B6	Valor C6	Valor D6
Valor A7	Valor B7	Valor C7	Valor D7
Valor A8	Valor B8	Valor C8	Valor D8
Valor A9	Valor B9	Valor C9	Valor D9
Valor A10	Valor B10	Valor C10	Valor D10
Valor A11	Valor B11	Valor C11	Valor D11
Valor A12	Valor B12	Valor C12	Valor D12
Valor A13	Valor B13	Valor C13	Valor D13
Valor A14	Valor B14	Valor C14	Valor D14
Valor A15	Valor B15	Valor C15	Valor D15
Valor A16	Valor B16	Valor C16	Valor D16
Valor A17	Valor B17	Valor C17	Valor D17
Valor A18	Valor B18	Valor C18	Valor D18
Valor A19	Valor B19	Valor C19	Valor D19
Valor A20	Valor B20	Valor C20	Valor D20

Figura 15 – Janela de exemplo de tabela com 400 linhas de dados e barra de rolagem

```
//Importações necessárias
import java.awt.BorderLayout;
import javax.swing.JFrame;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.table.DefaultTableModel;
//A Classe Tabela
public class Tabela extends JFrame{
    //Atributos
    JTable jt; //Estrutura da tabela
    DefaultTableModel dtm; //Objeto que permite manipulação
    JScrollPane jspane; //Painel com Barra de Rolagem
    //Método main para instanciar a Classe
    public static void main(String[] args) {
        Tabela tb = new Tabela();
    }
    //Método Construtor - Construindo a Janela com uma Tabela
    public Tabela() {
        //Abrindo a janela bem no centro da tela
        this.setSize(300, 400);
        this.setLocationRelativeTo(null);
        this.setDefaultCloseOperation(this.EXIT_ON_CLOSE);
        //Construindo o array de títulos
        String[] arrTitulo = {"Coluna 1", "Coluna 2", "Coluna 3",
                               "Coluna 4", "Coluna 5"};
        //Instanciando os elementos da tabela
        dtm = new DefaultTableModel(arrTitulo, 0);
        jt = new JTable(dtm);
        jspane = new JScrollPane(jt);
        this.add(jspane); // Inserido na parte central da Janela
        //.....
        //Desvinculando o tamanho das células com o da Janela
        jt.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
        //Mostrando a Barra de rolagem horizontal
        jspane.setHorizontalScrollBarPolicy(
            JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
        //.....
        //Construindo o array de dados para 400 linhas de tabela
        String[] arrDados = new String[5];
        for (int x = 1; x <= 400; x++) {
            arrDados[0] = "Valor A" + x;
            arrDados[1] = "Valor B" + x;
            arrDados[2] = "Valor C" + x;
            arrDados[3] = "Valor D" + x;
            arrDados[4] = "Valor E" + x;
            //Adicionando uma linha de dados na tabela
            dtm.addRow(arrDados);
        }
        //Deixando a Janela visível
        this.setVisible(true);
    }
}
```

3 EVENTOS

Os eventos são as possíveis ações de interação do usuário com o sistema o qual ele está utilizando. Eles podem acontecer:

- pelo acionamento de um botão;
- pela passagem do mouse sobre algum componente;
- pelo acionamento das ações de janela como minimização, maximização etc.;

A linguagem Java tem um modo particular de controlar os eventos. Para isso, ela conta com as interfaces chamadas Listeners, criadas para serem implementadas pelas classes de componentes cujos eventos precisam ser controlados.



Lembrete

Interface é um elemento da linguagem Java que se assemelha a uma classe totalmente abstrata, ou seja, com métodos abstratos, e atributos estáticos e finais.

São alguns exemplos de Listeners:

`WindowListener` (eventos de **janela**)

Esta interface de eventos possui os seguintes métodos abstratos:

- **`windowOpened(WindowEvent e)`** : método acionado na primeira vez que a janela é aberta.
- **`windowClosing(WindowEvent e)`** : método acionado quando o ícone que fecha a janela é selecionado (usado para encerrar o programa).
- **`windowClosed(WindowEvent e)`** : método acionado na efetivação do fechamento de uma janela.
- **`windowActivated(WindowEvent e)`** : método acionado quando uma janela é ativada, ou quando é dado o foco a ela (clicando sobre ela, por exemplo).
- **`windowDeactivated(WindowEvent e)`** : método acionado quando uma janela é desativada, ou quando é retirado o foco dela (clicando sobre outra janela, por exemplo).
- **`windowIconified(WindowEvent e)`** : método acionado quando uma janela é minimizada.

- **windowDeiconified(WindowEvent e)**: método acionado quando uma janela é restaurada a partir de um ícone.

MouseListener (eventos específicos de mouse)

Esta interface de eventos possui os seguintes métodos abstratos:

- **mouseClicked(MouseEvent e)**: método acionado quando o botão do mouse é clicado (e solto) sobre um componente.
- **mousePressed(MouseEvent e)**: método acionado quando o botão do mouse é clicado sobre um componente.
- **mouseReleased(MouseEvent e)**: método acionado quando o botão do mouse é solto sobre um componente.
- **mouseEntered(MouseEvent e)**: método acionado quando o mouse "entra" na área de um componente.
- **mouseExited(MouseEvent e)**: método acionado quando o mouse deixa a área de um componente.

MouseMotionListener (eventos de movimentação do Mouse)

Esta interface de eventos possui os seguintes métodos abstratos:

- **mouseMoved(MouseEvent e)**: método acionado quando o ponteiro do mouse se movimenta sobre um componente.
- **mouseDragged(MouseEvent e)**: método acionado quando o mouse se movimenta sobre um componente enquanto o seu botão está pressionado.

KeyListener (eventos de teclado)

Esta interface de eventos possui os seguintes métodos abstratos:

- **keyTyped(KeyEvent e)**: método acionado quando uma tecla do teclado é pressionada e solta (com exceção das teclas SHIFT, ALT, CTRL, direcionais, Insert, Delete, teclas de função etc.).
- **keyPressed(KeyEvent e)**: método acionado quando uma tecla do teclado é pressionada.
- **keyReleased(KeyEvent e)**: método acionado quando uma tecla do teclado é solta.

ActionListener (eventos de ação)

Esta interface de eventos possui o seguinte método abstrato:

- **actionPerformed(ActionEvent e)**: método acionado quando uma ação ocorre em algum componente da janela (por exemplo, um botão da janela é pressionado).



Saiba mais

Para aprender mais sobre o funcionamento e o tratamento de eventos, leia o capítulo 8 da obra a seguir:

HORSTMANN, C. S.; CORNELL, G. *Core Java*. Volume I: fundamentos. 8. ed. São Paulo: Pearson Prentice Hall, 2010.

3.1 Utilizando-se dos eventos (controlando-os)

A utilização dos eventos pode ser programada de duas formas:

- fazendo com que a classe implemente as interfaces relacionadas a cada um deles (lembrando que uma classe pode fazê-lo com mais de uma ao mesmo tempo);
- fazendo com que o componente implemente a interface ao adicionar o Listener no componente a ser controlado.

Caso optemos por implementar as interfaces dos Listeners à classe, devemos lembrar de usar todos os métodos que a interface possui.



Lembrete

Implementar uma interface é utilizar, na declaração da classe, a palavra reservada **implements** com o nome das interfaces. Quando implementamos a interface em uma classe, todos os métodos existentes na interface devem ser criados (implementados) na própria classe.

Exemplo de aplicação

Vamos considerar a seguinte interface:

```
public interface interqualquer {  
    public void metodoH(int par01, double par02);  
    public double metodoK(double par02);  
}
```

Ela contém 2 métodos (que são abstratos).

Para fazer com que a classe (como a `ClasseA` a seguir) implemente a interface, ela deverá conter a implementação dos métodos da interface. A implementação dos seguintes métodos não está realizando efetivamente uma ação específica, de forma que os métodos `metodoH(...)` e `metodoK(...)` devem existir na classe, já que ela está implementando a interface `interqualquer`.

```
public class ClasseA implements interqualquer {  
    public void metodoH(int par01, double par02) {  
        //...  
    }  
  
    public double metodoK(double par02) {  
        double d1;  
        //...  
        return d1;  
    }  
}
```

3.1.1 Passos (na programação) para uso dos eventos

Veremos agora que é possível realizar (programar) um controle de eventos de duas maneiras: implementando a interface equivalente ao evento, o que é feito na própria declaração da classe; e adicionando o evento ao componente controlado, sendo ambos programados ao longo de um método.

Com o objetivo de fazer com que a classe implemente as interfaces necessárias para controle dos eventos, siga os seguintes passos:

- Implementar as interfaces à classe (com o termo `implements`).
- Gerar todos os métodos das interfaces na classe.
- Adicionar os eventos diretamente ao componente controlado ("ligar" o componente), já com a programação de controle.
- Implementar a lógica nos métodos (ou evento específico) a serem controlados (observação: quando a interface possui vários métodos, apesar de ser necessário implementar os métodos na classe, não é preciso codificar o controle de todos eles, mas apenas para aqueles a que se quer efetivamente controlar).

Exemplo de aplicação

```
// Implementando o controle de eventos de janela  
public class TelaPrincipal extends JFrame implements  
    WindowListener {  
    // Declaração dos Atributos / Componentes
```

```
// Método de montagem da tela
public void montaTela() {
    // ...
    // Adicionando o Evento ao Componente (ligando-o)
    // Obs.: o parâmetro indica o componente controlador
    // ...que no caso, é a própria janela
    this.addWindowListener(this);
    // ...
}
// Métodos da interface (obrigatório)
// ...
public void windowClosing(WindowEvent e) {
    // Lógica quando o X for apertado (p.ex.: encerra o programa)
    System.exit(0);
}
// ...
// E mais TODOS os outros Métodos da interface (obrigatório)
}
```

Para o caso de controlar o evento utilizando a adição do evento ao componente, sem implementar a interface (Listener), mas o controle no parâmetro do método de adição de evento, devemos usar os seguintes passos:

- Adicionar o evento ao componente controlado.
- No parâmetro do método de adição da interface, instancia-se o Listener do evento a ser controlado (observação: neste caso, a implementação também exige a geração do método abstrato. Para alguns Listeners que possuem diversos métodos abstratos, existe uma classe Adapter a qual pode-se sobrescrever o método necessário, que se deseja controlar – como o WindowAdapter, ou o MouseAdapter etc.).
- Implementar o código de controle no método (ou evento específico) a ser controlado com a lógica necessária.



Observação

As classes [...]Adapter são abstratas que implementam a interface correspondente, e que possibilitam a sua utilização sem a necessidade de implementação de todos os métodos, como aqueles que não precisam ser controlados.

Exemplo de aplicação

```
public class TelaPrincipal extends JFrame {
    // Declaração dos Atributos / Componentes
    // Método de montagem da Tela
    public void montaTela() {
```

```
// ...
// Adicionando o Evento ao Componente, já com a lógica
// ...e neste caso utilizando o Adapter dos eventos de janela
// ...evitando ter que adicionar todos os outros métodos
this.addWindowListener (new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        // Implementando a lógica
        System.exit(0); // (p.ex.: encerrando o programa)
    }
});
// ...
}
```



Observação

Se utilizarmos o `WindowListener` ao invés do `WindowAdapter`, seremos obrigados a implementar todos os sete métodos existentes na interface `WindowListener`, mesmo que não coloquemos códigos em seu corpo.

3.2 Os eventos de ação

Os eventos de ação são os eventos (ou ações) ligados ao clicarmos em algum "botão" da tela, ou selecionarmos algum item de menu.

Ao implementar o `ActionListener`, que é a interface de eventos de ação, deve-se criar o método `actionPerformed(...)`:

```
public void actionPerformed(ActionEvent e) {
    // lógica de ação
}
```

(este é o método abstrato daquela interface)

Desta forma, sempre que um componente for "ligado" (ao qual for adicionado o `ActionListener`), por exemplo, ao clicarmos sobre um botão, este método será acionado (iniciado e rodado) automaticamente.



Observação

O objeto "e" (representando o `ActionEvent` do parâmetro do método) recebe todas as informações necessárias para programar a performance dos botões.

Sabendo que ao acionar um botão (ou um componente controlado por esse evento), seja ele qual for, este método será acionado, como poderíamos então diferenciar os diversos botões existentes na tela, e com isso diferenciar as ações que cada um deles deverá realizar?

Para diferenciar um botão do outro, podemos utilizar um objeto de comparação, que recebe o objeto do componente exato que foi acionado (obtido a partir do `ActionListener`, do parâmetro do método), e que pode ser comparado aos componentes existentes na tela (na classe da janela), desde que eles sejam atributos da classe que representa a janela. Assim, fazemos:

```
Object obj = e.getSource();
```



Observação

Ao estudarmos o conceito de polimorfismo de classes, vimos que uma classe "mãe" (ou uma superclasse) pode se comportar como uma de suas classes filhas (as subclasses).

Neste caso, estamos instanciando um objeto do tipo `Object`, que é a classe mãe de todas as classes do Java, o qual receberá o objeto representante em memória do componente acionado, lembrando que o objeto "e" (do exemplo) representa o evento de ação (e é recebido via parâmetro do método).

Exemplo de aplicação

Consta a seguir um exemplo de programa que controla o evento de ação ao acionarmos um botão de Ok:

```
public class TelaPrincipal extends JFrame implements
    ActionListener {
    // Declaração dos Componentes (atributos)
    // Método de montagem da Tela
    public void montaTela() { // ...por exemplo
        // ...
        // Adicionando o Evento ao Componente (ligando-o)
        botOk.addActionListener(this);
        // ...
    }
    public void actionPerformed(ActionEvent e) {
        // Para saber qual botão foi acionado...
        Object obj = e.getSource();
        if (obj.equals(botOk)) {
            // ...
            // Lógica de ação para o Botão Ok
            // ...
        }
    }
}
```

Pode-se colocar diretamente a lógica de funcionamento ao adicionar o evento de ação ao componente:

```
public class TelaPrincipal extends JFrame {  
    // Declaração dos Componentes (atributos)  
    // ...  
    // Método de montagem da Tela  
    public void montaTela() {  
        // ...  
        // Adicionando o Evento ao Componente, já com a lógica  
        botOk.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                // ...  
                // Lógica de ação para o Botão Ok  
                // ...  
            }  
        });  
        // ...  
    }  
}
```

3.3 Os eventos de janela

Os eventos de janela são aqueles que controlam o funcionamento geral da janela (e não especificamente de seus componentes), tais como: minimização, maximização, abertura inicial e fechamento da janela (ao sair do sistema), entre outros eventos.

Neste item, vamos mostrar o exemplo de como criar uma janela que é fechada (encerrando o programa) ao clicar sobre o ícone ( – ícone de fechamento) do canto superior direito de qualquer janela do Windows.

Ao criarmos uma janela com a linguagem Java e com a biblioteca Swing, o ícone de fechamento não vem preparado para fechar e encerrar automaticamente o programa que estamos rodando, de forma que a fim de fazer que isso funcione corretamente, precisamos inserir elementos de código que possibilitem tal execução.

Para este exemplo, criaremos uma janela simples (sem componentes internos), mas que estará preparada para fechar (e encerrar o programa) ao clicar no ícone de fechamento ().

Criaremos duas classes (Janela1 e Janela2), de forma que ambas estejam preparadas para funcionar com o ícone de fechamento, porém a Janela1 utilizará um processo mais geral, que funciona tanto com componentes do AWT quanto com aqueles do Swing. Já a Janela2 usará um processo que somente funciona com componentes do Swing, uma vez que o método utilizado pertence apenas à biblioteca Swing. Essa última é mais prática, porém como já dito, apenas funcionará com a biblioteca Swing.

- **Classe Janela1:** criando uma janela com a biblioteca AWT e utilizando uma forma de controle de fechamento e encerramento do programa que funciona em ambas as bibliotecas (AWT e Swing), mas que ainda é mais prática que a implementação da interface na declaração da classe.

```
import java.awt.Frame;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
public class Janela1 extends Frame {

    public Janela1() {
        //Método Construtor - que "constrói" a tela:

        this.setBounds(200, 200, 300, 300);
        //O método setBounds posiciona e dá tamanho ao componente
        //...componente.setBounds(posH, posV, tamH, tamV);

        // ##Adicionando o evento à Janela##
        //...vamos controlar o evento de
        //...fechamento e encerramento do Programa/Janela
        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
        //...nesta classe, não fizemos a implementação geral (total)
        //...da interface windowlistener,
        //...pois se o tivéssemos feito, teríamos que inserir
        //...todos os métodos desta interface.
        //... Assim, para esta classe (Janela1)
        //... utilizamos uma classe chamada WindowAdapter
        //... que nos facilitou a utilização dos métodos
        //... daquela interface.
        //... Como, por exemplo, o método "windowClosing(...)"

        this.setVisible(true);
    }

    public static void main(String[] args) {
        //... instanciando esta classe e
        //... consequentemente acionando seu Método Construtor
        //... e todas as suas funcionalidades
        Janela1 j1 = new Janela1();
    }
}
```

- **Classe Janela2:** criando uma janela com a biblioteca Swing e utilizando uma forma de controle de fechamento e encerramento do programa que funciona apenas para a biblioteca Swing.

```
import javax.swing.JFrame;

public class Janela2 extends JFrame {

    public Janela2() {
```

```
        this.setBounds(200, 200, 300, 300);

        // Acionando um método desta biblioteca
        // que faz funcionar o "ícone de fechamento"
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        this.setVisible(true);
    }

    public static void main(String[] args) {
        //... instanciando esta classe e
        //... consequentemente acionando seu Método Construtor
        Janela2 j2 = new Janela2();
    }
}
```

Nesta segunda janela, como utilizamos a biblioteca Swing, podemos usar o método que faz funcionar o ícone de fechamento ().

Observação: no segundo caso, para acionarmos ou controlarmos os outros eventos de janela (como o de minimização etc.), devemos utilizar a classe WindowAdapter, como fizemos no exemplo anterior, da Janela1.

4 JDBC

Java DataBase Connectivity (JDBC) é a biblioteca de persistência em BD relacionais do Java. O processo de armazenamento de dados é também chamado de persistência da informação.

Para que o Java se conecte com um determinado BD, é necessário que haja um driver que permita o acesso a ele. Assim como no processo do ODBC, o JDBC funciona através de drivers responsáveis pela conexão com o banco, e pela execução das instruções SQL.

Os drivers Java são implementações das interfaces do pacote java.sql. Eles geralmente são disponibilizados gratuitamente na forma de um arquivo JAR (ou seja, Java ARchive) pelo fabricante do BD (em seus sites) ou por terceiros.

Neste material sugere-se em nossos exercícios a utilização do MySQL como servidor de BD. Essa sugestão ocorreu por se tratar de um produto que pode ser baixado e instalado gratuitamente, e pela facilidade de manipulação (a partir de seu workbench).

Para a instalação e configuração do MySQL, caso ainda não esteja instalado, pode-se encontrar dicas e sugestões no **Apêndice A** deste livro-texto. É importante saber que os exercícios estarão voltados a este SGBD (servidor de banco de dados). No entanto, sua teoria e o conhecimento gerado sobre a parte Java, podem ser ampliados a qualquer outro banco de dados.

Constituem as classes principais utilizadas no acesso a um BD:

- **Classe DriverManager**: gerencia o driver do BD (que foi carregado pelo ClassLoader).
- **Classe Connection**: responsável pela conexão com o BD (conexão com a base de dados).
- **Classe Statement (ou PreparedStatement)**: representa o canal de comunicação com o BD, onde enviamos as queries para serem executadas pelo BD e de onde recebemos os dados vindos do BD.
- **Classe ResultSet**: recebe os dados em forma de listas de informações e utiliza ponteiros para controlar a posição dos registros.



Saiba mais

Para aprender mais sobre JDBC, leia o capítulo 24 da obra a seguir:

DEITEL, P.; DEITEL, H. *Java: como programar*. São Paulo: Pearson Education do Brasil, 2017.

4.1 Os conectores/drivers

Geralmente cada fabricante de BD fornece o driver Java (a biblioteca ".jar") para possibilitar a conexão com seu BD. É possível encontrar versões antigas do MySQL (como a 5.1, por exemplo), do driver no seguinte site:

<https://dev.mysql.com/downloads/connector/j/5.1.html>

Nas versões atuais do MySQL (8.0 ou posterior), o driver já está presente na instalação do próprio MySQL e geralmente se localiza no seguinte diretório:

C:\Program Files (x86)\MySQL\Connector J X.X (onde X.X é o número da versão instalada – por exemplo: C:\Program Files (x86)\MySQL\Connector J 8.0)

De todo modo, em qualquer um dos casos, o arquivo `mysql-connector-java-<versao>.jar` deverá ser importado (como Jar Externo) no Projeto do Eclipse, ou do NetBeans.

Driver do MySQL

Para gerar uma conexão e com isso acessar o BD propriamente dito, deve-se definir qual será o driver de acesso no programa (no código) utilizando-se o seguinte método da classe `Class`:

```
Class.forName(String name);
```

O que este método faz é solicitar ao ClassLoader (o responsável por carregar o driver fornecido pelo fabricante do BD) que ele instancie a classe especificada pela string definida em seu parâmetro (indicando o caminho para encontrar a classe do driver: pacote.Classe).

Por exemplo: `Class.forName("com.mysql.jdbc.Driver");`

Além disso, a classe `DriverManager` possui alguns métodos com o nome `getConnection()` em sobrecarga, eles são responsáveis por procurar entre os drivers carregados um que seja compatível com a URL fornecida, e com ele solicitar a abertura de uma conexão. Este método utiliza o driver carregado pelo `Class.forName(...)` para fornecer a conexão. A URL é basicamente:

`protocolo:subprotocolo://IP_servidor:numero_porta/nome_base_dados`

Por exemplo: `"jdbc:mysql://localhost:3306/aula_alpoo"`

Na string do exemplo anterior, o local do BD mysql é a própria máquina do usuário (por isso o localhost), e a base de dados se chama aula_alpoo.

Importando um arquivo .jar fornecido (como o driver do BD)

- **No Eclipse:** clique com o botão direito do mouse sobre o projeto, vá em Properties (última opção), selecione Java Build Path no navegador à esquerda, clique na aba Libraries, clique no botão Add External JARs, selecione o arquivo JAR do driver do fabricante do BD, e clique em OK.

Consta a seguir a figura demonstrando o processo:

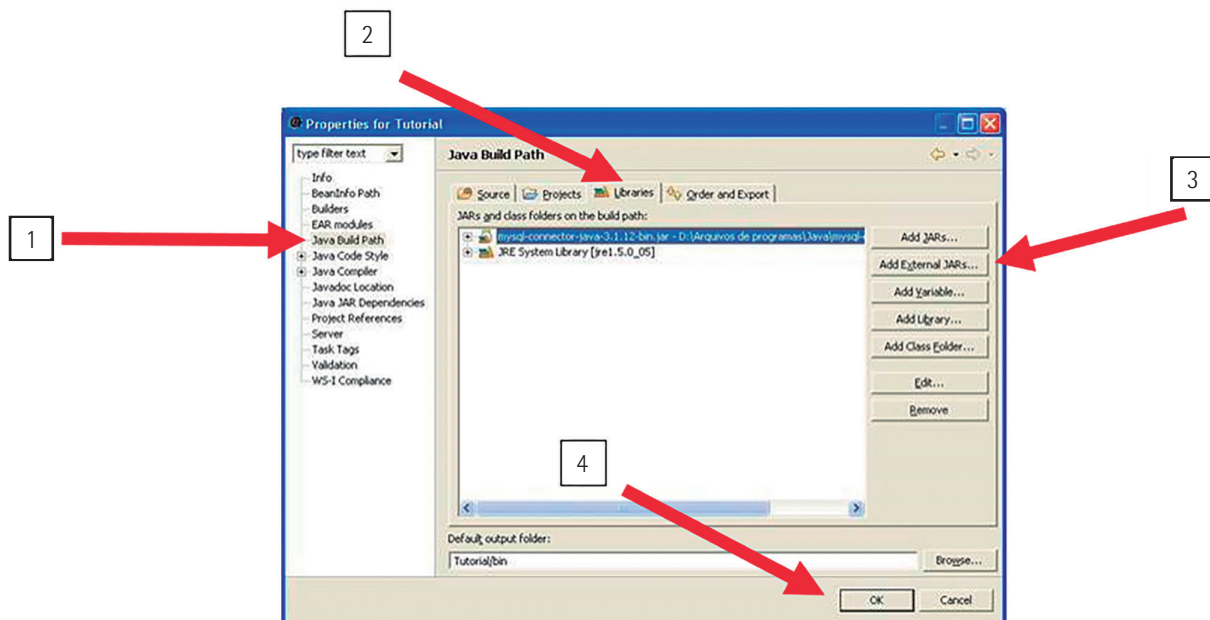


Figura 16 – Tela de propriedades do projeto: esquema de importação de “jar externo”

4.2 Conectando a um banco de dados do MySQL

Vimos que, para conectarmos um sistema em Java a um BD e manipularmos seus dados, utilizamos basicamente quatro classes: DriverManager, Connection, Statement (ou PreparedStatement), e ResultSet.

Com o objetivo de adquirirmos uma conexão com um BD do MYSQL, podemos aplicar a seguinte sequência de códigos (criando a classe GeraConexao).

Exemplo de aplicação

```
import java.sql.*;

public class GeraConexao {

    private String driver, url, login, senha;
    private Connection con;

    public GeraConexao() {
        driver = "com.mysql.cj.jdbc.Driver";
        //o driver acima é da versão 8.0 do mysql
        url = "jdbc:mysql://localhost:3306/escola";
        url += "?useTimezone=true&serverTimezone=UTC";
        //em alguns casos a linha acima será necessária
        login = "root"; // nome do usuário (ao instalar o mysql)
        senha = "1234"; // senha do usuário acima no mysql
        con = null;
        //acima: valor inicial da conexão (nulo) - antes do acesso
    }

    public Connection getConnection() {
        //retornará a Classe de conexão obtida
        String msg = "";
        try{
            Class.forName(driver);
            this.con = DriverManager.getConnection(url, login, senha);
        }catch(ClassNotFoundException ex){
            msg = ":: ERRO :: Driver JDBC não encontrado na aplicação!";
            System.out.println(msg);
        }catch(SQLException ex){
            msg = ":: ERRO :: Problemas na conexão com a fonte de dados";
            System.out.println(msg);
        }catch(Exception ex){
            msg = ":: ERRO :: Outros problemas na conexão...";
            System.out.println(msg);
        }
        return this.con;
    }
}
```

Importante: para que o código anterior funcione, o BD precisa conter os seguintes elementos:

Uma base de dados, chamada "escola";

Comentários sobre a classe do exemplo prévio

Pode-se observar que a classe apresentada importa a biblioteca (java.sql). Ela então utilizará esta biblioteca (que já é de seu conjunto de classes instaladas) para executar as linhas de código de acesso ao BD. A base de dados do mysql, a ser acessada pelo código deste exemplo, será a base escola e futuramente deverá conter uma tabela chamada alunos (especificada em exemplos posteriores neste livro-texto).

Além disso, ao instanciar a classe anterior, as principais strings utilizadas no acesso ao BD já são inicializadas (pelo método construtor da classe).

Sendo assim, a url indica qual é o sistema gerenciador do BD (no caso o mysql), além da localização da máquina do servidor do BD, que no caso informa uma instalação local (localhost – já que o BD está no próprio micro), assim como a porta de acesso 3306 (que é a porta padrão do mysql) e o nome do BD a ser acessado no mysql (que no exemplo é escola).

Com essa url, o nome do usuário (login) e a sua senha, que podem ser aqueles criados ao instalar o mysql, o programa solicita uma conexão (gera em memória um objeto do tipo Connection a partir do método getConnection(...), utilizando-se da classe DriverManager.

Veja que a classe do exemplo (classe GeraConexao) possui um método getConnection() que retorna uma conexão (um objeto do tipo Connection). Assim, para utilizá-la (GeraConexao), pode-se acionar as seguintes linhas de código.

Utilizaremos a classe Exemplo com um método main em que há uma simples chamada ao método que gera a conexão do objeto da classe GeraConexao mostrada anteriormente:

```
public class Exemplo() {
    Connection con;
    public static void main(String args[]) {
        GeraConexao gc = new GeraConexao();

        //gerando-se uma instância desta própria classe
        Exemplo ex = new Exemplo();
        ex.con = gc.getConnection();
        if (ex.con != null) {
            System.out.println("Conexão realizada com sucesso");
        }
    }
}
```



Lembrete

Para que o exemplo anterior funcione com as classes Exemplo e GeraConexao, o driver do mysql já deve ter sido importado (pelo processo de adicionar jar externo) ao projeto em que ambas estão construídas.

4.3 Lendo informações do BD

Para que possamos ler os dados de um BD (as informações das tabelas de suas bases de dados), precisamos antes saber algumas informações, tais como: o nome do BD, o nome da tabela a ser acessada e os campos (as colunas) da tabela com os dados necessários.

Com o objetivo de realizarmos essa leitura, utilizamos a linguagem PLSQL de acesso a BD.

O exemplo a seguir é de um programa que acessa o BD escola e todos os dados (das colunas: id, nome, ra, idade) da tabela alunos.

Para que essa leitura de dados ocorra efetivamente no BD, utilizaremos a classe GeraConexao já exibida (para resgatar uma conexão com o BD). A classe Aluno servirá para representar e guardar os dados de cada um dos alunos cadastrados, ou seja, cada linha da tabela alunos do BD escola.

Para este programa, a tabela alunos, no mysql, deverá possuir os seguintes campos (colunas) criados:

- **id:** representando a primary-key de cada linha de dado (um valor do tipo inteiro, ou "int").
- **nome:** representando o nome do aluno (um valor do tipo "varchar" – que pode ser de tamanho 50 –, ou uma string).
- **ra:** representando o registro do aluno na escola ("varchar" – que pode ser de tamanho 10 –, ou uma string).
- **idade:** representando a idade do aluno (um valor numérico do tipo inteiro, ou "int").

Antes de tentarmos fazer funcionar o método getallalunos(...) da classe a seguir, é importante que, no MySQL, a tabela alunos da base de dados escola contenha informações (dados de: id, nome, ra e idade) dos alunos (com valores fictícios) de uma determinada escola.

Sendo assim, a partir dessas informações, a classe AlunosBD utilizará a classe GeraConexao para fazer uma leitura no BD, capturará (fará uma leitura) da tabela alunos os valores de suas colunas e permitirá a exibição na tela de console das informações cadastradas em cada linha da tabela alunos (como já descrito), desde que a conexão ocorra com sucesso. Desta forma, a classe AlunosBD deve estar localizada no mesmo projeto que a classe GeraConexao.

Vejamos a seguir a classe Aluno:

```
public class Aluno {

    /*.....
    ESTA CLASSE É O QUE PODEMOS CHAMAR DE UM "JAVA BEAN", OU SEJA, UMA CLASSE
    JAVA QUE REPRESENTARÁ CADA DADO (CADA LINHA) DA TABELA ALUNO QUE LEREMOS (E
    ALTERAREMOS) NO BANCO DE DADOS. OBSERVE QUE ELA CONTÉM, EM FORMA DE ATRIBUTOS,
    CADA UM DOS CAMPOS DA TABELA DE DADOS.
    .....*/

    private int id;
    private String nome;
    private String ra;
    private int idade;

    /*Uma característica de um JavaBean, é que ele deve ser sempre uma classe
    Encapsulada, com os atributos privados e com os métodos "set" e "get"*/
    private int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getRa() {
        return ra;
    }
    public void setRa(String ra) {
        this.ra = ra;
    }
    public int getIdade() {
        return idade;
    }
    public void setIdade(int idade) {
        this.idade = idade;
    }
}
```

Vejamos a seguir a classe AlunosBD (com os métodos getAllAlunos(), mostraAlunos(...), abreCon() e fechaCon()):

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

public class AlunosBD {

    public Connection con;

    public ArrayList<Aluno> getAllAlunos() {
```

```

ArrayList<Aluno> arrAlunos = new ArrayList<Aluno>();
// ...abrindo a conexão
this.abreCon();
String query = "SELECT * FROM alunos";
try {
    Aluno al;
    PreparedStatement stmt = con.prepareStatement(query);
    ResultSet rs = stmt.executeQuery();
    // Montando um ArrayList de "Array e Strings"
    while (rs.next()) {
        al = new Aluno();
        al.setId(rs.getLong("id"));
        al.setNome(rs.getString("nome"));
        al.setRa(rs.getString("ra"));
        al.setIdade(rs.getInt("idade"));
        arrAlunos.add(al);
    }
    stmt.close();
} catch (Exception e) {
    System.out.println(":: ERRO :: Problemas com a leitura de dados no
BD...");
}
// ...fechando a conexão
this.fechaCon();
return arrAlunos;
}

public void mostraAlunos(ArrayList<Aluno> arrAlunos) {
    String txt = "";
    Aluno al;
    //Montando a String com todos os alunos cadastrados
    for (int x = 0; x < arrAlunos.size(); x++) {
        al = arrAlunos.get(x);
        txt += "\n" + al.getId();
        txt += "# " + al.getNome();
        txt += " - idade: " + al.getIdade();
        txt += " (ra: " + al.getRa() + ")";
    }
    // Mostrando na console a string já com todos os cadastros
    System.out.println(txt);
}

//.....PARA LIDAR COM O BANCO DE DADOS.....
//.....

private void abreCon() {
    GeraConexao gc = new GeraConexao();
    this.con = gc.getConnection();
}

private void fechaCon() {
    try {
        con.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

Esta classe funcionará com a classe GeraConexao (mostrada e criada no item 4.2).

No exemplo anterior, no método `getAllAlunos()`, utilizamos um objeto do tipo `ResultSet` para receber as informações lidas no BD (a partir da query – um comando – SQL `select...`).

Consequentemente, para cada linha lida no BD, o programa gera um objeto do tipo `Aluno`, sendo que cada um desses objetos é preenchido com as informações do aluno (lidas no BD), e em seguida o objeto (do tipo `Aluno`) é inserido em um `ArrayList` (uma matriz de objetos).

Ao final da execução do método `getAllAlunos()`, é retornado um objeto do tipo `ArrayList` que conterá vários objetos do tipo `Aluno`, cada um deles com as informações lidas nas linhas da tabela alunos da base de dados escola (do `MySQL`).

Já o método `mostraAlunos` recebe, via parâmetro, o `ArrayList` com todos os objetos do tipo `Aluno` (já preenchidos nos seus atributos com os dados do BD) lê um a um os dados de cada objeto do `ArrayList` montando uma string com todos os dados (de todos os objetos), mostrando o valor da string na tela da console.

A classe `TestaAlunos` (a seguir) utiliza a classe `AlunosBD` para efetivamente rodar os dois métodos da classe `AlunosBD` (um para pegar os dados do BD e o outro para mostrar na console os valores lidos) e imprimir na tela de console os dados da tabela alunos do BD (esta classe deve ser criada no mesmo projeto das classes dos exemplos anteriores).

Verifique os pré-requisitos de funcionamento da classe `AlunosDB` descritos logo após o exemplo em código.

```
import java.util.ArrayList;
public class TestaAlunos {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        AlunosBD ad = new AlunosBD();
        ArrayList<Aluno> arrAlunos = ad.getAllAlunos();
        ad.mostraAlunos(arrAlunos);
    }
}
```

4.4 Alterando informações no BD

A alteração de dados de um BD ocorre através das queries (ou seja, dos comandos SQL) do tipo: `INSERT`, `UPDATE` e `DELETE` – já que eles alteram informações no BD. Vejamos suas funcionalidades:

- **INSERT...:** é o comando que permite a inserção de uma nova informação na tabela (acrescenta uma linha de informação no BD).
- **UPDATE...:** é o comando que permite a modificação de informações já existentes na tabela.

- **DELETE...:** é o comando que permite a exclusão de uma linha de informações na tabela.



Observação

Esses comandos SQL podem ser estudados de forma mais profunda em outra disciplina deste mesmo curso que ensina a lidar com BD. Neste livro-texto, utilizaremos alguns exemplos da forma mais simples possível, sobre tais comandos, já que nosso objetivo aqui é utilizar a linguagem Java para acessar informações do BD.

Em um programa Java que realiza os tipos citados de alteração no BD, a diferença poderá ser observada nos exemplos a seguir, de forma que:

- nesses casos não utilizamos a classe `ResultSet`, já que essa classe é usada para receber informações do BD (como seria para um comando SQL do tipo `SELECT`).
- para esses casos, o método da classe `PreparedStatement` a ser rodado será `executeUpdate()`, e não o método `executeQuery`, como foi na leitura de informações, já que o `executeUpdate()` realiza alterações de informações no BD.

Mostraremos no exemplo a seguir um método cujo nome será `insert(...)` e que permite fazer a inserção de uma linha de novas informações no BD. Ele deve ser criado na classe `AlunosBD` apresentada previamente juntamente aos métodos que nela já existam.

Consta na sequência o método `insert()` a ser criado (inserido) na classe `AlunosBD`:

```
public void insert(Aluno al) {  
  
    // montando a query que inserirá uma informação no BD  
    String query = "insert into alunos ";  
    query += "(id, nome, ra, idade) ";  
    query += "values (?, ?, ?, ?);";  
    try {  
        // ...abrindo a conexão  
        this.abreCon();  
        PreparedStatement stmt = con.prepareStatement(query);  
        //Comandos Java que preenchem os  
        //..."pontos de interrogação" da query  
        stmt.setLong(1, al.getId());  
        stmt.setInt(4, al.getIdade());  
        stmt.setString(3, al.getRa());  
        stmt.setString(2, al.getNome());  
        //Executando a query no BD  
        stmt.executeUpdate();  
        stmt.close();  
        this.fechaCon();  
    } catch (Exception e) {
```

```
        e.printStackTrace();
    }
}
```

Este método recebe um objeto do tipo `Aluno` como parâmetro, já preenchido com as informações (de apenas um aluno) a serem inseridas no BD (como é somente de um aluno, será inserida apenas uma linha na tabela `aluno`).

Portanto, para testarmos o método mencionado, podemos criar a classe `TestaInsertAlunos` mostrada na sequência, que possui o método `main` e utiliza o método anterior para inserir um dado (uma linha de dados) no BD. Ela deve ser criada no mesmo diretório das classes `Aluno` e `AlunosBD`. Vejamos como executá-la:

```
public class TestaInsertAlunos {
    public static void main(String[] args) {
        // Instanciando a classe AlunosBD
        AlunosBD ad = new AlunosBD();
        // Criando um "JAVA BEAN" "Aluno"
        // (um objeto do tipo "Aluno" - com dados)
        // com as informações de um Aluno
        Aluno al = new Aluno();
        al.setId(15);
        //Obs.: este "id" não deve existir no BD
        al.setNome("Napoleão Bonaparte");
        al.setIdade(250);
        al.setRa("FRA1769AGO");
        //...ESTE DADOS SERÁ INSERIDO NO BD
        ad.insert(al);
    }
}
```

Para realizarmos outros tipos de alteração de dados em BD (como os realizados pelos comandos SQL `update` e `delete`), basta refazermos corretamente o texto da string query do exemplo prévio utilizando o mesmo método empregado para a operação de `insert` (o método `executeUpdate()` da classe `PreparedStatement`), que permite a execução de alterações no BD.



Resumo

Nesta unidade, aprendemos a dar funcionalidades aos componentes da nossa tela. Quando um usuário interage com um sistema, ele está ativando algum evento, de forma que na linguagem Java existe para cada tipo de evento uma interface específica que determina a sua funcionalidade. Vimos que essas ativações ocorrem ao adicionarmos a interface ao componente que se quer controlar.

Além de adicionarmos o evento ao componente, codificamos todo o controle das ações nos métodos implementados a partir das interfaces. Os eventos de interação (usuário x sistema) são representados na linguagem Java por interfaces que conhecemos como Listeners. Aprendemos que é possível controlar os seguintes tipos de eventos: de janela, de mouse, de teclado, e de ação (quando o usuário clica ou aciona um componente específico na tela), ou seja, tudo aquilo que utilizamos para interagir com um sistema. Um sistema construído na linguagem Java está preparado para acionar um método específico quando o usuário interage com o sistema.

Verificamos também que, para que o sistema se conecte com um BD específico (que nos exemplos deste livro-texto conectamos com o SGBD MySQL), precisamos de uma biblioteca de classes, geralmente fornecida pela empresa proprietária do BD (isto é, a companhia que criou o MySQL), biblioteca essa fornecida na forma de um arquivo com extensão .jar, que adicionamos ao projeto. Consequentemente, elaboramos uma série de exemplos que permitem: conectar com o BD; ler as informações de uma tabela do banco; inserir ou alterar informações naquela tabela, ações essas feitas a partir de queries (no formato SQL), geradas em strings e enviadas pelos statements, para execução direta no BD.

Vimos, ainda, que as interfaces gráficas nos possibilitam construir sistemas que permitem a interação do usuário com o sistema construído. Na construção das interfaces, existem duas grandes bibliotecas (conjuntos de classes do Java) que o desenvolvedor pode utilizar, em que encontra componentes usuais, já com suas funcionalidades próprias, além dos elementos que possibilitam o funcionamento geral da interação, como os possíveis eventos a serem acionados, dependendo da ação do usuário. Entendemos que cada elemento visual é considerado um componente, sendo a utilização e a disponibilização desses componentes os campos de texto, os campos de senha, os labels, os botões, assim como o frame, ou o panel, que são considerados containers, já que podem receber internamente

outros componentes. Desta forma, a fim de verificar como os componentes seriam inseridos ou disponibilizados nesses containers, observamos as definições dos possíveis layouts que cada container poderia assumir. Para muitos elementos básicos utilizados na construção das interfaces gráficas, existem algumas funcionalidades ligadas a eles (que podem ser acionadas pelo usuário), as quais chamamos de eventos, mas que na linguagem Java usam outros elementos básicos conceituais, como as interfaces e seus métodos abstratos.

Por fim, outros elementos de tela que apresentamos foram os componentes de janela, as tabelas e os menus, componentes esses que têm uma particularidade em sua funcionalidade. Para as tabelas, trabalha-se basicamente com três classes, sendo a `Table` aquela que a define como um todo, a `DefaultTableModel`, que permite a manipulação dos dados da tabela, e a `JScrollPane`, que é um painel (container) que receberá a tabela como um todo e tem a barra de rolagem para melhor visualização das informações dela. Quanto aos elementos de menus, exibimos as três classes básicas: a barra de menu (`MenuBar`), que representa a barra superior de menus de uma janela; os menus (`Menu`), que nas janelas fornecem outras opções de menus; e os itens de menus (`MenuItem`), que, ao serem acionados, geram (ou iniciam) alguma ação no sistema.



Exercícios

Questão 1. Os eventos são as possíveis ações de interação do usuário com o sistema que ele está utilizando. Eles podem acontecer pelo acionamento de um botão ou pela passagem do mouse sobre algum componente, entre outros. A linguagem Java controla os eventos por meio de interfaces denominadas listeners, criadas para serem implementadas pelas classes de componentes cujos eventos precisam ser controlados.

A respeito desse tema, avalie as afirmativas a seguir.

I – A interface `KeyListener` define métodos para o reconhecimento de quando uma tecla foi pressionada, solta ou digitada.

II – A interface `MouseListener` define métodos para o reconhecimento de quando o mouse teve seu botão clicado, entrou em um componente, saiu de um componente, e teve seu botão pressionado ou seu botão solto.

III – A interface `WindowListener` define métodos para o reconhecimento de quando um componente foi adicionado ou removido de um container.

É correto o que se afirma em:

A) I, apenas.

B) II, apenas.

C) I e III, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa A.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: a interface `KeyListener` é utilizada para monitorar eventos que são disparados pelo pressionamento, pela liberação ou pela digitação de uma tecla qualquer. Ou seja, ela monitora o uso do teclado.

II – Afirmativa incorreta.

Justificativa: a interface `MouseWheelListener` define um método para o reconhecimento de quando a roda do mouse foi movida. O texto da afirmativa descreve a interface `MouseListener`.

III – Afirmativa incorreta.

Justificativa: a interface `WindowListener` define métodos para o reconhecimento de quando uma janela foi ativada, fechada, desativada, desiconificada, iconificada, aberta ou abandonada. O texto da afirmativa descreve a interface `ContainerListener`.

Questão 2. (Consulplan/2017, adaptada) A maioria das aplicações, independentemente do porte, utiliza a persistência de dados atrelada aos seus sistemas. O banco de dados se faz necessário em uma aplicação não apenas para persistir as informações, mas para que nos comuniquemos com ele para recuperar, modificar e apagar informações. Portanto, podemos afirmar que o gerenciamento dos dados é de fundamental importância ao correto funcionamento da aplicação.

Quando se trata de persistência de dados em Java, geralmente a forma utilizada para guardar dados é um banco de dados relacional. A fim de abrir uma conexão com um banco de dados, precisamos utilizar sempre um driver. A classe `DriverManager` é responsável por realizar essa comunicação, e a API do Java `Java DataBase Connectivity (JDBC)` fornece a especificação de como a linguagem Java se comunicará com um banco de dados. O parâmetro passado é do tipo `String`, contendo a URL para localizar o banco de dados que, por sua vez, contém as informações para a conexão com o banco de dados.

Assinale a alternativa correta que contém os parâmetros URL padrão, a senha e o login para a conexão com o banco `MySQL`, por meio do comando `DriverManager.getConnection()`, considerando que o usuário, o servidor e a porta do banco são padrão, o banco não tem senha e o nome do banco de dados é teste.

A) `("mysql:jdbc://localhost:3306/teste";,"root")`.

B) `("jdbc:mysql://localhost:3306/teste";root,"")`.

C) `("mysql:jdbc://localhost:3306/teste";root,"")`.

D) `("jdbc:mysql://localhost:3306/teste";,"root")`.

E) `("root";jdbc:mysql://localhost:3306/teste";")`.

Resposta correta: alternativa B.

Análise da questão

Com o objetivo de abrir uma conexão com um BD, precisamos utilizar sempre um driver. A classe `DriverManager` é a responsável por se comunicar com todos os drivers disponíveis. Para isso, utilizamos o método `getConnection()`.

Os parâmetros do comando `DriveManager.getConnection()` utilizam a sintaxe a seguir.

```
DriverManager.getConnection("URL","login","senha")
```

Vamos, agora, comentar cada um dos parâmetros.

- **Parâmetro URL:** uma URL básica de conexão entre a linguagem Java e o BD MySQL tem o seguinte formato, considerando servidor e porta do banco padrão:

```
jdbc:mysql://localhost:3306/nome do banco de dados
```

Como o nome do BD da questão é teste, temos o formato a seguir:

```
jdbc:mysql://localhost:3306/teste
```

- **Parâmetro login:** trata-se do nome do usuário. Ao instalar o MySQL, o nome padrão do usuário é root.
- **Parâmetro senha:** trata-se da senha de acesso ao BD. De acordo com o enunciado, o banco não tem senha.

Logo, a instrução completa a ser inserida nos parênteses da instrução `DriveManager.getConnection()` é dada de acordo com o exposto a seguir.

```
"jdbc:mysql://localhost:3306/teste","root",""
```

[illegible]

Unidade II

5 DESIGN PATTERNS (PADRÕES DE PROJETOS)

No desenvolvimento de um sistema, espera-se que alguns requisitos sejam garantidos, por exemplo: o desempenho do programa (a eficiência e a eficácia do sistema, ou seja, que ele responda às necessidades do cliente e que funcione corretamente), a sua compreensão (a facilidade na manutenção por parte dos desenvolvedores), na utilização por parte dos usuários e na reutilização (à medida que a equipe de desenvolvimento possa fazer sem problemas uma implementação no sistema).



Saiba mais

Para aprender mais sobre design patterns, que trata entre outros assuntos acerca de padrões de projeto, leia o capítulo 9 da obra a seguir:

HORSTMANN, C. S.; CORNELL, G. *Core Java. Volume I: fundamentos*. 8. ed. São Paulo: Pearson Prentice Hall, 2010.

Os design patterns surgiram como um meio de agilizar e melhorar os processos de criação e de manutenção de sistemas a fim de facilitar o trabalho dos desenvolvedores, tornando-os mais rápidos e permitindo preços mais competitivos.

Assim, a partir das melhores práticas, reuniram-se as mais adequadas e eficazes soluções, transformando-as em padrões de desenvolvimento (design patterns). Alguns deles até se tornaram frameworks (ou seja, programas ou pacotes com bibliotecas prontas, com códigos que fornecem alguma funcionalidade ou solução específica).



Observação

É importante entendermos que design patterns são procedimentos, ou, ainda, formas e padrões de conduzir o desenvolvimento de um sistema para que haja mais agilidade e garantias de um trabalho eficaz (que seja bem feito e funcional), melhorando o trabalho do desenvolvedor, além da satisfação do cliente.

A ideia de seguir esses padrões (patterns) de desenvolvimento (de design) é conseguir trabalhar com mais certeza para produzir sistemas funcionais e manuteníveis (que permitam fácil manutenção).

Devemos lembrar que um sistema que esteja sendo construído hoje "por mim", poderá ser alterado ou implementado por outro profissional (daqui a algum tempo). Desta forma, utilizando-se de padrões, permite-se que o profissional que trabalhará sobre o "meu sistema" tenha um entendimento mais ágil, já que ele ainda não o conhece.

5.1 DTO/VO

O pattern Data Transfer Object (DTO) – ou ainda Objeto de Transferência de Dados – indica que quando vamos criar um método que receberá informações diversas em seus parâmetros, a maneira mais eficaz de fazê-la é utilizando um objeto no qual esses dados estão nos atributos do objeto.

Assim, com esse pattern, um objeto é utilizado para transferir os dados (as informações) de um local a outro (ou seja, quando um método de uma classe chama o método de outra). Isso é muito comum quando precisamos transferir dados entre a camada de visualização (view layer) e a camada de persistência dos dados (model layer) (sobre elas, estudaremos posteriormente quando virmos o pattern MVC – item 5.2).

Com o pattern DTO, um objeto (geralmente encapsulado) é preenchido com dados em seus atributos, e ele é transportado por entre as camadas de um MVC. Tal objeto é considerado um JavaBean, ou seja, uma classe encapsulada simples que representa uma entidade do BD (que pode ser tabela, view etc.).

Essa transferência de informação é na verdade a chamada de um método de uma classe a partir do método de outra (método de uma classe acionando o método de outra classe). Desta forma, tem-se uma classe com um método que inicialmente monta as informações em um objeto, preenchendo seus atributos, e que em seguida chama um método de outra classe, enviando aquele objeto como parâmetro de invocação do método. Trata-se da forma como uma classe se comunica com outra enviando ou recebendo informações.



Lembrete

Um método pode ter como parâmetro (ou ainda, receber como parâmetro) uma ou mais classes, de forma que os objetos enviados podem conter informações nos seus atributos.

Como exemplo, a classe a seguir representa uma pessoa qualquer, que contém atributos que são as informações sobre ela. Essa classe pode, por exemplo, ser utilizada em um programa de inscrição de indivíduos para um curso, de forma que cada objeto gerado em memória poderá conter os dados sobre alguém que está se inscrevendo no curso:

```
public class Pessoa {  
    public String nome;  
    public String cpf;  
    public int idade;  
    public boolean cursoPago;
```

```
// métodos da classe Pessoa...
//...
}

public class Curso {
    public String nome;
    public String codigo;
    public double valorMensal;

    public static void inscrever(Pessoa p1) {
        // Lógica de inscrição (enviando dados ao BD)...
    }
    // demais métodos da classe Curso...
    //...
}
```

Percebam que na classe `Curso`, o método `inscrever` tem como parâmetro um objeto da classe `Pessoa` (representado pelo objeto `p1`), de forma que ao chamá-lo, devemos já ter um objeto do tipo `Pessoa` com seus dados (atributos) preenchidos, os quais poderão ser utilizados para gerar o cadastro daquele aluno (que é a pessoa) no curso, para então gerar um registro no BD.

Essa forma de envio de informação é muito mais prática e fácil do que se tivéssemos que fazê-la com as informações da pessoa como se fosse um parâmetro do método de inscrição (que deveria ter muitos parâmetros).

5.2 MVC

O Pattern Model-View-Controller (MVC) ou, ainda, arquitetura MVC, é também uma arquitetura, já que seu objetivo é organizar um projeto em pacotes, os quais denominaremos aqui como camadas.

Nessa arquitetura, o sistema é dividido em três camadas (ou três pacotes, em que cada um deles conterá classes relacionadas às suas características) a fim de separar as classes por responsabilidades, de forma que:

- As classes responsáveis pelas telas (montagem das interfaces com o usuário) ficam na camada de visualização (camada **view**).
- As classes responsáveis pelo processo de persistência (acesso a BD) devem ficar na camada de modelagem (camada **model**).
- Na camada de controle (camada **control**) ficam as classes responsáveis pelo controle das informações que tramitam (que são transportadas) entre as camadas de visualização e de modelagem (ou seja, entre a tela que o usuário está preenchendo e o BD). Nela, são realizadas as operações de validação, controle e encaminhamento das informações.

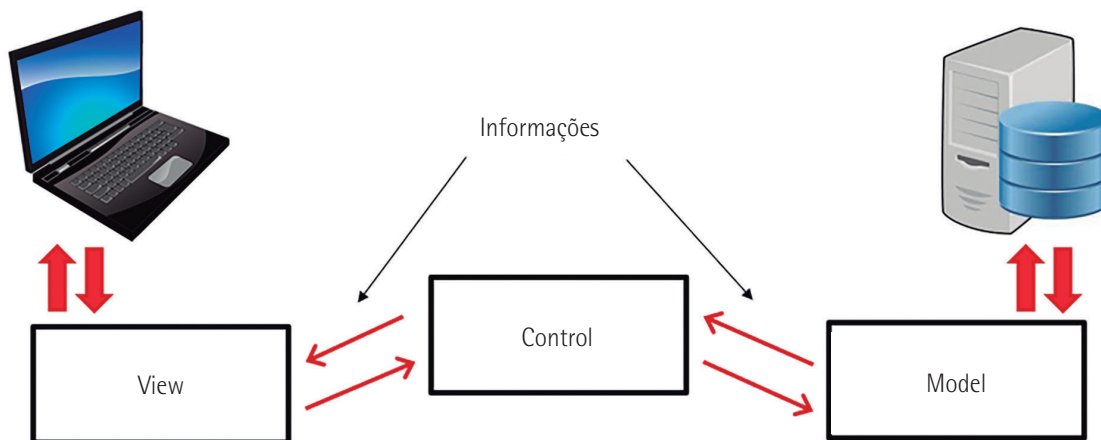


Figura 17 – Representação do fluxo de informação do design pattern MVC

Adaptada de: <https://bit.ly/3bAnhrk> e <https://bit.ly/3P78iCH>. Acesso em: 25 jul. 2022.

Cada um dos retângulos da figura anterior representa uma camada do pattern MVC. As setas indicam o fluxo das informações entre as camadas (quando um método aciona outro).

Observação

Quando dizemos que as informações são transmitidas entre os métodos, estamos apenas tratando acerca da forma como eles recebem e devolvem informações de/para quem (ou qual método) chama aquele método.

Quando um método chama outro, o primeiro deles envia informações na forma de parâmetros, os quais são recebidos e processados pelo método que está sendo chamado.

Um método devolve alguma informação a partir da definição do seu retorno, ou seja, é definido pelo tipo de retorno que o método chamado tem. No entanto, sabemos que qualquer método pode ser construído para receber e retornar classes, ou melhor, objetos (instanciados) que contêm dados em seus parâmetros.



Lembrete

É importante lembrarmos da sintaxe da declaração de um método, que é:

```
modificadores tipoRetorno nomeMetodo (parametros) {  
    //...lógica de ação do método...  
}
```

Assim, um método recebe dados ao ser chamado, de forma que ficam descritos nos parâmetros, devolvendo uma informação na variável que será retornada, de acordo com o tipo de retorno definido na declaração do método.

5.3 Data Access Object (DAO)

Em geral, todo e qualquer sistema necessita da persistência de alguma informação. Os DAO são os objetos que acessam dados em comunicação com o BD. Eles, por convenção geral, possuem a sigla DAO no nome da classe (por exemplo: classe AlunoDAO, ou ProdutoDAO etc.).

Segundo o pattern MVC, a camada de modelagem será a responsável pela persistência dos dados, direto do, ou para o, BD.

Assim, as classes da camada model, que ficarão responsáveis por preparar a comunicação direta com o BD, são as classes DAO. Elas ficam encarregadas por criar corretamente as queries (SQL) que o BD receberá para manipular as informações.

6 O HIBERNATE

O Hibernate é uma ferramenta de consulta ou ainda de persistência objeto/relacional de alta performance. Trata-se de uma solução ORM (de mapeamento objeto/relacional) muito flexível e poderosa, que faz o mapeamento de classes Java em relação às tabelas de um BD, e de tipos de dados Java para tipos de dados SQL. Ele fornece consultas e facilidades para o retorno dos dados que reduzem significativamente o tempo de desenvolvimento.

O Hibernate abstrai o seu código SQL de forma que toda a camada JDBC e SQL será gerada em tempo de execução. E vai além, ele gera o SQL que servirá para um determinado BD, já que cada BD utiliza um dialeto diferente dessa linguagem (possui diferenças, às vezes, significativas). Assim, há também a possibilidade de troca de BD sem precisar alterar o código Java, uma vez que isso fica como responsabilidade da própria ferramenta.



Saiba mais

Compreenda mais sobre a utilização do Hibernate acessando o texto a seguir:

UTILIZANDO o Hibernate em uma aplicação Java Swing. *Apache NetBeans*, [s.d.]. Disponível em: <https://tinyurl.com/yeyymjkm>. Acesso em: 28 jul. 2026.

Usando a classe a seguir (classe Tarefa) como exemplo, vamos configurá-la (mapeá-la) para que o Hibernate consiga identificar onde e quais serão seus campos e colunas.

Inicialmente veremos a classe Tarefa sem o mapeamento (mas que representa um item de uma tabela, cujo nome é Tarefa, no BD).

```
public class Tarefa {  
  
    private Long id;  
    private String descricao;  
    private boolean finalizado;  
    private Calendar dataFinalizacao;  
    // Métodos Setters e Getters  
    // ...  
}
```

A classe anterior é encapsulada como qualquer outra que já aprendemos a escrever em Java. Precisamos então configurá-la (mapeá-la) para utilizar o Hibernate, para que ele saiba da existência dessa classe e, desta forma, entenda que deve inserir uma linha (um dado) na tabela Tarefa, toda vez que for requisitado, fazendo com que um objeto desse tipo seja salvo.

Para mapear a classe Tarefa, basta adicionar algumas poucas anotações em nosso código. Anotação é um recurso do Java que permite inserir metadados em relação a nossa classe, atributos e métodos. Essas anotações depois poderão ser lidas por frameworks e bibliotecas (como o Hibernate, por exemplo), para que eles tomem decisões baseadas nessas pequenas configurações.

Mapeando a classe Tarefa:

```
@Entity  
public class Tarefa {  
  
    @Id  
    @GeneratedValue  
    private Long id;  
  
    private String descricao;  
    private boolean finalizado;  
  
    @Temporal(TemporalType.DATE)
```

```
private Calendar dataFinalizacao;  
  
// Métodos Setters e Getters  
// ...  
}
```

- A anotação `@Entity` indica que os objetos dessa classe se tornarão persistentes no BD.
- A anotação `@Id` indica que o atributo `id` será nossa chave primária (é preciso ter uma chave primária em toda entidade).
- A anotação `@GeneratedValue` indica que queremos que essa chave seja populada e criada pelo banco (isto é, ela será utilizada como um auto increment, ou sequence, dependendo do BD).
- A anotação `@Temporal` configura um mapeamento do tipo `Calendar` para o BD, de forma que aqui usamos apenas como `data` (sem a hora), mas poderíamos fazê-lo também como apenas hora (`TemporalType.TIME`) ou ainda como `timestamp` (`TemporalType.TIMESTAMP`), dependendo da necessidade.



Observação

A anotação `@Id` não recebe esse nome porque o nome do atributo é `id`, o que significa que, apesar de esse nome representar a chave primária, muitas vezes ele pode ser: `id`, `cod`, `num`, `cp`, `pk` etc. Independentemente disso, o nome da anotação será sempre `@Id`.

Essas anotações precisam dos devidos imports, que pertencem ao pacote `javax.persistence`. Mas em que tabela essa classe será gravada? Em quais colunas? Que tipo de coluna?

Na ausência de configurações mais específicas, o Hibernate vai usar convenções: a classe `Tarefa` será gravada na tabela de nome também `Tarefa`, e o atributo `descrição` em uma coluna de nome `descricao`.

Se quisermos configurações diferentes das convenções, basta usarmos outras anotações, que são completamente opcionais.

Por exemplo, para mapear o atributo `dataFinalizacao` em uma coluna chamada `data_finalizado`:

```
"...  
@Column(name="data_finalizado", nullable=true)  
private Calendar dataFinalizacao;  
..."  
  
Para usar uma tabela com o nome tarefas:  
"  
@Entity  
@Table(name="tarefas")  
public class Tarefa {  
..."
```

Repare que nas entidades há todas as informações sobre as tabelas. Baseado nelas, podemos até pedir para gerar as tabelas no banco, mas para isso é preciso configurar o JPA.

Configurando as propriedades do banco

Em qual BD vamos gravar nossas tarefas? Qual é o login? Qual é a senha?

O JPA necessita dessas configurações, de forma que para isso criaremos o arquivo persistence.xml.

Com o objetivo de configurar a conexão com o BD, o Hibernate precisa saber como se conectar a ele, e isso é feito através do arquivo persistence.xml, que deve ficar dentro de uma pasta nomeada como META-INF, localizada na raiz do projeto (criada enquanto se está programando o sistema).

Para este nosso sistema, precisamos de quatro linhas com configurações que já conhecemos do JDBC: a string de conexão com o banco, o driver, o usuário e a senha que permitem acessar o BD. Além disso, temos de dizer qual dialeto SQL deverá ser utilizado no momento em que as queries forem geradas automaticamente pelo Hibernate.

O exemplo a seguir mostra a configuração caso utilizemos o MySQL como SGBD. Ele está no formato xml (arquivo .xml).

```
"...
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.0"
  xmlns="http://java.sun.com/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd" >

  <persistence-unit name="hibAlunos">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>Aluno</class>
    <properties>
      <property name="javax.persistence.jdbc.driver" value="com.mysql.
cj.jdbc.Driver" />
      <property name="javax.persistence.jdbc.url" value="jdbc:mysql://
localhost:3306/testehiber?useTimezone=true&serverTimezone=UTC" />
      <property name="javax.persistence.jdbc.user" value="root" />
      <property name="javax.persistence.jdbc.password" value="1234" />

      <property name="hibernate.dialect" value="org.hibernate.dialect.
MySQL8Dialect" />
      <property name="hibernate.max_fetch_depth" value="3" />

      <property name="hibernate.hbm2ddl.auto" value="update" />

      <property name="hibernate.show_sql" value="true" />
    </properties>
  </persistence-unit>
</persistence>
..."
```

Os itens de maior atenção desse arquivo são:

- **<persistence-unit name="nome">**: configura o nome do contexto que será utilizado pela aplicação.

As propriedades a seguir definem as configurações de acesso ao BD

- **javax.persistence.jdbc.driver**: nome completo da classe do driver JDBC do BD
- **javax.persistence.jdbc.user**: nome do usuário que será utilizado para estabelecer a conexão com o BD.
- **javax.persistence.jdbc.password**: senha do usuário.
- **javax.persistence.jdbc.url**: string de conexão com o BD.
- **hibernate.dialect**: configura o dialeto que o Hibernate utilizará para a montagem dos comandos SQL.

Comunicando-se com o banco

O primeiro passo é fazer com que o JPA leia a nossa configuração, tanto o arquivo persistence.xml quanto as anotações que colocamos na entidade Tarefa. Para tal, usaremos a classe com o mesmo nome do arquivo XML: Persistence. Ela é a responsável por carregar o XML e inicializar as configurações. O resultado dessa configuração é uma EntityManagerFactory:

```
EntityManagerFactory factory = Persistence.createEntityManagerFactory("tarefas");
```

Antes de gravar uma tarefa, precisamos que exista uma tabela correspondente no nosso BD. Em vez de criarmos o script que define o schema (ou data definition language – DDL – de um banco) do nosso banco (os famosos CREATE TABLE), podemos deixar isso a cargo do próprio Hibernate. Ao inicializarmos a EntityManagerFactory também será gerada uma tabela Tarefas, pois configuramos que o banco deve ser atualizado pela propriedade do Hibernate: hbm2ddl.auto.

6.1 Persistindo dados com o Hibernate

Persistindo objetos

Para facilitar a criação de objetos EntityManager, foi construída a classe EntityManagerProvider, que encapsula todo o trabalho necessário à criação de tais objetos, classe que está demonstrada na sequência:

```
...  
package hiber;
```

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class EntityManagerProvider {
    private static EntityManagerFactory emf = null;
    private EntityManagerProvider() {}

    }

    public static EntityManager getEntityManagerInstance() {
        if (emf == null) {
            emf = Persistence.createEntityManagerFactory("hibAlunos");
        }
        return emf.createEntityManager();
    }
}
..."
```

O parâmetro utilizado na criação do objeto tem o valor artigo (ou tarefa). Esse é exatamente o nome definido na propriedade persistence-unit do arquivo persistence.xml, ou seja, é através deste parâmetro que o Hibernate entende em que BD deve estabelecer a conexão, e com quais parâmetros.

A classe EntityManager disponibiliza métodos pelos quais os objetos podem ser manipulados.

- **Método remove:** exclui do BD o registro na tabela referente ao objeto passado como parâmetro.
- **Método createQuery:** permite a consulta de dados persistidos no BD, recuperando-os na forma de objetos.
- **Método persist:** envia efetivamente os dados do objeto criado ou alterado para o BD.
- **Método find:** realiza a busca por um objeto através de seu identificador.
- **Método getTransaction:** obtém a referência de um objeto do tipo EntityTransaction, o qual permite o controle das transações.
- **Método begin:** indica o início de um bloco que deve ser controlado em um contexto transacional.
- **Método commit:** possibilita efetivar todas as ações executadas no BD, desde a execução do método begin.
- **Método rollback:** propicia desfazer todas as ações executadas no BD, desde a execução do método begin.

Apesar de a classe EntityManager possuir métodos bem definidos para a manipulação dos objetos, ainda existe a necessidade de iniciar a transação, executar a ação desejada sobre o objeto e encerrar a transação, confirmando ou cancelando as ações.



Resumo

Nesta unidade, vimos que os padrões de desenvolvimento, mais conhecidos como design patterns, surgiram das melhores práticas realizadas ao longo do desenvolvimento (criação ou implementação) em sistemas de software à medida que se identificava um processo que aumentava muito a eficiência da codificação. Além da eficácia do produto gerado, esse processo e todo o conhecimento adquirido em torno dele tornaram-se um padrão de desenvolvimento a ser seguido pelos desenvolvedores da linguagem.

Entendemos que alguns padrões estão ligados à forma básica de se codificar (como o DTO, que é um padrão que define como as informações devem ser enviadas através das classes ao acionarmos seus métodos). Outros padrões, como o MVC e o DAO, definem qual a melhor maneira de organizar um sistema quando o projeto prevê uma interação entre o usuário e um BD, de forma que o sistema que permitirá tal interação deverá ser criado em camadas, e cada uma delas terá uma finalidade no sistema.

Por fim, demonstramos a existência de padrões na codificação básica e na construção (na arquitetura) do projeto, sendo que em alguns casos um determinado padrão é tão específico que se transforma em um sistema à parte (framework).

Nesse sentido, criou-se um framework conhecido como Hibernate, que é um sistema instalável, gerado, e muito utilizado, para facilitar e padronizar a persistência de informações em BD. Para utilizarmos o framework, precisamos instalá-lo, configurá-lo no projeto, e mapear as classes que representarão as entidades dos bancos de dados e que conterão as informações trazidas pelo ou a serem levadas ao BD.



Exercícios

Questão 1. (IADES/2018, adaptada) A sigla do padrão de projeto MVC vem do inglês Model-View-Controller. A respeito dele, avalie as afirmativas a seguir:

I – A camada model é responsável pela exibição de dados na tela.

II – A camada view é responsável pelas regras de negócio.

III – A comunicação entre as interfaces e as regras de negócio é responsabilidade da camada controller.

É correto o que se afirma em:

A) I, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa B.

Análise das afirmativas

I – Afirmativa incorreta.

Justificativa: o MVC é um padrão de arquitetura de software focado na separação de conceitos em três camadas interconectadas. Enfim, nela, o sistema é dividido em três camadas, sendo que cada uma delas conterá classes relacionadas às suas características, a fim de separá-las por responsabilidades. A camada model, responsável pela modelagem da aplicação, gerencia o comportamento dos dados por meio de regras de negócios, de regras lógicas e de funções. Dessa forma, ela modela o problema que se pretende resolver com a aplicação. As classes responsáveis pelo processo de persistência (acesso ao BD) devem ficar na camada model.

II – Afirmativa incorreta.

Justificativa: a camada view, de visualização, é responsável pela exibição dos dados oriundos do model. Portanto, ela é utilizada para apresentar as informações ao usuário, utilizando combinações de gráficos e textos. As classes responsáveis pelas telas (montagem das interfaces com o usuário) ficam na camada view.

III – Afirmativa correta.

Justificativa: a camada controller tem foco na interação do usuário, sendo responsável pela mediação entre as interfaces com o usuário e as regras de negócio. Ela interpreta as ações recebidas pelos periféricos de entrada (como teclado ou mouse), vindas do usuário, e mapeia essas ações, que são enviadas para o model ou para o view. Aqui, ficam as classes responsáveis pelo controle das informações transportadas entre as camadas view e model.

Questão 2. (CCV-UFC/2019) Leia o texto a seguir, a respeito da ferramenta Hibernate.

Com a popularização do Java em ambientes corporativos, logo se percebeu que grande parte do tempo do desenvolvedor era gasto na codificação de queries SQL e no respectivo código JDBC responsável por trabalhar com elas.

Além de um problema de produtividade, algumas outras preocupações aparecem: SQL, apesar de ter um padrão ANSI, apresenta diferenças significativas dependendo do fabricante. Não é simples trocar um banco de dados por outro. Há ainda a mudança do paradigma. A programação orientada a objetos difere muito do esquema entidade relacional, e precisamos pensar das duas maneiras para fazer um único sistema. Para representarmos as informações no banco, utilizamos tabelas e colunas. As tabelas geralmente possuem chave primária e podem ser relacionadas por meio da criação de chaves estrangeiras em outras tabelas. Quando trabalhamos com uma aplicação Java, seguimos o paradigma orientado a objetos, onde representamos nossas informações por meio de classes e atributos. Ademais, podemos utilizar herança, composição para relacionar atributos, polimorfismo, enumerações, entre outros. Essa diferença entre esses dois paradigmas gera bastante trabalho: a todo momento, devemos transformar objetos em registros e registros em objetos.

Ferramentas para auxiliar nesta tarefa tornaram-se populares entre os desenvolvedores Java e são conhecidas como ferramentas de mapeamento objeto-relacional (ORM). O Hibernate é uma ferramenta ORM open source e é a líder de mercado, sendo a inspiração para a especificação Java Persistence API (JPA). Ele nasceu sem JPA, mas, hoje em dia, é comum acessar o Hibernate pela especificação JPA. Como toda especificação, ela deve possuir implementações. Entre as implementações mais comuns, podemos citar: Hibernate da Red Hat, EclipseLink da Eclipse Foundation e o OpenJPA da Apache.

Apesar de o Hibernate ter originado a JPA, o EclipseLink é a implementação referencial. O Hibernate abstrai o seu código SQL. Assim, toda a camada JDBC e o SQL serão gerados em tempo de execução. Mais que isso, ele vai gerar o SQL que serve para um determinado banco de dados, já que cada banco fala um dialeto diferente dessa linguagem. Assim, há também a possibilidade de trocar de banco de dados sem ter de alterar o código Java, já que isso fica como responsabilidade da ferramenta.

Adaptada de: <https://bit.ly/3zxAqta>. Acesso em: 1º jul. 2022.

Sobre o framework Hibernate, avalie as afirmativas a seguir.

I – O Hibernate é uma solução tecnológica para o mapeamento objeto-relacional (ORM) que aceita o uso da Java Persistence API (JPA).

II – Devido à utilização do Hibernate, o Java Database Connectivity (JDBC) não é mais utilizado nas aplicações, tendo sido descontinuado.

III – O objetivo do Hibernate é fornecer uma forma de mapeamento dos objetos Java para o banco de dados apenas para fins de consulta das informações.

IV – A Java Persistence API (JPA) substitui todas as funções disponibilizadas pelo Hibernate, tornando-o desnecessário e obsoleto.

É correto apenas o que se afirma em:

A) I.

B) III.

C) I e II.

D) I e IV.

E) II, III e IV.

Resposta correta: alternativa A.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: de acordo com o texto do enunciado, o Hibernate é uma ferramenta ORM open source, que inspirou a especificação JPA. Hoje, é comum o utilizarmos por meio da JPA. Logo, é correto afirmar que o Hibernate é uma solução para o mapeamento objeto-relacional que aceita o uso da Java Persistence API (JPA).

II – Afirmativa incorreta.

Justificativa: o JDBC é uma API de conexão com um BD, sendo a API o padrão que aplicações Java utilizam para interagir com um BD. O Hibernate, por sua vez, é um framework para o mapeamento objeto-relacional, que utiliza o JDBC para interagir com um BD. Desse modo, não é correto dizer que o JDBC foi descontinuado devido à utilização do Hibernate.

Unidade III

7 INTRODUÇÃO A APLICAÇÕES WEB – JSP E JSTL

O que é uma página Java Server Pages (JSP)?

JSP é um arquivo codificado com código HTML juntamente aos códigos em Java. Uma página escrita em JSP (ou ainda, uma JSP) possui a forma de uma página HTML com trechos de código Java embutidos, além de outras tags especiais.

A JSP simplifica a geração de conteúdo dinâmico para web designers e tem o objetivo de facilitar a construção de páginas dinâmicas, ela é automaticamente transformada (pelo servidor de aplicação) em um servlet no qual a parte dinâmica da página é gerada pelo código Java.

De uma forma geral, a JSP é uma tecnologia com padrões baseados em templates para servlets. O mecanismo de tradução de página e códigos script está embutido (é feito automaticamente) no servidor de aplicação.

Servlets

Os servlets (para o nosso caso) são linhas de códigos em Java que forçam o programador a embutir código HTML dentro de um código (na string de impressão na tela).

Trabalhar com servlets acaba sendo desvantajoso, se a maior parte do que deve ser gerado for texto ou código HTML estático (como é para uma página web informativa com informações que nunca se alteram).

- **Mistura de conceitos:** a fim de utilizar os servlets, o programador Java precisa ser um bom web designer, para criar páginas completas sem se utilizar das ferramentas de web design, já que ele vai embutir o código HTML na string de um código Java.

7.1 Servidores de aplicação

As páginas JSP podem ser rodadas em servidores de aplicação Java, como o Apache/Tomcat. Um servidor de aplicação é um programa servidor fornecedor de serviços que permite a disponibilização de sistemas construídos (criados) em linguagens dinâmicas (por exemplo: em JSP).

No caso do servidor utilizado neste curso, o Apache é um servidor http (servidor web) e o Tomcat é um servidor de aplicações que possibilita o acionamento de sistemas (que foram criados para rodar em servidores e disponibilizados para vários usuários) em Java.

Existem outros servidores de aplicação, tais como:

- JBoss (Red Hat JBoss Middleware).
- IBM WebSphere.
- GlassFish Server.
- Jetty.

A sugestão para este curso é a utilização do servidor Apache Tomcat, cuja instalação pode ser verificada no **Apêndice B** deste livro-texto.

Como rodar uma página JSP?

As páginas JSP podem ser rodadas a partir dos servidores de aplicação Java, como o Apache/Tomcat.



Observação

Apache é o servidor http (servidor web – que fornece serviços web), enquanto Tomcat é o servidor de aplicações.

7.2 Um pouco de HyperText Markup Language (HTML)

O HTML é uma linguagem de marcação de hipertexto. Ela é utilizada na produção de documentos (páginas) web. Essa linguagem permite a criação de documentos que podem ser lidos em praticamente qualquer tipo de computador (desde que o micro possua um browser instalado). Seus documentos, ou páginas, são transmitidos pela internet, inclusive por correio eletrônico (e-mail).

Para escrever documentos (ou páginas) HTML, é necessário apenas um editor de texto simples (como o Bloco de Notas), além do conhecimento de suas tags. As tags são os marcadores, ou seja, textos que marcam e definem um espaço que conterá uma formatação ou uma função específica na página. São os elementos que compõem as linguagens HTML, XHTML e HTML5.

Desta forma, as tags servem para indicar e aplicar a função de cada elemento no conteúdo descrito naquela página. Uma tag estará sempre escrita com os sinais "<" e ">".

Em geral, cada tag é utilizada na página, de forma que ela deverá possuir um marcador indicando o início da área de ação (<tag...>) e outro informando o fim de sua área de ação (</tag>), aplicando sua ação sobre um determinado conteúdo.

Por exemplo:

```
<html> ... Seu conteúdo ... </html>
```

Nesse exemplo, dizemos que o conteúdo está dentro da tag "<html>".

Estrutura da página web

Uma página web possuirá três tags básicas (<html>, <head> e <body>) na seguinte sequência:

```
<html>
  <head>
  </head>
  <body>
  </body>
</html>
```

Onde:

- **Tag <html> ... </html>:** define toda a página (seu início e seu fim).
- **Tag <head> ... </head>:** define o cabeçalho da página, que contém algumas características gerais da página web, e está dentro da tag html.
- **Tag <body> ... </body>:** define a área do conteúdo visível da página, e também está dentro da tag html.

Uma página web padrão terá a seguinte estrutura básica:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width" />
<title>Título da ABA</title>
</head>
<body>
<!-- Conteúdo -->
</body>
</html>
```

Onde:

O Doctype indica ao browser que tipo de documento mostrará, que no caso é um documento escrito em HTML. Ele não é uma tag, mas uma declaração e deve estar sempre no início de um documento HTML.

A meta tag charset define quais caracteres podem ser utilizados na página. O charset UTF-8 (para o HTML5) é o mais completo com mais de 10.000 caracteres diferentes (W3SCHOOLS, s.d.c.).



Observação

A meta tag é uma tag que pode conter dados de configuração e informações para mecanismos de busca.

Constam a seguir algumas tags iniciais:

- **<html> ... </html>**: define o início e o fim de uma página HTML.
- **<head> ... </head>**: define algumas características da página.
- **<body> ... </body>**: define o conteúdo visível da página.
- **<title> ... </title>**: define o título que aparece na aba da página, no browser (e deve estar declarado sempre dentro da tag head).
- **<!-- ... --> (de comentários)**: inserimos comentários em uma página a fim de explicar o seu conteúdo, para que o código da página seja compreendido entre os desenvolvedores. Sendo assim, essa explicação servirá aos desenvolvedores para entenderem o objetivo e a estrutura da página, isto é, os objetivos de seu conteúdo e a forma como ela foi construída.
- **<p> ... </p>**: define um parágrafo.
- **
**: inclui uma quebra de linha no ponto onde estiver localizada.
- **</h1>... </h1>, <h2> ... </h2> e <h6> ... </h6>**: definem áreas pré-formatadas para textos específicos, como os títulos de determinados conteúdos e mesmo notas de rodapé. A formatação desses headers (cabeçalhos) inclui desde textos maiores e com mais destaque (<h1> ... </h1> – para títulos principais) até textos bem pequenos (<h6> ... </h6> – para notas de rodapé). Observação: a formatação desses headers pode vir a ser alterada, porém, com a utilização do CSS.
- **<hr/>**: insere uma linha horizontal no ponto da página onde estiver localizada.

Constam a seguir algumas tags de formatação que ainda persistem na versão atual do HTML (o HTML5):

- ** ... **: deixa em negrito o conteúdo textual que estiver em seu interior.
- **<u> ... </u>**: deixa sublinhado o conteúdo textual que estiver em seu interior.
- **<i> ... </i>**: deixa em itálico (inclinado) o conteúdo textual que estiver em seu interior.



Saiba mais

Para compreender melhor sobre a programação em HTML e a utilização do CSS, veja o tutorial da W3C que possui todas as explicações de cada tag usada nos exemplos anteriores, além das demais tags da linguagem e das características de utilização do CSS. Acesse-o no link a seguir:

HTML tutorial. *W3Schools*, [s.d.]. Disponível em: <https://bit.ly/3cNYK1S>. Acesso em: 25 jul. 2022.

Já o tutorial sobre CSS da W3C pode ser encontrado em:

CSS tutorial. *W3Schools*, [s.d.]. Disponível em: <https://bit.ly/3S9RyOa>. Acesso em: 25 jul. 2022.

7.3 Salvando uma página como JSP

Uma JSP (uma página com código em JSP) deve ser salva com a extensão .jsp. Ela é um arquivo de texto simples e que possui a extensão .jsp, de forma que por ser um arquivo de texto pode ser aberta e editada a partir de um editor de texto simples (como o Bloco de Notas, ou o Notepad++, entre outros).

7.3.1 JSP e seus elementos

Vejamos agora a sintaxe dos elementos JSP, que podem ser utilizados em documentos misturados a códigos HTML ou XML. Todos eles serão interpretados pelo servidor de aplicação.

São eles:

- **Scriptlets:** `<% ... %>`
- **Expressões:** `<%= ... %>`
- **Declarações:** `<%! ... %>`
- **Diretivas:** `<%@ ... %>`

Scriptlets

Os scriptlets são blocos de código Java inseridos em uma página JSP, eles devem iniciar com `<%` e terminar com `%>`.

Por exemplo:

```
<%  
    ArrayList<String> lista = new ArrayList<String>();  
    lista.add("Algoritmos");  
    lista.add("LTP");  
    lista.add("Web");  
%>
```

Expressions (expressões)

Trata-se de um código Java inserido no JSP. Ele deve resultar em uma string, que será exibida na página resultante (em HTML) no lugar onde ela se encontra.

Uma expression deve iniciar com `<%=` e terminar com `%>`

O retorno de uma expression é o valor de uma variável ou o valor do retorno de uma função. Observação: neste caso, as linhas de código expressas não terminam com ponto e vírgula.

Por exemplo:

```
<%= nomeAluno %>  
<%= lista.get(0) %>
```

Declarations (declarações)

As declarações definem elementos de uma classe dentro de uma página web, são utilizadas para declarar (definir) atributos e métodos (como se a página fosse uma classe).

Uma declaration deve iniciar com `<%!` e terminar com `%>`

Por exemplo:

```
<%!  
    String nome;  
    double calculaSoma(int i, int j){  
        return i+j;  
    }  
%>
```

No trecho de código anterior, a declaration está definindo um atributo `nome` e um método `calculaSoma(...)`.

Diretivas

As diretivas começam com o símbolo `<%@` e terminam com `%>`

Existem três tipos de diretivas, são eles:

- **Include:** que inclui o código de um arquivo externo no JSP.
- **Taglib:** que define uma tag library.
- **Page:** que define propriedades da página.

Vejamos agora a explicação de cada um deles:

Diretiva include

Esta diretiva permite a inclusão de arquivos externos em JSP (páginas inteiras codificadas em outro arquivo). É ideal para incluir arquivos com informações que se repetem em várias páginas, como: menus e cabeçalhos.

Por exemplo:

```
<%@ include file="menu.jsp" %>
```

Neste caso, o código descrito no arquivo menu.jsp será incluído no ponto onde a diretiva estiver declarada.

Diretiva page

Ela define propriedades para a página JSP. Podemos estabelecer diversas propriedades (atributos):

```
language  
contentType  
pageEncoding  
import
```

Observação: os atributos language, contentType e pageEncoding definem a linguagem e a codificação da página, como elementos de configuração de páginas HTML.

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"  
pageEncoding="ISO-8859-1" %>
```

Diretiva page import

O atributo import é utilizado para importar classes que serão utilizadas pelos códigos de uma página JSP, ele funciona de modo semelhante ao import entre classes do Java.

Para a descrição da classe que será importada, devemos utilizar o pacote + nome da classe (ou pacote + .*, a fim de importar todas as classes do pacote).

Por exemplo:

```
<%@ page import="java.util.ArrayList, java.util.Calendar" %>
```

ou

```
<%@ page import="java.util.*" %>
```

7.3.2 Fazendo funcionar uma JSP

Antes de criar uma página JSP (para que funcione a partir do servidor de aplicação Apache/Tomcat), estude o **Apêndice B** deste livro-texto com o objetivo de verificar sua instalação, acionamento e funcionamento.

Neste item, vamos gerar uma página JSP simples, com um programa que escreve "Ola mundo" na página aberta no browser.

Elaboraremos uma página que escreverá o texto "Ola Mundo", porém a partir de um comando JSP. Devemos criar um arquivo de texto com a extensão .jsp (por exemplo, o arquivo olamundo.jsp, com o seguinte texto:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="iso-8859-1">
<title>Página Inicial</title>
</head>
<body>
<%
out.print("Ola Mundo!!!");
%>
</body>
</html>
```

Para testarmos a página anterior, podemos seguir os seguintes passos:

Acionar (se ainda não estiver acionado) o serviço do servidor de aplicação Apache Tomcat a partir da central de serviços do Windows (veja o item 13 do **Apêndice B** deste livro-texto).

Após iniciado o serviço, colocar o arquivo olamundo.jsp dentro do diretório webapps – ROOT que fica dentro do diretório de instalação do Tomcat (que por sua vez pode ter sido instalado no diretório programfile ou em arquivos de programas).

Com o arquivo .jsp já copiado dentro do diretório ROOT, abra um browser (que pode ser o Google Chrome) e digite no campo da url (dos endereços das páginas) o seguinte endereço:

<http://localhost:8080/olamundo.jsp>

Desta forma, todo arquivo JSP inserido no diretório ROOT (de instalação do Apache/Tomcat), poderá ser acessado a partir da url <http://localhost:8080/>.

Sendo assim, para saber se a página foi aberta corretamente, o que se verá nela é o texto "Ola Mundo!!!" aparecendo no canto superior esquerdo da tela do browser.

7.4 O JSP Standard Tag Library (JSTL)

O JSTL é uma biblioteca padrão de tags para funcionar com JSP; trata-se de uma coleção de bibliotecas (arquivos .jar que, por serem arquivos compactados, contêm uma série de outros arquivos com extensão .class, .xml, entre outras), de forma que cada uma delas (.jar) possui um objetivo definido, possibilitando escrever páginas JSPs (que possuem a dinâmica JSP, sem utilizar o código Java), deixando todos os elementos em forma de tags, aumentando assim a facilidade do design e da compreensão do código para os programadores web, promovendo a interação entre desenvolvedores e web designers.

Importante

Para que uma página JSTL (com código JSTL) funcione, isto é, seja aberta no browser e opere a partir de um servidor de aplicação, é necessário que seja inserida uma série de bibliotecas no diretório de bibliotecas de arquivos .jar do servidor (que para o Apache Tomcat é o diretório "lib").

Essa biblioteca é um conjunto de arquivos .jar (bibliotecas Java, relacionadas ao JSTL) que não são instalados normalmente quando da instalação do Java, mas que podem ser adquiridos na web.

Os seguintes grupos de arquivos permitem o funcionamento das páginas JSTL (o que significa que não é necessário ter (no diretório lib) todos os arquivos dos dois grupos de arquivos, mas aqueles de apenas **um** dos grupos a seguir):

- **1º grupo de arquivos:**

- jstl.jar;
- jstl-1.2.jar;
- jstl-standard.jar.

Os arquivos deste 1º grupo podem ser encontrados nos seguintes links (do site java2s – que é um site de bibliotecas Java).

O arquivo jstl.jar pode ser adquirido no link:

<http://www.java2s.com/Code/Jar/j/Downloadjstljar.htm>

O arquivo jstl-1.2.jar pode ser adquirido no link:

http://www.java2s.com/Code/Jar/j/Downloadjstl12jar.htm#google_vignette

O arquivo `jstl-standard.jar` pode ser adquirido no link:

<http://www.java2s.com/Code/Jar/j/Downloadjstlstandardjar.htm>

- **2º grupo de arquivos:**

- `taglibs-standard-compat-1.2.5.jar`;
- `taglibs-standard-impl-1.2.5.jar`;
- `taglibs-standard-jstlel-1.2.5.jar`;
- `taglibs-standard-spec-1.2.5.jar`.

Os arquivos deste 2º grupo podem ser encontrados no seguinte link (do site `archive.apache` – que é um site de bibliotecas do Apache):

<https://archive.apache.org/dist/tomcat/taglibs/taglibs-standard-1.2.5/>

Eles também podem ser encontrados no seguinte link:

<https://tomcat.apache.org/download-taglibs.cgi>

Com os arquivos `.jar` de um desses grupos de arquivos inseridos no diretório `lib` do servidor de aplicação (no `ApacheTomcat`, por exemplo), é possível fazer funcionar os exemplos sobre JSTL descritos neste livro-texto, de modo que, sem que estejam inseridos no diretório correto, eles não funcionarão.

Uma página JSTL (página com código JSTL) é uma página JSP que contém um conjunto de tags JSTLs. Cada tag realiza um determinado tipo de processamento (equivalente a um código Java quando descrito em scriptlets dentro da página JSP).



Saiba mais

Para aprender mais sobre os recursos de JSTL, acesse o link a seguir:

THE JAVA EE 5 tutorials. *Oracle*, [s.d.]. Disponível em: <https://bit.ly/4fuLrBG>. Acesso em: 28 jul. 2026.

Consta a seguir o exemplo de uma página JSTL:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!-- A Próxima diretiva, prepara a página para utilizar JSTL: -->
```

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>

<html>
<head>
<meta charset="ISO-8859-1">
<title>JSTL Novo</title>
</head>
<body>
<p>Imprimindo os números de 1 a 10, pulando (passo) de 2 em 2</p>
<!-- Próximas linhas com código JSTL: -->
<c:forEach var="i" begin="1" end="10" step="2">
    <c:out value="${i}" />
    <br />
</c:forEach>
</body>
</html>
```

O exemplo anterior apresenta na página (tela do browser) os números ímpares de 1 a 9. Observe que nela não existe nenhum código Java (nem JSP – scriptlet), de modo que está sendo utilizada uma forma de escrita específica do JSTL.

Na quarta linha, temos o seguinte conteúdo:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
```

Essa diretiva informa ao compilador que ele utilizará uma biblioteca de tags (a biblioteca core), cujas tags serão reconhecidas a partir do prefixo "c", evitando conflitos com outras tags de mesmo nome que possam pertencer a eventuais outras bibliotecas. A biblioteca utilizada neste caso está identificada pelo atributo uri.

No exemplo anterior, a tag `<c:forEach...>` é utilizada para criar um laço de repetição do tipo for, que é controlado por uma variável contadora (no caso, a variável "i").

Expression Language (EL) – Linguagem de Expressões

Em uma página com código JSTL, o valor de qualquer variável, ou expressão matemática, pode ser acessado pelo termo: `${expressão}`.

Por exemplo:

```
<p>Olá ${nomeAluno}</p>
```

Onde, na página, o resultado poderá ser, por exemplo:

Olá Fulano de Tal

Isso se o valor da variável nomeAluno for Fulano de Tal.

Exemplos de expressões com operadores

Segue uma lista de operadores de comparação e uma breve descrição deles:

- Operador + (soma)
- Operador == Igualdade (igual a)
- Operador != Desigualdade (diferente de)
- Operador < Menor que
- Operador > Maior que
- Operador <= Menor ou igual a
- Operador >= Maior ou igual a
- Operador and Operador lógico (E)
- Operador or Operador lógico (OU)
- Operador not Operador lógico de (Negação)
- Exemplo Resultado
- `{5 == 5}` true
- `{5 != 5}` false
- `{5 < 7}` true
- `{5 > 7}` false
- `{4 + 5}` 9
- `{not true}` false

Bibliotecas padrão do JSTL

A biblioteca core é chamada/acionada normalmente utilizando-se o prefixo "c". Ela é utilizada, por exemplo, para acessar e modificar dados em memória, assim como para acessar os comandos condicionais do JSTL, como o Loop `<c:forEach>`.

A biblioteca de processamento de XML é chamada/acionada normalmente utilizando-se o prefixo "x". Ela é utilizada, por exemplo, para realização de parsing (ou leitura) de documentos e arquivos, ou para a impressão de partes de documentos XML.

Um exemplo é a tomada de decisão baseada no conteúdo de um documento XML onde se utiliza a tag `<x:forEach>`.

A biblioteca de internacionalização e formatação é chamada/acionada normalmente utilizando-se o prefixo `fmt`. Ela é utilizada, por exemplo, para realizar a leitura e impressão de números e de datas, o que ajuda a aplicação a funcionar em mais de uma língua (por exemplo: a tag `<fmt:formatDate>`).

A biblioteca de acesso a BD via SQL é chamada/acionada normalmente utilizando-se o prefixo `sql`. Ela é utilizada, por exemplo, para realizar a leitura e escrita em BD `<sql:query>`.

Algumas tags básicas da biblioteca core

Tags de iteração

A biblioteca core do JSTL fornece tags de iteração para executar trechos repetidamente, de maneira similar aos comandos `for` e `while` da linguagem Java.

Tag `<c:forEach>`

Permite realizar um loop (ou uma iteração).

O exemplo a seguir imprime os valores entre dois e cinco utilizando "foreach".

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body bgcolor="#FFFFFF">
<c:forEach var="i" begin="2" end="5">
  <c:out value="${i}"/>;
</c:forEach>
</body>
</html>
```

A saída da execução desse código imprime ao longo da página:

2;3;4;5;

Tag `<c:forTokens>`

Quebra uma string em substrings, de acordo com o delimitador indicado em seus atributos. O exemplo a seguir é uma página JSTL que imprime em partes o conteúdo de um texto no qual se definiu a vírgula como separador de palavras ou trechos.

Arquivo "teste.jsp"

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body bgcolor="#FFFFFF">
<c:forTokens var="i" delims="," items="2,3,4,5">
  <c:out value="\${i}"/>;
</c:forTokens >
</body>
</html>
```

A saída para a página anterior (ao abri-la no browser com o servidor):

2;3;4;5;

Tags condicionais

Tag <c:if>

Esta tag equivale ao comando if da lógica de programação. O atributo test realiza o teste condicional (define a condição).

Por exemplo:

Arquivo "teste.jsp"

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
<body bgcolor="#FFFFFF">
Elementos pares:
<c:forTokens var="i" delims="," items="2,3,4,5">
  <c:if test="\${i % 2 == 0}">
    <c:out value="\${i}"/>;
  </c:if>
</c:forTokens >
</body>
</html>
```

A saída para o conteúdo JSTL da página anterior será:

Elementos pares: 2; 4;

Na biblioteca JSTL percebe-se a inexistência do complemento do comando if, ou seja, do comando else (como estamos acostumados em linguagem de programação). Neste caso, se o programador desejar criar fluxos alternativos, deve utilizar a tag <c:choose> com as tags <c: when> e <c: otherwise>.

Tag <c:choose>

Esta tag equivale ao comando switch das linguagens de script.

No exemplo a seguir, as tags utilizadas foram:

- **<c:when>**: realiza o teste condicional.
- **<c:otherwise>**: se todos os testes condicionais falharem, ele será utilizado.

Assim, o comando definido pela tag choose escolherá entre os comandos when e otherwise, de acordo com a condição definida na tag <c:when>.

Por exemplo:

Arquivo "teste.jsp"

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
<body bgcolor="#FFFFFF">

<c:forTokens var="i" delims="," items="2,3,4,5">
  <c:choose>
    <c:When> test="${i % 2 == 0}">${i} (par)</c:when>
    <c:otherwise>${i} (impar)</c:otherwise>
  </c:choose>
  ;
</c:forTokens >

</body>
</html>
```

A saída para a página anterior será:

2 (par) ; 3 (impar) ; 4 (par) ; 5 (impar) ;

Tags de atribuição e importação

Tag <c:import>

Esta tag permite importar páginas web do mesmo contexto web, também de contextos diferentes e até mesmo de máquinas variadas. Para as suas diversas formas, utilizamos os seus atributos, tais como:

- **url**: deve receber a URL a ser importada.
- **contexto**: "/" seguido do nome da aplicação web local.

- **var**: é o nome do atributo onde será armazenado o conteúdo da página importada.
- **scope**: indica o escopo do atributo onde será armazenado o conteúdo da página importada.

Tag <c:set>

Esta tag permite atribuir valores a variáveis em um determinado escopo.

- **value**: indica a expressão a ser processada.
- **var**: informa o nome do atributo onde será armazenado o resultado do processamento (em seu atributo value).
- **scope**: é o escopo do atributo.

No exemplo a seguir, a variável title é criada com o valor Welcome to Page 1 com escopo request. Na sequência, a página header.jsp é carregada, e seu conteúdo é armazenado na variável headerText.

Finalmente, imprimimos o conteúdo da variável title e da variável headerText.

Arquivo "header.jsp"

```
<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core" %>
<html>
<body>
<c:set scope="request" var="title" value="Welcome to Page 1"/>
<c:import var="headerText" url="header.jsp"/>
<br>
Minha página:${title}
<br>
Texto importado:${headerText}
</body>
</html>
```

A saída para a página anterior será:

Minha página:Welcome to Page 1

Texto importado: #Welcome to Page 1#

Os exemplos anteriores se detiveram em apresentar apenas as tags JSTL básicas.

7.4.1 JSTL – core “c:catch”: tratando erros

Tag JSTL <c:catch>

Esta é uma tag JSTL de tratamento de exceção. Ela é utilizada para o tratamento de erros e captura qualquer erro ou exceção derivada da classe Throwable, do Java, que venha a ocorrer no interior da tag, seja em um programa JSP, seja em um código JSTL. Vejamos na prática com a simulação de um erro.

No primeiro exemplo, criaremos uma variável que receberá a execução de uma conta de uma divisão por zero. No segundo, tentaremos realizar a importação de uma página web não existente.

Muitas vezes é preciso exibir para o usuário qual o erro ocorrido para que se possa saber o que está acontecendo, de forma que para isso utilizaremos a tag <c: if>, que implementa um laço de decisão, de modo que, se ocorrer algum erro (ou alguma exceção), é possível exibi-lo, ou até mesmo mostrar qualquer outra mensagem.

Para as capturas de exceções com este recurso (o termo catch da tag “c” – biblioteca core – do JSTL), pode-se utilizar quaisquer exceções da biblioteca Java criadas como filha da classe Throwable, que podem ocorrer no corpo da página, ao longo (e depois) do processo de sua abertura.

Em geral, esse recurso é utilizado para tratamento de erros em páginas JSP e JSTL para lidar de forma mais fácil com os possíveis problemas que ocorrem no programa (assim como foi visto no item que fala sobre tratamento de exceções de programas Java, do livro-texto da disciplina [Linguagem de Programação Orientada a Objetos](#)).

A tag <c:catch> capturará qualquer exceção que ocorrer na execução dos códigos que estiverem entre as tags <c:catch> e </c:catch> descritas no programa (lembrando que este será um programa web).

Vamos criar um exemplo de uma página JSTL com uma captura simples de uma exceção com a tag <catch>:

Exemplo de aplicação

Na página do exemplo a seguir o código está tentando realizar um cálculo matemático impossível, que é a divisão de um número inteiro por 0. No caso, está tentando dividir o valor inteiro 5 por 0.

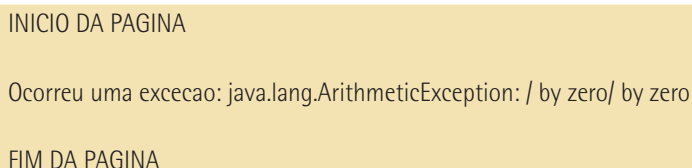
```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
<head>
<title>Exemplo da Tag “c” (Core)</title>
</head>
<body>
<p>INICIO DA PAGINA</p>

<c:catch var =“excecaoCapturada”>
```

```
<% int x = 5/0;%>
</c:catch>

<c:if test = "${excecaoCapturada != null}">
<p> Ocorreu uma excecao: ${excecaoCapturada} <br />
${excecaoCapturada.message}</p>
</c:if>
<p>FIM DA PAGINA</p>
</body>
</html>
```

Assim, o resultado da abertura (ou execução) da página do exemplo anterior, no browser, será o seguinte texto:



```
INICIO DA PAGINA

Ocorreu uma excecao: java.lang.ArithmeticException: / by zero / by zero

FIM DA PAGINA
```

Figura 18



Lembrete

Na matemática, realizar uma divisão de um número inteiro por zero resulta em erro de operação (é considerada uma operação não aceita, ou operação impossível).

No exemplo anterior, estamos lançando de propósito uma exceção matemática (aritmética) de forma que a mensagem de erro ficará guardada na variável `excecaoCapturada`, utilizando-se do padrão de mensagens do Java, o padrão Expression Language (EL), resultando na mensagem do quadro prévio.

Percebe-se que o valor completo da variável `excecaoCapturada` indica qual a biblioteca da exceção ocorrida, com a mensagem de texto padrão do erro ("java.lang.ArithmeticException: / by zero"). Já o valor da mensagem da variável ("`excecaoCapturada.message`") contém apenas a indicação do problema ocorrido ("/ by zero"), que significa problema de divisão por zero.

Sintaxe de utilização da tag (`<c:catch>`)

```
<c:catch var ="nome_da_variavel">
```

//Conjunto de Códigos que podem ocasionar em exceção

```
</c:catch>
```

O nome da variável pode ser qualquer um (seguindo as regras de nomes de variáveis de programas), de forma que será nessa variável que a mensagem de exceção ficará guardada. Já a parte do "Conjunto de Códigos..." poderá ser tanto um código JSTL (como no exemplo a seguir em que a core "c:import" tenta importar uma página não existente) como um código JSP (como no exemplo anterior em que um scriptlet JSP tentou fazer uma conta matemática impossível).

Desta forma, ocorrendo efetivamente a exceção ao longo da execução do código que estiver dentro da core catch (entre o "c:catch" e o "/c:catch"), a variável denominada no atributo var da core guardará a mensagem da exceção ocorrida.

Exemplo de aplicação

A seguir, utilizaremos um exemplo de tentativa de importação a partir do comando JSTL "c:import", uma (outra) página, sendo que ela não existe.

Para poder testar esse exemplo, copie o texto a seguir (desde o termo "<%@ taglib ..." até o termo "</html>") criando um arquivo texto com extensão .jsp, que poderá ser acionado a partir do servidor de aplicação.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
<head>
<title>Exemplo da Tag "c" (Core)</title>
</head>
<body>
<p>INICIO DA PAGINA</p>
<c:catch var="excecaoAcessoAoSite">
  <c:import url="http://www.site-nao-existente.com.br"/>
</c:catch>
<c:if test="${not empty excecaoAcessoAoSite}">
  Site INACESSIVEL!!!
  <c:out value="${excecaoAcessoAoSite.message}"/>
</c:if>
<p>FIM DA PAGINA</p>
</body>
</html>
```

Para o exemplo anterior, o resultado da abertura (ou execução) da página, no browser, deverá ser o seguinte texto:

INICIO DA PAGINA

Site INACESSIVEL!!! Problem accessing the absolute URL "http://www.site-nao-existente.com.br". java.
net.UnknownHostException: www.site-nao-existente.com.br

FIM DA PAGINA

Figura 19

No exemplo prévio, assim como no anterior a ele, estamos lançando uma exceção de propósito, porém desta vez tentando abrir (importar) uma página inexistente da web, de forma que a mensagem de erro fique guardada na variável `excecaoAcessoAoSite`, também utilizando-se do padrão de mensagens do Java, o padrão de Expression Language (EL), e resultando na mensagem do quadro anterior.

Importante: se nenhuma exceção ocorrer, a variável que controla a mensagem da exceção terá valor nulo (null), de forma que é por esta razão que existe o bloco de condição "if" onde é testado se tal variável possui valor "diferente de nulo".



Lembrete

Para que o servidor (Apache Tomcat) possa abrir uma página JSTL, o conjunto de bibliotecas JSTL deve existir no diretório "lib" do Apache Tomcat.

8 RELATÓRIOS: GERANDO RELATÓRIOS COM O MS EXCEL

Através da linguagem Java, pode-se criar relatórios de diversas formas. Uma delas é gerando-se e preenchendo-se uma planilha em Excel, da qual é possível posteriormente realizar a formatação de modo adequado. Neste capítulo aprenderemos alguns procedimentos que possibilitam gerar com um programa em Java um arquivo Excel a partir de comandos (códigos). Portanto, aprenderemos como elaborar uma planilha e preencher cada uma de suas células, bem como ler os dados de uma planilha do Excel.

Na sequência veremos alguns exemplos de métodos úteis na geração de relatórios. Para que funcionem em conjunto, todos devem pertencer a uma única classe (verifique a utilidade de cada método, descrita nos comentários após sua declaração).

Como sugestão, os métodos da classe a seguir podem ser utilizados na criação de relatórios em Excel, a partir de dados do banco de dados. Elas utilizam a classe exemplo `AlunosBD` do item 4.3 deste livro-texto e, portanto, devem ser colocadas no mesmo projeto.



Observação

Para que o exemplo da classe de relatório funcione, e possa ser compilada corretamente, deve-se importar uma biblioteca própria (um arquivo jar externo – o arquivo `jxl.jar`), assim como fizemos no item 4 deste livro-texto com a biblioteca que continha o driver do banco de dados (pelas Propriedades do Projeto, em "Java Build Path", na aba "Libraries", clicando em "Add External JARs", e selecionando o arquivo "jar" que continha o driver do banco de dados).

Além disso, deve-se adicionar o jar que permite a manipulação de dados em arquivos do M.S. Excel, que é o arquivo "jxl.jar", que possui uma biblioteca de classes que permite a manipulação de dados em arquivos do Excel.



Saiba mais

Na página a seguir encontra-se um link para baixar um arquivo compactado (com o nome de "jexcelapi_2_6_12.zip"). Ele (que possui o nome do arquivo compactado) iniciará o download do arquivo "zip".

A JAVA library for reading/writing Excel Files. *Sourceforge*, [s.d.]. Disponível em: <https://tinyurl.com/2he5uzcd>. Acesso em: 28 jul. 2026.

Após completado o download deve-se abrir o arquivo compactado (a partir de qualquer programa que abra arquivos compactados), ou simplesmente verificar seus arquivos internos, procurando (provavelmente no primeiro diretório interno do arquivo) um arquivo com o nome "jxl.jar". Será ele que utilizaremos como biblioteca de classes para fazermos funcionar nosso programa que gerará o arquivo do Excel.

É importante que compreendamos que sem este arquivo (jxl.jar), nossa classe do exemplo a seguir não funcionará (e nem poderá ser compilada), gerando erros de compilação enquanto estivermos criando a classe no Eclipse.

Exemplo de aplicação

Vejamos a classe RelatorioClass:

```
import java.io.File;
import java.util.ArrayList;

import jxl.Workbook;
import jxl.WorkbookSettings;
import jxl.write.Label;
import jxl.write.WritableSheet;
import jxl.write.WritableWorkbook;
import model.Aluno;

public class RelatorioClass {
    //Esta classe contém métodos que permitirão
    //... gerar e preencher dados em um arquivo Excel.

    public WritableWorkbook wWork = null;
    public WritableSheet sheet = null;

    //Método que permite criar, em algum diretório, um arquivo do Excel:
    public void gerarArquivoExcel(String pathArq) {
        try {
            File file = new File(pathArq);
            //A variável (ou o parâmetro) "pathArq" deve receber o texto...
            //... com o caminho completo da localização do arquivo do Excel.
```

```

        WorkbookSettings wSet = new WorkbookSettings();
        wWork = Workbook.createWorkbook(file, wSet);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

public void pegarPrimeiraPlanilha() {
    //Este método define o atributo "sheet" como sendo a
    //... primeira (e única) planilha criada no arquivo do Excel.
    sheet = wWork.createSheet("RelatórioX", 4);
}

```

```

public void inserirDadosPlanilha(String strTit, ArrayList<Aluno> arrListAl)
{
    //Este método recebe (como parâmetro) o título da planilha...
    //...que será inserido logo na primeira linha da planilha, ...
    //...além de um ArrayList do tipo "Aluno", com os dados de cada um dos
    //...Alunos
    int linha = 0; // primeira linha
    int coluna = 0; // primeira coluna
    Label lab;
    try {
        // Escrevendo o Título na 1a linha:
        // ..cria-se o Label, apontando-o para uma célula,
        lab = new Label(coluna, linha, strTit);
        // ..e adiciona-o à planilha
        sheet.addCell(lab);
        // Capturando cada um dos Alunos inseridos no ArrayList
        for (int ii = 0; ii < arrListAl.size(); ii++) {
            //obs.: cada linha da planilha representará um aluno
            //...cada iteração, incrementa-se 1 à variável "linha"
            linha++; // ...para controle da linha da Planilha
            Aluno al = arrListAl.get(ii); //pega-se o Aluno do Array
            // Inserindo os dados do Aluno numa coluna daquela linha:
            //...(o id)
            String txt = "\"" + al.getId();
            coluna = 0; // esta é a primeira coluna
            lab = new Label(coluna, linha, txt);
            sheet.addCell(lab); //insere na planilha
            //...(o nome)
            txt = al.getNome();
            coluna = 1; // esta é a segunda coluna
            lab = new Label(coluna, linha, txt);
            sheet.addCell(lab);
            //...(o ra)
            txt = "\"" + al.getRa();
            coluna = 2; // esta é a terceira coluna
            lab = new Label(coluna, linha, txt);
            sheet.addCell(lab);
            //...(a idade)
            txt = "\"" + al.getIdade();
            coluna = 3; // esta é a quarta coluna
            lab = new Label(coluna, linha, txt);
            sheet.addCell(lab); //insere na planilha
        }
    }
}

```

```
    }  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}  
  
public void salvarArquivo () {  
    //Método que salva o arquivo (Excel) com os dados nele inseridos  
    try {  
        wWork.write(); // gravou/salvou o arquivo Excel (.xls)  
        wWork.close();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}  
}
```

Vejamos a classe TestaRelatorio:

Esta classe utilizará as classes criadas no tópico 4.3 deste livro-texto (classes Aluno, AlunosBD), e portanto, deverá ser gerada no mesmo projeto (ou em um novo projeto que contenha aquelas duas classes, e todas as outras as quais aquelas utilizam (como a classe GeraConexao – do tópico 4.2 – por exemplo).

```
import java.util.ArrayList;  
import model.Aluno; //verificar localização (pacote) da classe  
import model.AlunosBD; //verificar localização (pacote) da classe  
  
public class TestaRelatorio {  
    //Esta classe permite testar a classe que preenche dados no Excel  
    //..aqui primeiro se lerá os dados do BD...  
    //..e depois se criará o Excel  
  
    public static void main(String[] args) {  
        TestaRelatorio te = new TestaRelatorio();  
        te.geraExcel();  
    }  
    public void geraExcel() {  
        // Capturando os dados do BD (incluindo-os num ArrayList:  
        System.out.println("LENDO O BANCO DE DADOS...");  
        AlunosBD aBD = new AlunosBD();  
        ArrayList<Aluno> listaAlunos = aBD.getAllAlunos();  
  
        // Os dados do ArrayList (que vieram do BD) serão copiados para o Excel  
        System.out.println("MONTANDO O RELATÓRIO NO EXCEL COM OS DADOS...");  
        // Instanciando a Classe que permite a geração do relatório  
        RelatorioClass ec = new RelatorioClass();  
        // Indicando onde o arquivo Excel será salvo (e o nome do arquivo).  
        String caminhoArquivo = "c:/teste/relatorioAlunos01.xls";  
        // Gerando o arquivo.  
        ec.gerarArquivoExcel(caminhoArquivo);  
        // Pegando a primeira planilha  
        ec.pegarPrimeiraPlanilha();  
        // Inserindo um título na 1ª linha, e os dados dos Alunos nas demais
```

```
ec.inserirDadosPlanilha("Dados dos Alunos:", listaAlunos);  
// Salvando o arquivo já preenchido  
ec.salvarArquivo();  
System.out.println("RELATÓRIO FINALIZADO...");  
}  
}
```

No exemplo anterior, a ideia foi fazer com que fosse gerado um arquivo Excel e nele estivessem gravadas todas as informações sobre cada aluno da tabela de alunos do banco de dados (tabela essa que foi utilizada ao longo do tópico 4, que descreve toda a parte de JDBC neste livro-texto).



Lembrete

Sempre que vamos acessar um banco de dados, o driver deste BD, para o Java, deve fazer parte das bibliotecas do projeto (adicionando um "jar externo", que no caso é o arquivo jar fornecido pela empresa do BD, sendo o exemplo deste livro-texto o driver do MySQL).



Resumo

Nesta unidade, vimos como criar sistemas que funcionam e ficam instalados localmente no micro do usuário, o que chamamos de sistemas desktop. Podemos, contudo, construir sistema que permitem a interação com o usuário a partir de páginas web, ou seja, para serem utilizados através de qualquer browser instalado no micro.

Esses sistemas são construídos para funcionar em duas frentes, o front-end e o back-end. A parte do front-end nesses casos é geralmente construída utilizando-se o HTML, o CSS e o JavaScript. A parte do back-end, já que neste curso aprendemos a linguagem Java (relacionada a OO), pode ser codificada, utilizando-se o JSP, que permite misturarmos a parte do front-end com códigos em Java que vão ser rodados no servidor de aplicações (no back-end), o qual aqui utilizamos o Apache Tomcat, por ser de fácil instalação e utilização, e por ter distribuição gratuita.

Demonstramos que existem, para o JSP, algumas tags padrão, como os scriptlets (onde colocamos o código Java), expressões (onde disponibilizamos na tela o valor de uma variável Java), declarações (onde definimos atributos e métodos) e diretivas (com ações e configurações específicas da página) e que são disponibilizadas ao longo da página web.

Para facilitar e agilizar ainda mais o trabalho dos desenvolvedores na construção de páginas web com o JSP, entendemos que é possível utilizar uma biblioteca de tags, conhecida como JSTL que permite, em alguns casos, a substituição de códigos Java (em JSP), para códigos gerados com tags (baseados em xml). A fim de que possamos utilizar tal recurso, as bibliotecas JSTL (os arquivos .jar) devem ser inseridas na área de bibliotecas do servidor de aplicações. Em alguns dos exemplos exibidos, utilizamos o JSTL com a "core" "catch" (ou seja, com a tag "catch" do elemento "core") para tratamento de erros e exceções nas páginas web.

Por fim, trabalhamos com a interação indireta dos usuários e os bancos de dados, construindo sistemas que permitem essa interação. Desta forma, é muito comum a necessidade de extração dos relatórios das informações do banco de dados. Logo, mostramos também como gerar arquivos em Excel, de modo a permitir que, após essa extração, o usuário configure a planilha gerada no formato que ele quiser.



Exercícios

Questão 1. (IFB/2017, adaptada) No desenvolvimento de uma aplicação web, o conteúdo dos arquivos em HTML pode ser gerado dinamicamente de diversas maneiras, e várias linguagens de programação que executam em um servidor de aplicação podem ser utilizadas para isso. Sob a ótica da tecnologia de servlets e JSP, analise as afirmativas a seguir.

I – Uma página escrita em JSP é bem parecida com uma página escrita em HTML: a diferença é que podemos inserir um código Java na página JSP, e ele é executado pelo interpretador do navegador web.

II – A tecnologia JSP é incompatível com a linguagem XML.

III – Uma página HTML pode ser escrita por comandos codificados em Java, por meio da classe `HttpServlet`.

É correto o que se afirma em:

A) I, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa B.

Análise das afirmativas

I – Afirmativa incorreta.

Justificativa: de fato, uma página escrita em JSP é bem parecida com outra escrita em HTML: a diferença é que podemos inserir um código Java na página JSP. No entanto, ele será executado pelo servidor de aplicação, não pelo interpretador do navegador web.

II – Afirmativa incorreta.

Justificativa: a tecnologia JSP permite que desenvolvedores criem páginas web geradas dinamicamente, baseadas em HTML, XML ou outros tipos de documentos.

III – Afirmativa correta.

Justificativa: a plataforma Java utiliza servlets para atrelar conteúdo dinâmico ao HTML. Pode-se usar a classe abstrata `HttpServlet` para juntar HTTP, servlets, conteúdo estático e conteúdo dinâmico. Essa classe, por ser abstrata, deve ter seus métodos, como `doGet` e `doPost`, implementados. Eles representam, respectivamente, os métodos HTTP GET e POST. Portanto, uma página HTML pode ser escrita por comandos codificados em Java por meio da classe `HttpServlet`. Esses comandos, codificados em Java, são executados no servidor de aplicação.

Questão 2. (FCC/2017, adaptada) Em uma aplicação Java para web, uma página `exibir.jsp` recebeu de uma servlet, a partir de uma operação de consulta em um banco de dados, uma lista contendo um conjunto de objetos de uma classe `Cliente.java`, que possui os atributos `id`, `nome`, `telefone` e `renda`. O trecho presente na servlet é o seguinte:

```
ArrayList lista_clientes = dao.consultar();
request.setAttribute("listaclientes", lista_clientes);
RequestDispatcher d = request.getRequestDispatcher("exibir.jsp");
d.forward(request, response);
```

Figura 20

Na página `exibir.jsp`, para receber a lista e exibir os dados utilizando a biblioteca core do JSTL, foi utilizado o trecho a seguir:

```
.....  
I  
.....  
    <tr>  
        <td>${cliente.id}</td>  
        <td>${cliente.nome}</td>  
        <td>${cliente.telefone}</td>  
        <td>${cliente.renda}</td>  
    </tr>  
II  
.....
```

Figura 21

As lacunas I e II são preenchidas, correta e respectivamente, por:

- A) `<c:forEach var="cliente" items="${listaclientes}">` e `</c:forEach>`.
- B) `<c:when var="cliente" list="${listaclientes}">` e `</c:when>`.
- C) `<c:while var="cliente" items="${listaclientes}" test="${cliente!=null}">` e `</c:while>`.
- D) `<c:while{cliente.next} items="${listaclientes}">` e `</c:while>`.
- E) `<c:doWhile var="cliente" list="${lista_clientes}">` e `</c:doWhile>`.

Resposta correta: alternativa A.

Análise da questão

A biblioteca core do JSTL é acionada, normalmente, utilizando-se o prefixo "c". Ela é usada, por exemplo, para acessar e modificar dados em memória, assim como para acessar os comandos condicionais do JSTL. Entre suas tags, a tag `<c:forEach>` permite realizar um loop, ou seja, um laço de iteração. Ela é similar ao comando `for` da linguagem Java.

A tag `<c:forEach>` tem os atributos listados a seguir. Vale notar que nenhum atributo listado é obrigatório. Vejamos o quadro a seguir.

Quadro 1

Atributo	Descrição	Valor padrão
var	Nome da variável capaz de expor o item corrente	Nenhum
items	Informação sobre a qual será executado o loop	Nenhum
begin	Elemento de início do loop	0
end	Elemento de final do loop	Último elemento
step	Indica o passo da iteração	1
varStatus	Nome da variável capaz de expor o status do loop	Nenhum

Na página `exibir.jsp`, cujo trecho é reproduzido no enunciado, vemos as tags `<tr>` e `<td>`, indicando que as informações oriundas do banco de dados serão exibidas em uma tabela.

Desse modo, o loop pode ser reproduzido por meio dos atributos `var` e `items` da tag `<c:forEach>`. Em `var`, atribuiremos o nome da variável para a qual indicaremos o atributo buscado, que é `cliente`. Em `items`, informaremos o nome do atributo sobre o qual o loop será realizado, que é `listaclientes`.

Após inserirmos o código da montagem da tabela que exibirá os dados dos clientes, devemos inserir uma tag de fechamento, `</c:forEach>`.

O código completo para receber a lista e exibir os dados, portanto, é dado de acordo com o mostrado a seguir.

```
<c:forEach var="cliente" items="${listaclientes}">
  <tr>
    <td>${cliente.id}</td>
    <td>${cliente.nome}</td>
    <td>${cliente.telefone}</td>
    <td>${cliente.renda}</td>
  </tr>
</c:forEach>
```

A respeito das outras tags presentes nas alternativas da questão, temos o que segue.

- A tag `<c:when>` faz parte da biblioteca core do JSTL, mas é utilizada com a tag `<c:choose>`.
- A tag `<c:choose>` é similar ao comando `switch` de Java, que permite escolher uma entre as alternativas presentes. Enquanto o comando `switch` trabalha com o comando `case`, a tag `<c:choose>` funciona com a tag `<c:when>`.
- As tags `<c:while>` e `<c:doWhile>` não são válidas na biblioteca core.

REFERÊNCIAS

Textuais

CSS tutorial. *W3Schools*, [s.d.]. Disponível em: <https://tinyurl.com/3juty8dm>. Acesso em: 28 jul. 2026.

DEITEL, P.; DEITEL, H. *Java: como programar*. São Paulo: Pearson Education do Brasil, 2017.

FURGERI, S. *Java 7: ensino didático*. 2. ed. São Paulo: Érica, 2012.

FURGERI, S. *Java 8: ensino didático – desenvolvimento e implementação de aplicações*. São Paulo: Érica, 2015.

GOMES, Y. M. *Java na Web com JSF, Spring, Hibernate e Netbeans 6*. Rio de Janeiro: Ciência Moderna, 2008.

GONÇALVES, E. *Dominando eclipse: tudo que o desenvolvedor Java precisa para criar aplicativos para Desktop*. Rio de Janeiro: Ciência Moderna, 2006.

HORSTMANN, C. S.; CORNELL, G. *Core Java*. Volume I: fundamentos. 8. ed. São Paulo: Pearson Prentice Hall, 2010.

HTML tutorial. *W3Schools*, [s.d.]. Disponível em: <https://tinyurl.com/ys89nytx>. Acesso em: 28 jul. 2026.

HTML unicode (UTF-8) reference. *W3Schools*, [s.d.]. Disponível em: <https://tinyurl.com/27552xn4>. Acesso em: 28 jul. 2026.

JANDL JUNIOR, P. *Java: guia do programador*. Atualizado para Java 6. São Paulo: Novatec, 2007.

A JAVA library for reading/writing Excel Files. *Sourceforge*, [s.d.]. Disponível em: <https://tinyurl.com/2he5uzcd>. Acesso em: 28 jul. 2026.

THE JAVA EE 5 tutorials. *Oracle*, [s.d.]. Disponível em: <https://tinyurl.com/ye25ad9r>. Acesso em: 28 jul. 2026.

THE JAVA™ Tutorials. *Oracle*, [s.d.]. Disponível em: <https://tinyurl.com/336f5bha>. Acesso em: 28 jul. 2026.

MECENAS, I. *Java 6: fundamentos, Swing, BlueJ e JDBC*. Rio de Janeiro: Alta Books, 2008.

PUGA, S.; RISSETTI, G. *Lógica de programação e estruturas de dados, com aplicações em Java*. 3. ed. São Paulo: Pearson Education do Brasil, 2016.

WINDER, R. *Desenvolvendo software em Java*. Rio de Janeiro: LTC, 2009.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

APÊNDICE A

1 INSTALANDO O MYSQL

Neste livro-texto utilizamos a linguagem Java acessando um Servidor de Gerenciamento de Banco de Dados (SGBD) – ou ainda, Sistema Gerenciador de Banco de Dados. O curso sugere a utilização do SGBD "MySQL", que é um SGBD de distribuição gratuita, e cujo arquivo de instalação pode ser encontrado no seguinte site:

<https://www.mysql.com/downloads/>

A utilização e consequente instalação desse SGBD já foi comentada em disciplinas anteriores deste curso (por exemplo, a disciplina de Introdução à Programação Estruturada, que tratou do acesso ao SGBD através de programas gerados na linguagem Python). No entanto, sua sequência de instalação pode ser encontrada no texto de um artigo publicado no seguinte site:

<https://www.alura.com.br/artigos/mysql-do-download-e-instalacao-ate-sua-primeira-tabela>

Assim como pode ser verificada no seguinte site:

<https://www.devmedia.com.br/instalando-e-configurando-a-nova-versao-do-mysql/25813>

APÊNDICE B

1 INSTALANDO O APACHE TOMCAT

O software Apache Tomcat é a junção de dois servidores:

- um servidor web, ou servidor HTTP (que é o Apache);
- um servidor de aplicação Java (que é o Tomcat).

Para fazer o download e instalar o Apache Tomcat, basta utilizar os seguintes passos:

1) Em qualquer browser, abra o site do Apache Tomcat: <https://tomcat.apache.org/> e selecione uma de suas versões (pode ser a última versão).

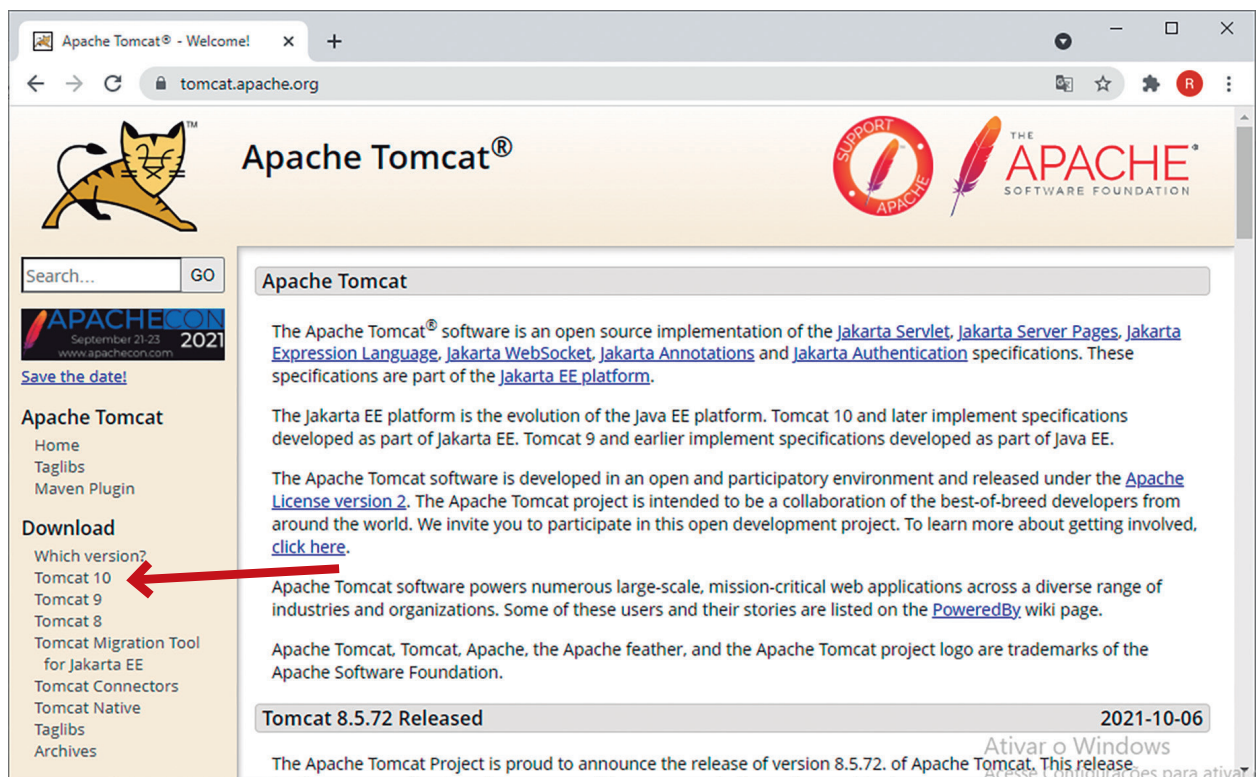


Figura B1 – Tela principal do Apache Tomcat para fazer o seu download

2) Seleccione:

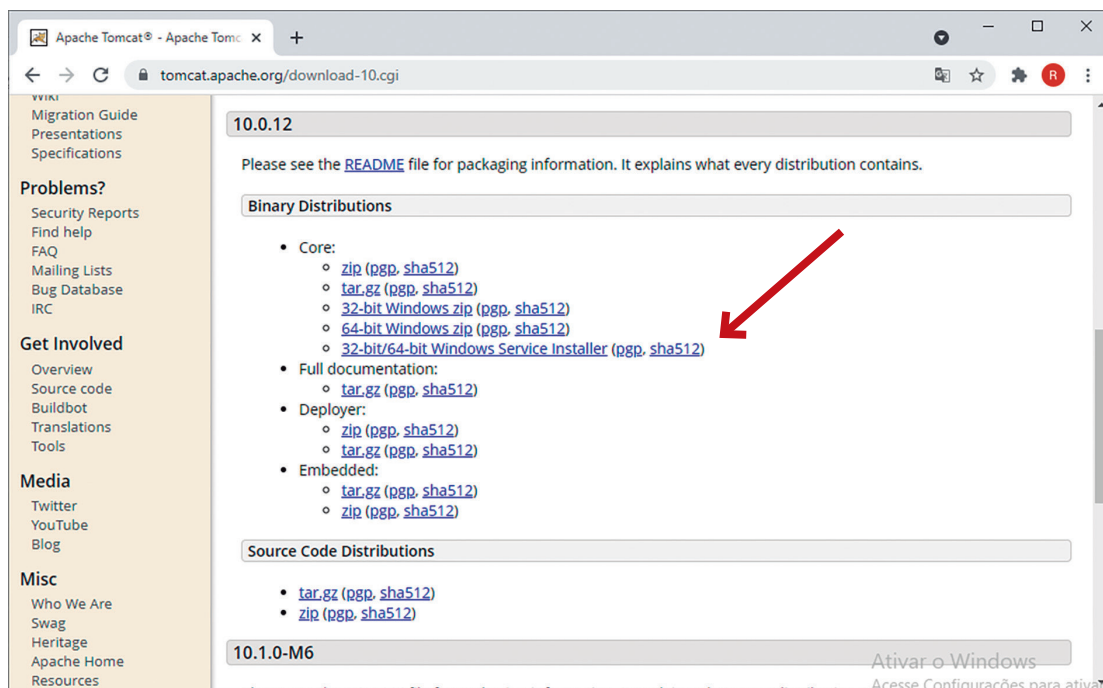


Figura B2 – Tela do Apache Tomcat com as opções de download

3) Ao acionar (com um duplo clique) o arquivo baixado, a tela seguinte será apresentada:

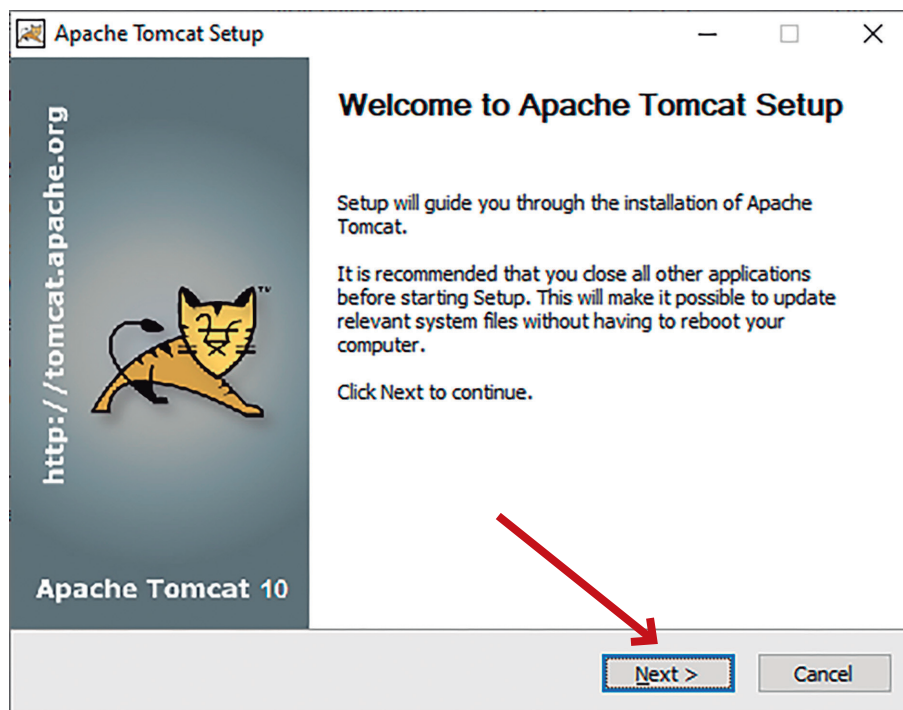


Figura B3 – Primeira tela do processo de instalação do Apache Tomcat

4) Seleccione:

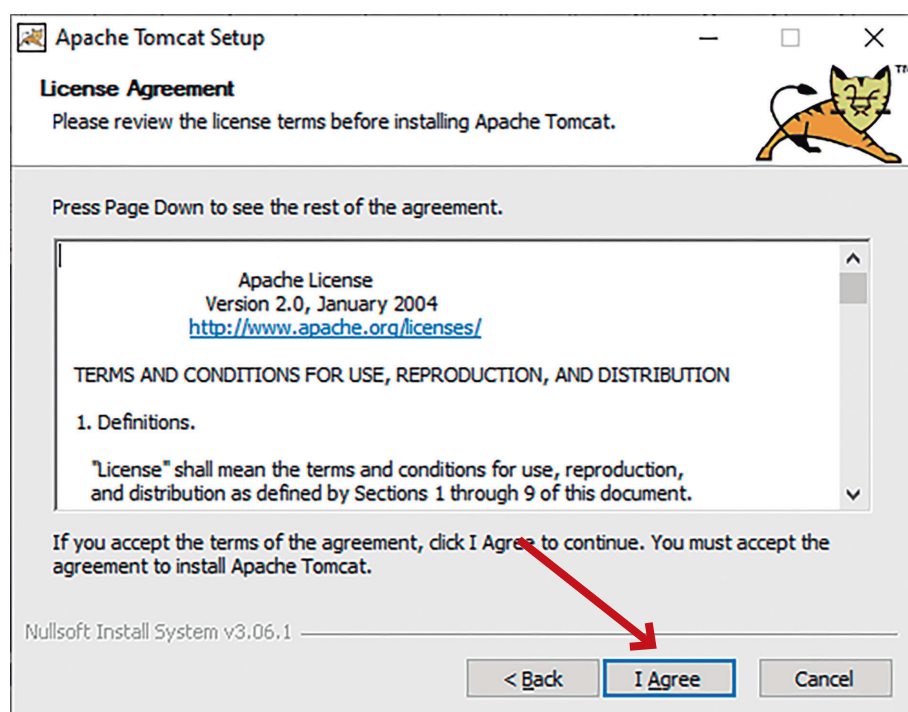


Figura B4 – Segunda tela do processo de instalação do Apache Tomcat

5) Seleccione:

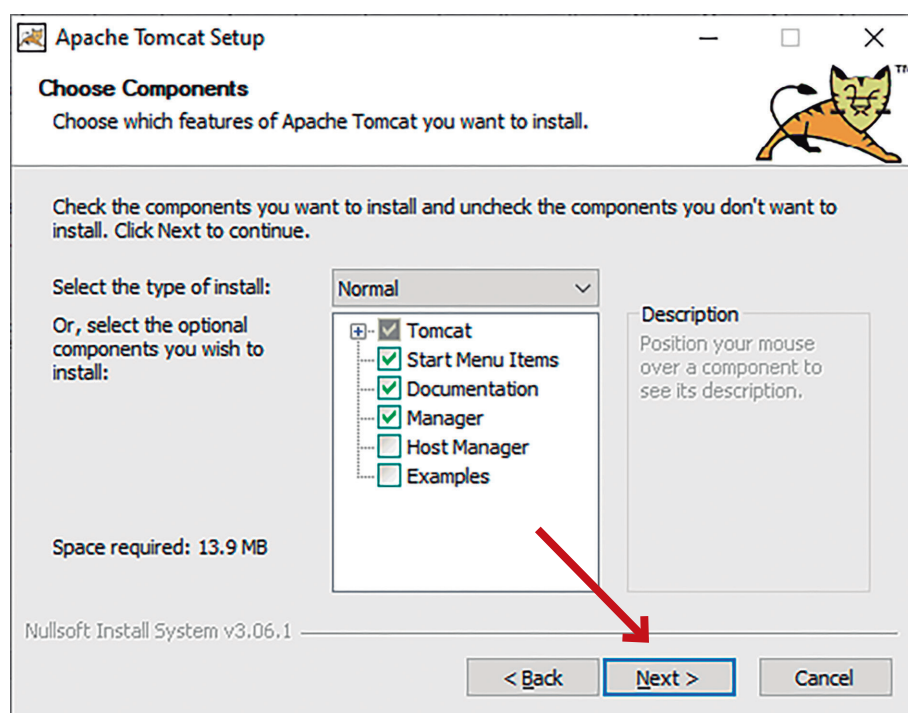


Figura B5 – Terceira tela do processo de instalação do Apache Tomcat

6) Após clicar em "Next >", na tela seguinte, insira um usuário e uma senha (Sugestão: usuário "admin" e senha "admin"):

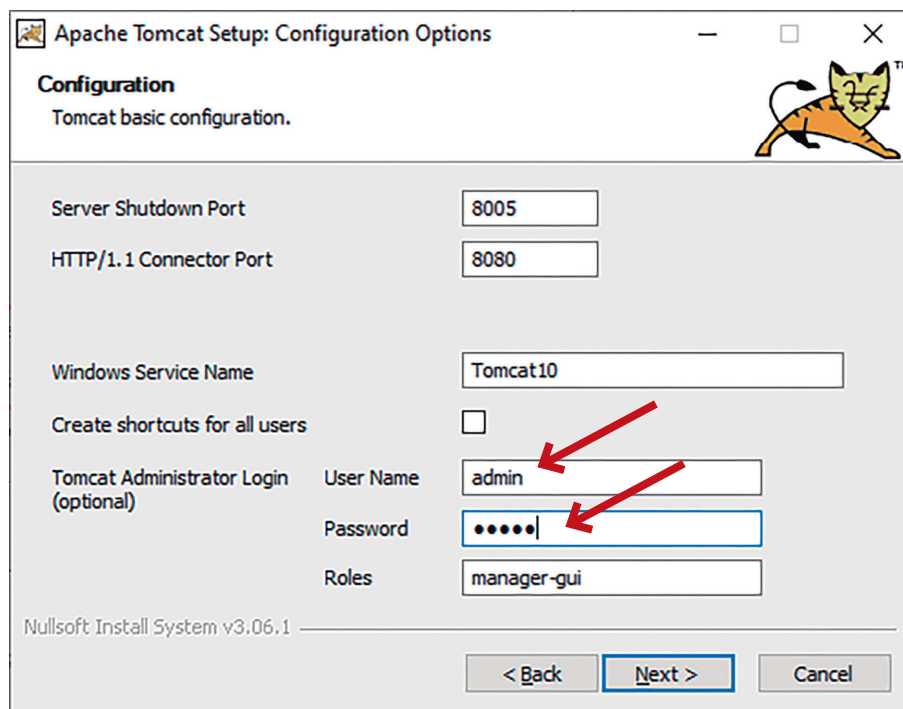


Figura B6 – Quarta tela do processo de instalação do Apache Tomcat

7) Após clicar em "Next >", na tela seguinte:

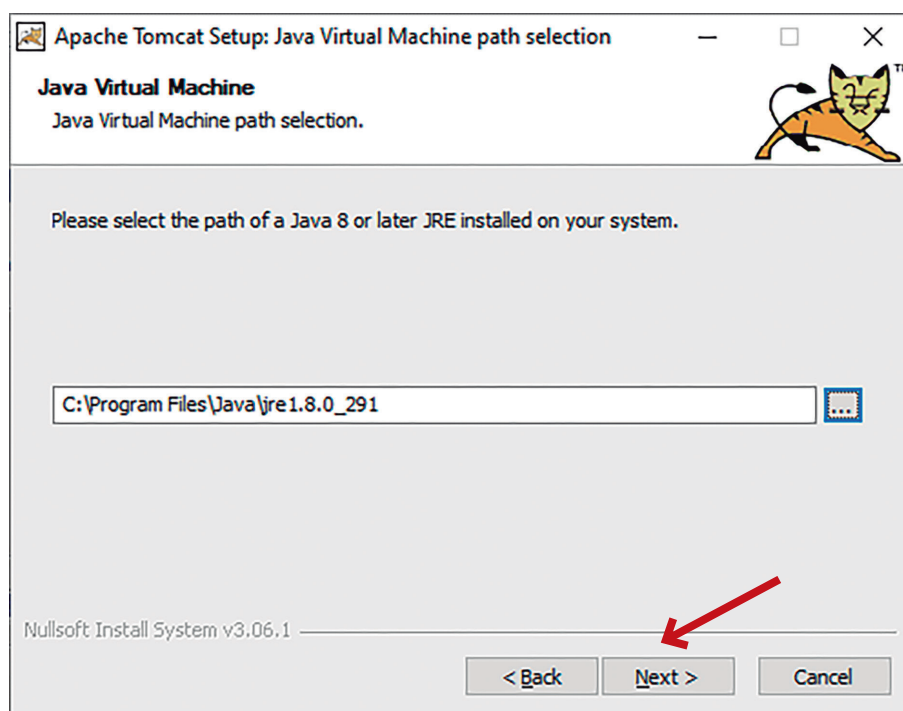


Figura B7 – Quinta tela do processo de instalação do Apache Tomcat

8) Selecione:

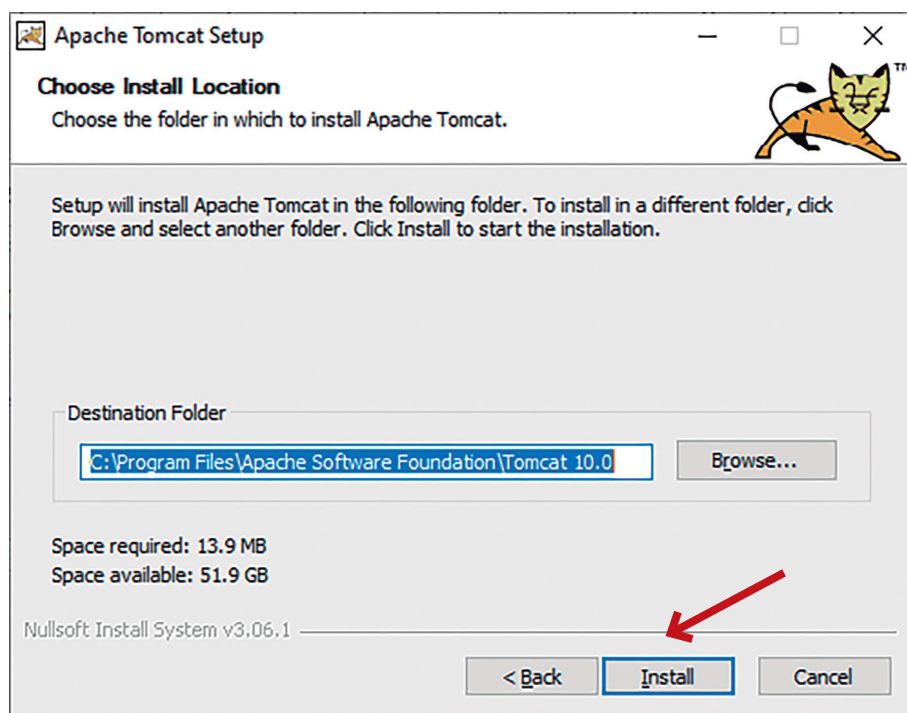


Figura B8 – Sexta tela do processo de instalação do Apache Tomcat

9) Selecione:

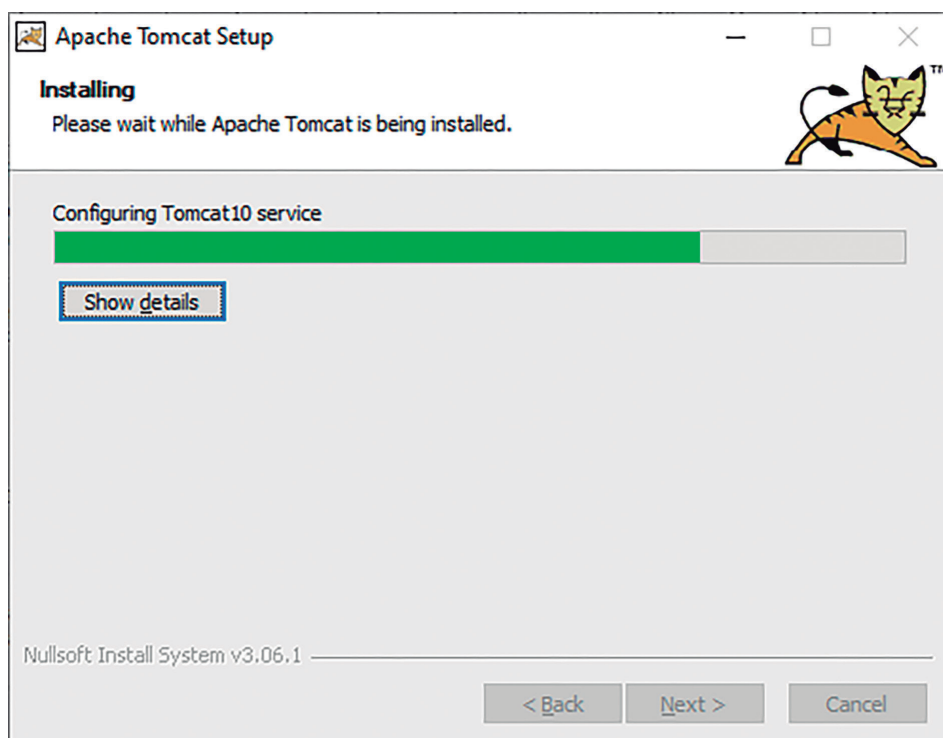


Figura B9 – Sétima tela do processo de instalação do Apache Tomcat

10) Seleccione:

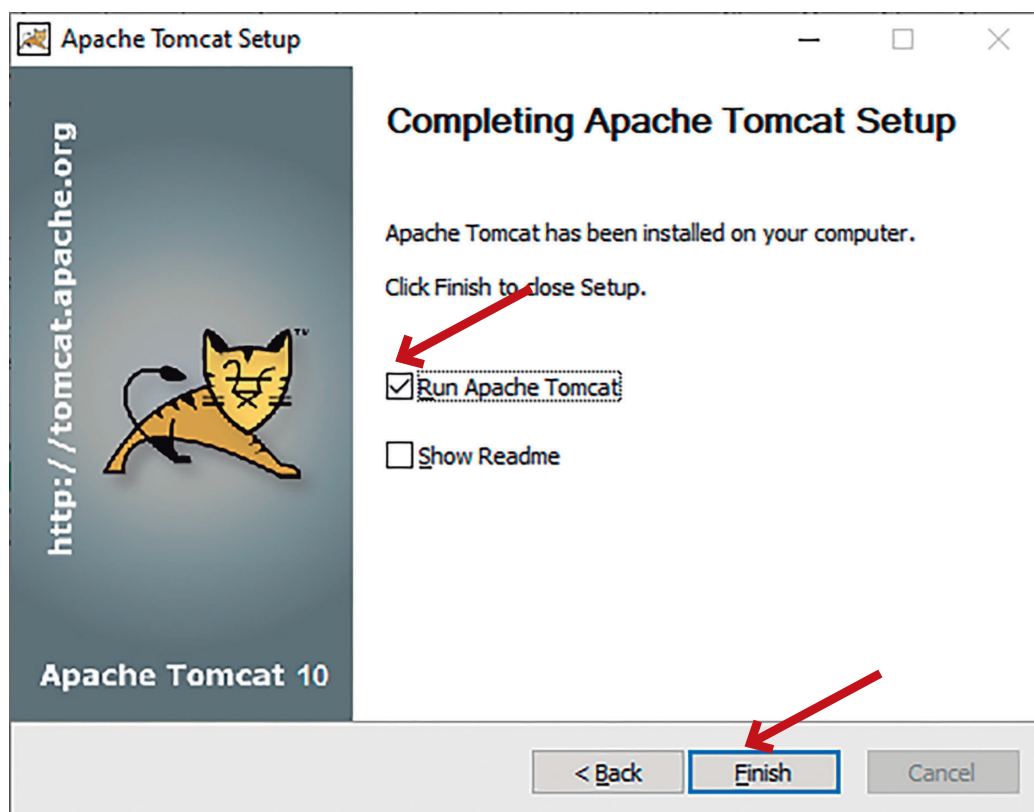


Figura B10 – Última tela do processo de instalação do Apache Tomcat

11) Seleccione:

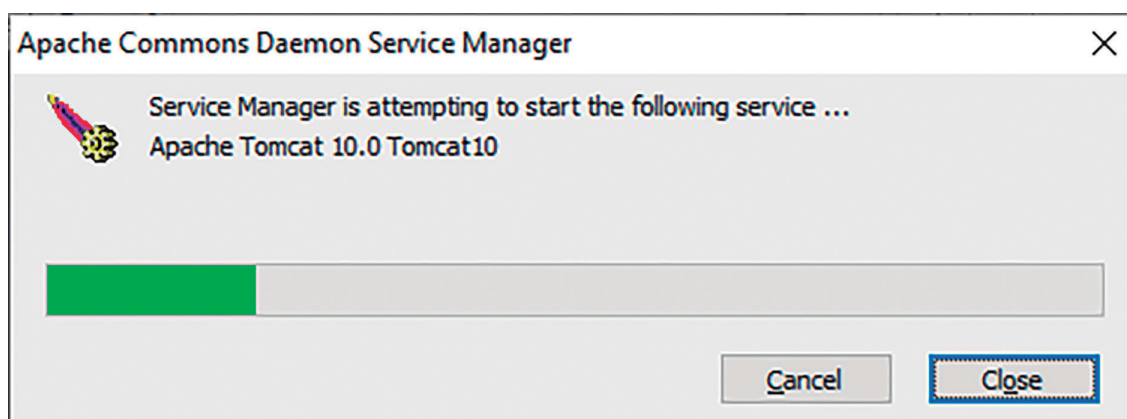


Figura B11 – Tela de progressão de acionamento de serviço, no caso, do serviço do Apache Tomcat

Esta tela se fechará automaticamente após o término da inicialização do serviço.

12) Selecione:

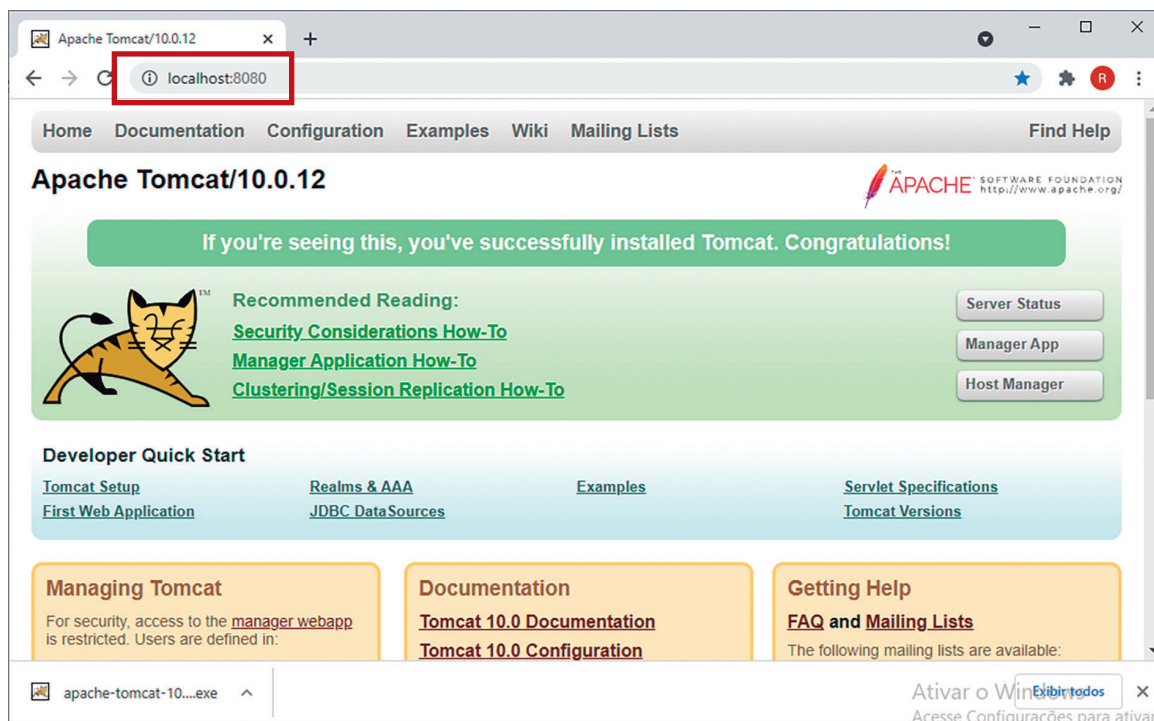


Figura B12 – Tela que mostra que o serviço web do Apache Tomcat está acionado

13) Para acionar (iniciar) diariamente (ou somente quando necessário) o serviço do Apache Tomcat (o servidor de aplicações web), deve-se abrir a relação de serviços do micro, assim como fizemos para acionar o serviço do MySQL, e selecionar o serviço do Apache Tomcat iniciando a sua execução clicando no link "iniciar" que aparece na lateral esquerda da tela, após ter selecionado o serviço "Apache Tomcat". Depois de iniciado o serviço, o status (uma das colunas das informações da tela de serviços) se torna "em execução" (veja a figura B13).

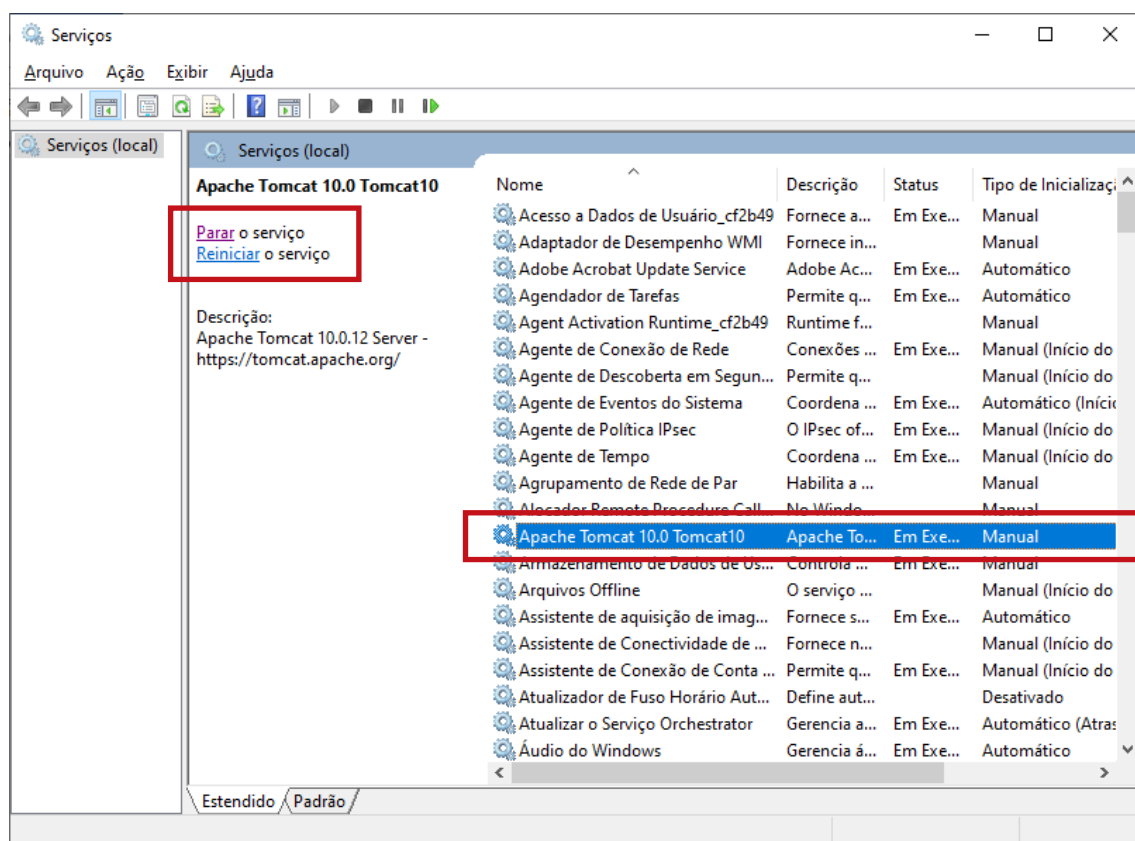


Figura B13 – Tela em que se vê o serviço do Apache Tomcat acionado (em execução)



Informações:
www.sepi.unip.br ou 0800 010 9000